Handwriting robot

Final Report

D.R. Braune
23539195

Submitted as partial fulfilment of the requirements of Project EPR400
in the Department of Electrical, Electronic and Computer Engineering
University of Pretoria

November 2026

Study leader: Prof. H.C. Myburgh

Part 1. Preamble

This report describes the engineering process followed to complete my final year, which involved creating a handwriting robot to classify and reproduce text in author's handwriting.

1. Project proposal and technical documentation
This main report contains an unaltered copy of the approved Project Proposal (as Part 2 of the report).

All the code that I developed appears as a separate submission on the AMS.

2. Project history
The project was done on my own, with advice from an ex-project student, algorithms and techniques inspired by referenced papers, like classifier structure.

3. Language editing
This document has been language edited by a knowledgeable person. By submitting this document in its present form, I declare that this is the written material that I wish to be examined on.

My language editor was _____.

_____ _____

Language editor signature *Date*

4. Plagiarism declaration

I, _____ understand what plagiarism is and have carefully studied the plagiarism policy of the University. I hereby declare that all the work described in this report is my own, except where explicitly indicated otherwise. Although I may have discussed the design and investigation with my study leader, fellow students or consulted various books, articles or the internet, the design/investigative work is my own. I have mastered the design and I have made all the required calculations in my lab book (and/or they are reflected in this report) to authenticate this. I am not presenting a complete solution of someone else or that I found somewhere.

Wherever I have used information from other sources, I have given credit by proper and complete referencing of the source material so that it can be clearly discerned what is my own work and what was quoted from other sources. I acknowledge that failure to comply with the instructions regarding referencing will regarded as plagiarism. If there is any doubt about the authenticity of my work, I understand that I may have to attend an oral ancillary examination/evaluation about the work.

5. AI usage declaration

Artificial intelligence tools assisted with code development, analysis support and report editing.

The author is responsible for checking the submitted work, understanding the implementation and acknowledging assistance in accordance with the University's requirements.

6. Software library usage declaration

The project includes first-principles implementations of the network operations, training and image-processing algorithms described in Design. NumPy provides array operations. Supporting libraries provide image handling, the web interface and hardware access. A PyTorch route provides reference training and comparisons. The evaluated checkpoint provenance is recorded in Appendix C.

7. Unaltered Project Proposal

I certify that the Project Proposal appearing as the Introduction section of the report is an unaltered copy of the approved Project Proposal.

Signature

With my signature below, I hereby certify that all statements, declarations, and information presented above are complete.

_____ _____

D.R. Braune Date

TABLE OF CONTENTS

Part 2. Project definition: approved Project Proposal

1. Project description
2. Technical challenges in this project
3. Functional analysis
4. System requirements and specifications
5. Field conditions
6. Student tasks

Part 3. Main Report

1. Literature study 1
    - 1.1. Handwriting recognition and writer identification 1
        - 1.1.1. Offline recognition and available datasets 1
        - 1.1.2. Image conditioning and text separation 1
        - 1.1.3. Neural networks and feature extraction 2
        - 1.1.4. Training and assessment 4
        - 1.1.5. Techniques for writer identification 4
    - 1.2. Handwriting synthesis in a writer's style 4
    - 1.3. Physical reproduction and pen-path mapping 5
    - 1.4. Application of the literature 6

2. Approach 7
    - 2.1. Mechanical Hardware Design 7
    - 2.2. Page condition and line segmentation 7
    - 2.3. Text and writer classification 8
    - 2.4. Style extraction and handwriting reproduction 8
    - 2.5. Integration and testing 8

3. Design and implementation 9
    - 3.1. Design summary 9
    - 3.2. System block diagram 9
    - 3.3. Hardware Design 11
        - 3.3.1. Plotter designs and mechanics 11
        - 3.3.2. Stepper motor and driver 14
        - 3.3.3. Step resolution, microstepping and calibration 15
        - 3.3.4. Pen up/down mechanism 16
        - 3.3.5. End stop switches and calibration 19
        - 3.3.6. Gcode generation and motion mapping 21
    - 3.4. Page conditioning and line segmentation 24
        - 3.4.1. Lighting normalisation and ink extraction 24
        - 3.4.2. Page orientation and skew correction 25
        - 3.4.3. Line detection and separation 25
        - 3.4.4. Assigning ink to the correct line 27
        - 3.4.5. Preparing the classifier crops 29

3.4.6. Complete processing sequence and output 29
3.4.7. Segmentation verification and limitations 30
3.5. Neural network design, training and classification 32
3.5.1. Training data, labels and evaluation separation 32
3.5.2. Shared convolutional feature extractor 32
3.5.3. Text recognition using sequence modelling and CTC 34
3.5.4. Writer identification and stroke normalisation 36
3.5.5. Training procedure and model selection 37
3.5.6. First principles and framework implementations 37
3.5.7. Network verification and design limitations 38
3.6. Style extraction and handwriting reproduction 39
3.6.1. Choice of reproduction representation 39
3.6.2. Character alignment and glyph extraction 39
3.6.3. Skeleton tracing and pen path construction 40
3.6.4. Glyph selection and library quality 41
3.6.5. Writer style measurement and storage 43
3.6.6. Text layout and connected handwriting synthesis 44
3.6.7. Candidate selection for legibility and writer identity 44
3.6.8. Machine coordinate mapping and G-code generation 46
3.6.9. Reproduction verification and design limitations 47
3.7. System integration and embedded operation 47
3.7.1. Classification and reproduction workflows 47
3.7.2. Embedded execution, interfaces and job control 48
3.7.3. Integration verification and remaining constraints 48

4. Results 49
4.1. Summary of results achieved 49
4.2. Qualification tests 49
4.2.1. Test material, configuration and measurement definitions 49
Qualification test 1: Reproduction of digital text in a trained writer's style (R1)
Objectives of the test or experiment 50
Equipment used 50
Test setup and experimental parameters 50
Steps followed in the test or experiment 50
Results or measurements 50
Observations 55
Statistical analysis 55
Qualification test 2: Reading handwritten content and identifying its writer (R2)
Objectives of the test or experiment 55
Equipment used 55
Test setup and experimental parameters 55
Steps followed in the test or experiment 55
Results or measurements 55
Observations 56
Statistical analysis 57
Qualification test 3: Complete reproduction time on the embedded platform (R3)

Objectives of the test or experiment 57
Equipment used 57
Test setup and experimental parameters 57
Steps followed in the test or experiment 57
Results or measurements 57
Observations 57
Statistical analysis 58
Qualification test 4: Time to decode a photographed page and identify its writer (R4)
Objectives of the test or experiment 58
Equipment used 58
Test setup and experimental parameters 58
Steps followed in the test or experiment 58
Results or measurements 58
Observations 58
Statistical analysis 60
Qualification test 5: Gantry dimensional accuracy and geometric consistency (R5)
Objectives of the test or experiment 60
Equipment used 60
Test setup and experimental parameters 60
Steps followed in the test or experiment 60
Results or measurements 60
Observations 61
Statistical analysis 61
Qualification test 6: Storage and use of multiple trained styles (R6)
Objectives of the test or experiment 61
Equipment used 61
Test setup and experimental parameters 61
Steps followed in the test or experiment 61
Results or measurements 61
Observations 62
Statistical analysis 62
4.2.2. Qualification under the proposal's field conditions 62

5. Discussion 63
5.1. Critical evaluation of the design 63
5.1.1. Interpretation of results 63
5.1.2. Critical evaluation 64
5.1.3. Unsolved problems 65
5.1.4. Strong points of the design 65
5.1.5. Expected failure conditions 65
5.2. Considerations in the design 66
5.2.1. Ergonomics 66
5.2.2. Health and safety 66
5.2.3. Environmental impact 66
5.2.4. Social and legal impact 66
5.2.5. Ethics clearance 67

6. Conclusion 68
6.1. Summary of the work completed 68
6.2. Summary of observations and findings 68
6.3. Contribution 68
6.4. Suggestions for future work 68

7. References 69

A. Supporting implementation calculations 73
A.1. Numerical network derivatives 73
A.2. Optimisers and gradient scaling 74
A.3. Morphology, thinning and path simplification 74
A.4. Profile and trajectory quantities 75
A.5. Detailed training procedures 77
A.6. Expanded numerical calculation checks 77
A.7. Implementation and interface reference tables 78
A.8. Motion-file conversion procedure 80
A.9. Controller pin and supply planning 80
A.10. Worked values from the implemented system 83
A.10.1. Page conditioning and line segmentation 83
A.10.2. Network calculations 83
A.10.3. Glyph extraction and reproduction calculations 84
A.10.4. System integration calculation 85
A.10.5. Hardware calculation detail 86

B. Subsystem visual evidence and independent software checks 87
B.1. Actual capture variation and input resolution 87
B.2. Independent component and seam references 87
B.3. CTC probability and derivative check 88
B.4. Stored profiles, emitted files and physical page 88
B.5. Candidate-scoring configuration and scope 91
B.6. Post-design seam-coefficient sensitivity 91
B.7. Real-input calculation traces 91
B.8. Visual checks of numerical training operations 93
B.9. Stroke-width quantisation in the validation sweep 93
B.10. Glyph coordinates and measured connectedness 95
B.11. Extended figures from the Design section 97

C. Supporting qualification evidence 100
C.1. Evaluated checkpoints and populations 100
C.2. Reader validation breakdown 100
C.3. Physical reproduction by writer 101
C.3.1. Per-line classification rates and the board-to-host projection 101
C.4. New geometric style comparison on physical pages 102
C.5. Archived geometric style study 102
C.6. Timing selection and optimisation scope 104
C.7. Cost, consent and final records 105
C.8. Supporting timing and dimensional figures 105

C.9. Triangle geometry diagnostic 107
C.10. Complete physical-page records 107
C.11. Paired-line scoring record 114
C.12. Complete confirmed reproduction timings 118
C.13. Supplied first-principles training records 119

LIST OF ABBREVIATIONS

ADC	analogue-to-digital converter
BiLSTM	bidirectional long short-term memory
CAD	computer-aided design
CER	character error rate
CNN	convolutional neural network
CPU	central processing unit
CTC	connectionist temporal classification
DC	direct current
GPIO	general-purpose input/output
GPU	graphics processing unit
GT2	2 mm pitch timing-belt profile
IAM	Institute of Informatics and Applied Mathematics handwriting database
I/O	input/output
LSTM	long short-term memory
NEMA	National Electrical Manufacturers Association
NLL	negative log-likelihood
PWM	pulse-width modulation
QT	qualification test
ReLU	rectified linear unit
RNN	recurrent neural network
USB	universal serial bus
VNC	virtual network computing
WER	word error rate

Part 2. Project definition: approved Project Proposal

This section contains the problem identification in the form of the complete approved Project Proposal, unaltered from the final approved version that appears on the AMS.

For use by the Project lecturer	Approved	✓	Revision required	

Feedback

For use by the Project module lecturer only

Approved - the project is clearly delineated.
However, a number of things to pay attention to in especially section 4.

Symbol awarded: B

To be completed by the student

PROJECT PROPOSAL 2026

Project no	HCM4	Revision no	0

Title	Surname	Initials	Student no	Study leader (title, initials, surname)
Mr	**Braune**	**D.R**	23539195	Prof. H.C. Myburgh

Project title (this is the title on the project concept note)

Handwriting robot

Language editor details	Language editor signature
M.T. Braune	M.T. Braune
Student declaration I understand what plagiarism is and that I have to complete my project on my own.	**Study leader declaration** This is a clear and unambiguous description of what is required in this project. Approved for submission (Yes/No)
Student signature	Study leader signature and date 20 April 2026

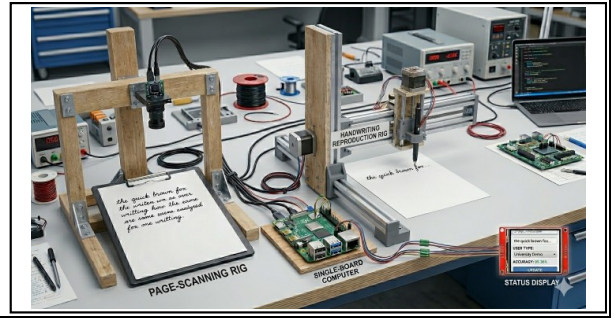
1. Project description.

What is the problem to be solved with your project? Describe in non-technical language what your project is about. What does your system have to do? Refer explictly to Figure 1 that you inserted on the right.

The need for handwriting still exists for authenticity, ease of communication and personal touches. The problem is that many individuals do not possess the time, skills or ability to write neatly and legibly. This leads to the solution presented in this proposal: a handwriting robot capable of recognizing a user's handwriting style and being able to replicate it.

The proposed solution is a handwriting robot capable of training on multiple different user's handwritten text (using a page-scanning rig to capture the handwritten page like the one seen in the image) and being able to produce any desired piece of text in any of these user's writing styles (using a text reproduction rig like the one seen in the image). The system will be able to recognize which user authored a piece of text and identify what text was written (shown to the user through a display as seen in the image). A gantry with 2-dimensional movement will perform actions with a pen to automatically reproduce the desired text.

Figure 1. Insert an image below that captures the idea with your project. Explain in the text on the left.



2. Technical challenges in this project

Describe the technical challenges that are *beyond* those encountered up to the end of third year and in other final year modules.

2.1 Primary *design* challenges

What is the intended core technical challenge in the project? Which aspect or aspects of the design of the system do you expect to be the most challenging?

1. Text image processing will be performed from first principles, including the isolation of letters from an image of various fonts and sizes of handwritten text.
2. A neural network will be designed and trained from first principles to be able to categorize text to different user's styles based on certain style elements, and identify the text written.
3. A gantry rig will be designed to be capable of performing the necessary precise 2-dimensional movements using a pen, whilst holding the pen and paper securely in place.
4. Pen path planning will be implemented, which involves the conversion of written characters to a 2D motion the gantry needs to perform, and then mapping the movement to stepper motor controls.

2.2 Primary *implementation* challenges

Which aspect or aspects of the implementation do you expect to be the most challenging?

1. Image processing and deep learning will be performed on a single board computer (instead of on a personal computer), which has limited memory, storage and power.
2. Smooth motor control will be needed to produce precise and consistent pen movement to create accurate text.
3. Near real-time classification of handwritten text and reproduction of digital text on paper will need to be achieved.

All processing will be done on an embedded platform, and not on a PC platform. Confirmed by having ticked this box. ☑

Library functions will be used for hardware interfacing only. Confirmed by having ticked this box. ☑

3. Functional analysis

3.1 Functional description

Describe the design in terms of system functions as shown on the functional block diagram in section 3.2. This description should be in *narrative format*. **DO NOT** use a bullet list.

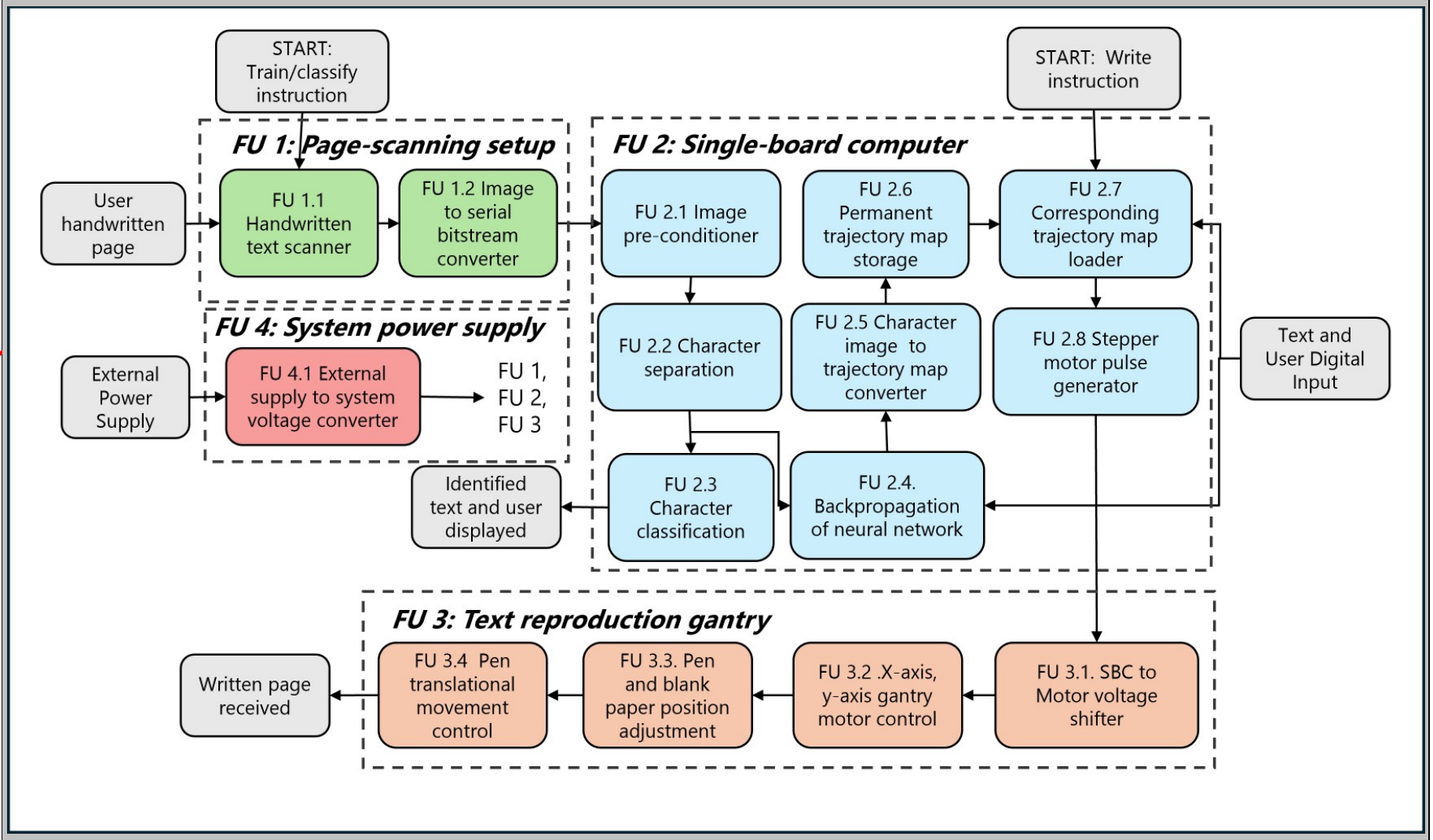
There are three main modes of operation for the handwriting robot: training, classification and reproduction

Training/Classification: First, the user will input their page of handwritten text to the scanner (FU 1.1). The user will select to either train or classify, and will input the intended digital text alongside this through an input interface. Once the page is scanned, a camera USB driver will be used to transfer the image to the single board computer (FU 1.2). The image will be conditioned first to fix warping or noise in the image (FU 2.1), and then the character separation will occur. This will separate the page characters (or groups of characters that are connected) to match the desired neural network input size (FU 2.2). The difference between training and classification occurs here: if the 'training' option is selected, the separated characters will be used to adjust the neural network neuron weightings against the desired digital text that was input (FU 2.4). Each separated character or group of characters will also be converted into a trajectory map (coordinate map of required pen motion) (FU 2.5). This trajectory map will be stored permanently within the system (FU 2.6). If the 'classify' option is selected, the separated characters or group of characters will be passed into the trained neural network classifier (FU 2.3), and the corresponding digital text and detected user will be outputted to the display interface.

Reproduction: If the user sends a 'write' instruction, the digital text and user style desired to be reproduced will be provided. The corresponding trajectory for each character/group will be retrieved from storage (FU 2.7), and will be converted into stepper motor controls required to produce the desired pen movements (FU 2.8). These stepper motor controls will be sent to the corresponding motors via a stepper motor driver to allow for forward and reverse movements (FU 3.1). The x-motor, y-motor and pen lift mechanism will be controlled through the stepper motor driver (FU 3.2/3.4). After the pen has been adjusted and is held securely by the pen holder in the robot (FU 3.3), precise handwritten imitations of each character will be written.

Remove power supply (FBD should shown system functions).
Remove the "start" blocks - this is not a flow diagram, but rather an FBD.

3.2 Functional block diagram (this should not be a flow diagram)



4. System requirements and specifications

These are the core requirements of the system or product (the mission-critical requirements) in table format **IN ORDER OF IMPORTANCE**. Requirement 1 is the most fundamental requirement.

	Requirement 1: the fundamental functional and performance requirement of your project	**Requirement 2** (Second in order of importance)	**Requirement 3** (Third in order of importance)
1. Core mission requirements of the system or product. Focus on requirements that are **core** to solving the engineering problem. These will reflect the solution to the problem.	The system must be able to accurately reproduce any desired digital text on paper in the writing style of any writer from the training set.	The system must be able to accurately decode the content of the handwritten text, and identify which writer from the training set wrote the handwritten text.	The system must reproduce text on paper at a near real time speed, at a rate that can be considered practical and useful.
2. What is the target specification (this must be in *measurable* terms) to be met in order to achieve the requirement in 1. above?	The system should reproduce desired text with a minimum character accuracy of 85% while having a high stylistic correlation of 85% to the chosen writer's style.	The system must be able to identify the writer of the handwritten text from the training set and the content of the written text with an overall accuracy of 95%.	The system must be capable of reproducing the desired text in the writing style of any writer in the training set at a rate of no slower than 4 seconds per character.
3. Explain where this specification originated : i.e., **explain** the value that you selected. I.e., *how* will meeting the specification given in point 2 above *solve the problem*?	An accuracy of 85% (averaged from Babushkin et al. results [1, p8]) is high enough to represent the input text, ~~but practical to account for the build up of precision loss in the system.~~ see class discussion on what specs should be based on.	An overall classification accuracy of 95% (similar to existing classifiers mentioned in Acta Informatica Pragensia [2, p6]) ensures practical usability and consistency.	Averaging the time required to produce text between 0% and 1% error (Ossi Teikari [3, p35]) gives approximately 4 seconds per character, allowing the system to reproduce text at a practical rate with minimal error.
4. How will you demonstrate at the examination that this requirement and specification (points 1 and 2 above) have been met? Be explicit about how you will ***prove*** these were met.	A sentence of text will be reproduced in a specific user style, and the amount of characters that can be visibly seen to match the input text (in the style of the chosen user) must be >= to 85% of the total characters produced. This is not a proper way to measure write in text	A predefined paragraph of text must be handwritten by a writer in the training set and classified by the system. The classified text and writer must match the input text and writer with an overall accuracy of >= 95% .	A sentence of x characters long will be sent to the system to be reproduced, the system must be able to reproduce it within 4*x seconds. this means "convolution"
5. Your own design contribution: what are the aspects that *you will design and implement yourself* to meet the requirement in point 2? If none, *remove this requirement.*	Accurate conversion from character image to pen trajectory map, precise motor control from pen trajectory map, and a structurally sound gantry design for accurate drawing will be designed and implemented by the student.	A rig to capture an image of written text, image conditioning and noise removal, character separation and processing, classification with a neural network, and communicating necessary data to a display will be designed by the student.	Techniques to optimize pen movements for quicker drawing, smooth motor control and robust design for efficient gantry movements will be designed and implemented by the student.
6. What are the aspects to be taken off the shelf to meet this requirement? If none, indicate "none". Clearly specify for what tasks **library functions** will be used (if relevant to the project).	Physical motors, pen, single board computer, stepper motor driver, and separate components for gantry design e.g. screws or belts will be taken off the shelf.	A camera, camera driver (library function), single board computer, separate components for image-capturing rig, and a physical device where data is displayed will be taken off the shelf.	Efficient physical motors, single board computer, stepper motor driver, and separate components for gantry design e.g. smooth ball bearings will be taken off the shelf.

the NUMBER of characters, NOT amount.

System requirements and specifications page 2

	Requirement 4 (Fourth in order of importance)	**Requirement 5**	**Requirement 6**
1. Core mission requirements of the system or product. Focus on requirements that are core to solving the engineering problem. These will reflect the solution to the problem.	The system must be able to identify the writer of the handwritten text, and decode the content of the handwritten text in a short, practical amount of time.	The gantry of the system must have precise pen movements and accurately draw the desired characters with minimal positional error.	The system should be able to be trained on multiple different writing styles and be capable of storing all the writers' necessary character trajectory maps within the system permanently.
2. What is the target specification (this must be in *measurable* terms) to be met in order to achieve the requirement in 1. above?	The system must identify the writer of the written text from the training set and decode the content of the text at a rate of no slower than 25ms per character with an extra 2 seconds for image capture/conditioning.	The gantry of the system must be capable of drawing the desired character trajectory map while maintaining an absolute positional error that does not exceed 0.5mm. 0.5 mm relative to text that is how large?	The system should be capable of training on a minimum of 10 different users' writing styles, and be capable of storing the necessary character mappings for each of these 10 user's permanently.
3. Explain where this specification originated: i.e., explain the value that you selected. I.e., *how* will meeting the specification given in point 2 above solve the problem?	2 seconds was given to image capturing and conditioning due to transmission times and complex conditioning and noise removal algorithms, and 25ms per character classification due to simple forward propagation calculations. same comment as before. Specs are selected to best solve the problem, not because "processing will take long".	According to research by T. Huang and R. Xiong [4] an absolute positional error of 0.3mm with a gantry is achievable and indicates sufficient accuracy, therefore a maximum absolute error of 0.5mm was chosen for slightly more leeway. imcorrect motivation	Having 10 trained user styles stored at the same time proves multi-user neural network recognition and multi-user storage. More than 10 is not required due to neural network complexity and storage limitations. same comment again
4. How will you demonstrate at the examination that this requirement and specification (points 1 and 2 above) have been met? Be explicit about how you will *prove* these were met.	A page with a sentence of x handwritten characters will be given to the system. The time from the initial instruction to the received classified text and user should be within 2 + x*0.025 seconds.	The system should be instructed to recreate the same word twice without varying the dataset characters, and the two words should appear identical on-top of each other, and not exceed an absolute positional error of 0.5mm.	The system should display the current amount of users in the system, and any random user should be selected out of the 10 and have text reproduced in their style to prove the system can be trained with 10 different users.
5. Your own design contribution: what are the aspects that *you will design and implement yourself* to meet the requirement in point 2? If none, *remove this requirement.*	A rig to capture an image of written text, image conditioning,optimized and efficient character separation, neural network character classifier, and calculation and display of time taken will be designed and implemented by the student.	Accurate conversion from pen trajectory map to precise stepper motor control, robust gantry system to write characters, secure pen adjuster and holder, and a secure paper holder will be designed and implemented by the student.	A rig with a camera to capture new text, sufficiently complex neural network to be able to classify minimum 10 user's styles, and a storage strategy to store 10 user's character trajectory mappings will be designed by the student.
6. What are the aspects to be taken off the shelf to meet this requirement? If none, indicate "none". Clearly specify for what tasks **library functions** will be used (if relevant to the project).	A camera, camera driver, single board computer, separate components for image-capturing rig, and a physical device where time is displayed will be taken off the shelf.	Precise stepper motors, separate components needed for gantry, pen and paper holders, and a pen will be taken off the shelf. This show repeatability, not accuracy	A camera, camera driver, single board computer, components for image-capturing rig, and permanent storage device will be taken off the shelf. NUMBER of users. Not amount.

System requirements and specifications page 3

	Requirement 7 (Seventh in the order of importance)	**Requirement 8**	**Requirement 9**
1. Core mission requirements of the system or product. Focus on requirements that are core to solving the engineering problem. These will reflect the solution to the problem.			
2. What is the target specification (this must be in *measurable* terms) to be met in order to achieve the requirement in 1. above?			
3. **Explain where this specification originated**: i.e., explain the value that you selected. I.e., *how* will meeting the specification given in point 2 above solve the problem?			
4. **How will you demonstrate at the examination** that this requirement and specification (points 1 and 2 above) have been met? Be explicit about how you will *prove* these were met.			
5. Your own design contribution: what are the aspects that *you will design and implement yourself* to meet the requirement in point 2? If none, *remove this requirement.*			
6. What are the aspects to be taken off the shelf to meet this requirement? If none, indicate "none". Clearly specify for what tasks l**ibrary functions** will be used (if relevant to the project).			

5. Real world field conditions

These are the ACTUAL, REAL WORLD CONDITIONS under which your project has to work *and will be demonstrated.*

	Real world field condition 1 (required to include)	**Real world field condition 2** (optional to include)	**Real world field condition 3** (optional to include)
Field condition requirement. In which real world conditions does the system have to operate?E.g., "lab conditions" or "outside in uncontrolled conditions". Describe the one, two or three most important field conditions.	The system is expected to perform in well-lit areas like a lab or office environment, at an operating range of 400-1000 lux, to ensure minimal noise in image capturing.	The system is expected to run on a stable, level and vibration free surface, to ensure the page scanning is accurate and the pen in the gantry can write precisely without fluctuations.	

6. Student tasks

6.1 Design and implementation tasks of the student

List your primary design and implementation tasks in bullet list format (5-10 bullets). These are *NOT* product requirements, but *YOUR* tasks.

- The student will research types of neural networks, their structure, the effect of neurons and layers, and how to extract features of text from different layers.
- The student will research image conditioning, noise removal, character separation strategies, and pen trajectory mapping.
- The student will design and implement the convolution neural network, image conditioning, and character separation algorithms.
- The student will simulate the classifier on existing handwritten text datasets with adjusted character separation strategies and neuron/layer numbers to meet requirements.
- The student will construct a rig to capture images of handwritten text on an A4 page, and implement the neural network and algorithms on a chosen single board computer.
- The student will test the new classifier implementations on the single board computer with the captured images to ensure requirements are met.
- The student will design the gantry system in CAD software and construct it with the necessary store-bought individual components.
- The student will design and implement the pen trajectory mapping algorithm and the stepper motor control.
- The student will test the constructed gantry with the implemented classifier to ensure the remainder of the requirements are met.

6.2 New knowledge to be acquired by the student

Describe what the theoretical foundation to the project is, and which *new* knowledge you will acquire (*beyond* that covered in any other undergraduate modules).

- The student will acquire new knowledge of training and classification of character images using a neural network.
- The student will acquire new knowledge involving the extraction of text features from neural network layers.
- The student will acquire new knowledge about the process of image processing with character isolation.
- The student will acquire new knowledge about pen trajectory map creation.
- The student will acquire new knowledge on 2D gantry design and implementation with precise motor control.
- The student will acquire new knowledge about the implementation of neural networks on a single board computer.

7. References

Format your reference list either in IEEE reference format or in APA reference format. Check what your study leader prefers and use consistently.

[1] V. Babushkin, H. Alsuradi, M. O. Al-Khalil, and M. Eid, “Analyzing handwriting legibility through hand kinematics,” Frontiers in Artificial Intelligence, vol. 8, Mar. 2025, doi: https://doi.org/10.3389/frai.2025.1426455.

[2] Sidar Agduk and Emrah Aydemir, “Classification of Handwritten Text Signatures by Person and Gender: A Comparative Study of Transfer Learning Methods,” Acta Informatica Pragensia, vol. 11, no. 3, pp. 324–347, Dec. 2022, doi: https://doi.org/10.18267/j.aip.197.

[3] O. Teikari, "Reproducing Handwriting with Position-Controlled Robots," Master's thesis, Dept. Pervasive Systems, Tampere University of Technology, Tampere, Finland, 2016.

[4] T. Huang and R. Xiong, "Cost-Effective Robotic Handwriting System with AI Integration," in Proc. 2024 IEEE Long Island Systems, Applications and Technology Conference (LISAT), 2024.

Part 3. Main Report

1. Literature study

1.1 Handwriting recognition and writer identification

1.1.1 Offline recognition and available datasets

The ability of a computer or system to identify written text after being trained on large amounts of data is known as HWR (Handwriting Recognition) or HTR (Handwritten Text Recognition) [1, p. 39]. One type of handwritten text recognition is offline recognition, which means the data being identified by the system is static and captured post-completion of the page of handwritten text. This is ideal for identifying the text from a fixed image of handwritten content by utilising image processing techniques and algorithms to identify the text that was written [2, p. 268]. The other type is online recognition, which is a more dynamic process that uses information recorded during the creation of the handwriting. The addition of a time variable includes data about the order of strokes as well as pen-up or pen-down movements [3, p. 18].

This project uses offline recognition because its inputs are photographed or scanned pages. The text and writer must therefore be identified from visible ink geometry. The absence of recorded stroke order also influences how the handwriting can later be reproduced: a path recovered from an image is a geometric estimate, rather than a recording of the writer's movements.

Multiple datasets have been used for training handwritten text recognition modules. The EMNIST (Extended MNIST) By_Class dataset consists of 814 255 handwritten characters and numbers, each being 28 by 28 pixels in size [1, p. 40]. There is only a single character per image, which suits isolated-character classification but does not itself supply examples of connected handwritten words. The IAM handwriting dataset provides complete handwritten English pages, lines and words with their transcriptions and writer metadata [4]. Hemanth et al. report the expanded database as 5 685 labelled sentences and 115 320 labelled words from 657 writers [5, p. 2460].

The dataset influences which variations the classifier encounters and which labels are available. For this project, line images and their transcripts support recognition without manually labelling every character boundary. Writer-labelled pages also support author classification and extraction of individual style profiles. IAM supplies public handwriting variation, while the personal dataset represents the project's photographed input conditions. A small isolated-character dataset alone would not address both requirements.

1.1.2 Image conditioning and text separation

The image conditioning of handwriting pages includes techniques performed on the handwritten text before it is passed into the classification and feature-recognition algorithms. This accounts for dimensional adjustments, removal of inconsistencies and normalisation of the data, to make the images more appropriate for text separation and classification [5, pp. 2458–2460]. Normalisation reduces the influence of unwanted variation; examples include min–max scaling and z-score normalisation [1, pp. 40–41].

Binarisation differentiates the background page from the ink of the written text, computed by pixel. A global threshold taken from the grey-level histogram uses one decision level for the image [6]. A local threshold instead uses statistics from a window around each pixel and can accommodate uneven illumination [7, pp. 2–3]. Deskewing corrects page rotation, while deslanting corrects the lean of individual characters by applying a horizontal shear. These are different transformations: aligning text lines does not require removing the writer's characteristic slant [8, 9].

For the camera input in this project, illumination correction and deskewing are needed before the ink is separated into lines. Slant must remain available as a writer-specific measurement; it can be removed for glyph storage and reapplied when the text is synthesised. The purpose is to suppress capture variation while retaining the information needed for reproduction.

Text separation is the process of breaking up a page of handwritten text into smaller segments or blocks that can be used in feature extraction and neural-network classifiers. Existing methods separate text into lines, words or individual characters. Hemanth et al. obtain lines and words through filtering, dilation and bounding boxes [5, pp. 2459–2460]. When lines are mostly straight and separated by blank rows, boundaries can be found from minima in the row ink profile. Where ascenders and descenders overlap, a boundary that bends between lines is more suitable. Minimum-cost seam methods provide a way to trace such curved boundaries [10, p. 3].

The project therefore separates whole lines before recognition. This avoids requiring a reliable cut between every pair of connected letters. Character boundaries are still needed to construct a glyph library, but there the known transcript and a trained line reader can constrain the extraction. Line segmentation and glyph extraction consequently solve different problems, even though both divide an image.

1.1.3 Neural networks and feature extraction

Many different types of neural-network models have been implemented for handwriting recognition, including artificial neural networks (ANNs), convolutional neural networks (CNNs), recurrent neural networks (RNNs) and combinations of them. ANNs consist of interconnected neurons, weights, hidden layers and biases that learn a mapping from the input to an output. CNNs use local connections and shared kernels to extract spatial patterns with fewer independent parameters than an otherwise comparable fully connected image layer [11, pp. 6–8]. RNNs maintain a state across a sequence, allowing a prediction to depend on neighbouring evidence [2, p. 269].

The convolutional layers of a CNN perform feature extraction by sliding a convolutional kernel over the image. Each local input value is multiplied by a corresponding kernel value, the products are summed, and a bias and activation function produce an output feature value. Reusing the kernel allows the same pattern to be detected at different locations [11, p. 6]. For unit stride and no padding, an N by N input with an f by f kernel returns an $(N-f+1)$ by $(N-f+1)$ feature map. Pooling then reduces spatial resolution and retains selected responses [11, pp. 6–7]. Figure 1 illustrates these operations.

The LeNet-5 model is an established example of this structure. It uses a 32×32 input and seven trainable layers, including convolution, subsampling and fully connected stages; its original output uses radial-basis units [11, pp. 7–8]. A deeper alternative is a residual network, whose

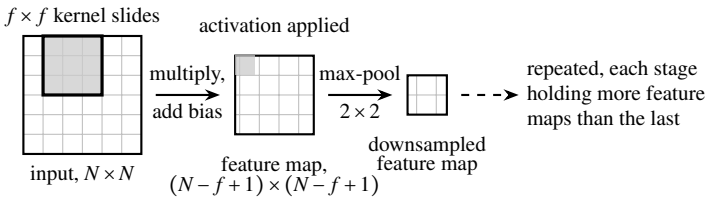


Figure 1. One convolutional layer and one max-pooling layer. The kernel slides over every position of the input, so an $N \times N$ input gives an $(N-f+1) \times (N-f+1)$ feature map, which pooling then downsamples [11, pp. 6–8]. This size expression assumes unit stride and no padding.

skip connections provide additional routes for information and gradients through the network [2, p. 270]. These examples show the choice between compact feature extraction and a more complex model; increasing depth does not guarantee an improvement for every dataset or processor.

A combination of CNNs and recurrent layers allows an entire line to be read without word or character separation. The convolutional layers convert the cropped line into feature columns, and a bidirectional recurrent layer reads the sequence in both directions, providing context from before and after each position [12, pp. 4–5]. In this offline setting, the sequence axis is the horizontal position in the image, rather than the measured time at which a pen moved. The full line crop is available before classification. Related convolutional–recurrent recognition systems use the same distinction between image features and sequence prediction [13, pp. 2–3].

Long sequences make simple recurrent networks difficult to train because gradients can diminish or grow through repeated updates. Long short-term memory (LSTM) uses an additive memory state and gates to control what is retained, replaced and exposed [3, pp. 4–5]. The convolutional feature columns usually outnumber the transcript characters, so their alignment is initially unknown. Connectionist temporal classification (CTC) adds a blank symbol and sums the probabilities of the paths consistent with the transcript. Repeated adjacent symbols are collapsed before blanks are removed [14, pp. 2–4]. Figure 2 shows the resulting recognition path.

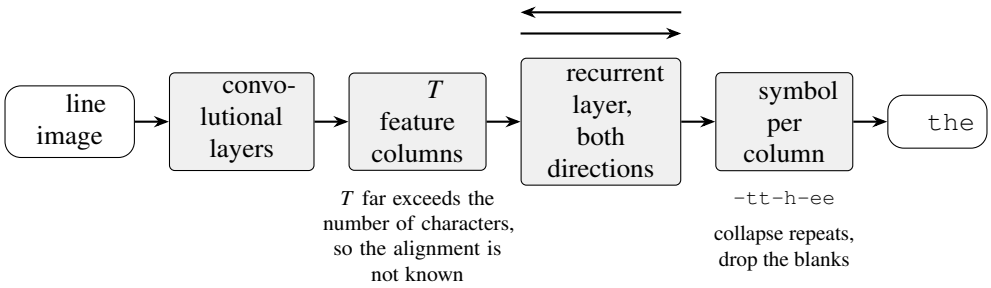


Figure 2. Reading a whole text line without cutting out characters, after the arrangement of Kizilirmak and Yanikoglu [12, pp. 4–5]. The convolutional layers produce T feature columns, a recurrent layer reads them in both directions, and a symbol including a blank is emitted per column; collapsing repeats and removing the blanks gives the transcription.

This combination is used for the project's reader because the training labels describe complete lines. The CNN extracts stroke patterns, the bidirectional LSTM retains their horizontal context, and CTC trains the network without supplied character positions. For author identification, the same convolutional architecture is useful, but its features must describe the writer across a line

rather than decode an ordered transcript. The two tasks therefore require different output heads and separately trained weights.

1.1.4 Training and assessment

The way networks are trained corresponds to the architecture they are built from. Batch normalisation subtracts the batch mean and divides by the standard deviation, with a stabilising constant and learned scale and offset [15, p. 4]. Dropout reduces overfitting by removing a random subset of a layer's outputs during training, limiting dependence on individual features [16, pp. 1931, 1934]. Data augmentation introduces small rotations, shears or smooth distortions while retaining the transcription [17]. These methods address different sources of variation and do not replace separate validation data.

Recognition is commonly assessed through character error rate (CER) or word error rate (WER), using insertions, deletions and substitutions relative to a reference transcript [14, p. 2]. An error in one character can spoil a whole word, but word edit error is not simply a count of words at matching positions. The metric and its reference population must remain explicit when comparing systems.

For this project, numerical forward passes, backward passes and optimiser updates make the learning calculations inspectable. A corresponding library implementation supports training and comparison, with stored arrays used for NumPy inference on the ODROID. The report distinguishes the selected library-trained weights from the independently trained numerical models. Character edits, writer decisions and physical drawing measurements are evaluated separately because they measure different functions.

1.1.5 Techniques for writer identification

There are multiple approaches to identifying the author of a written piece of text. One uses measured, text-independent features such as slant, height, stroke width, spacing and connectivity. Another describes the distribution of edge directions and recurring letter fragments, giving a texture and shape description of the writing [18, p. 701]. Alternatively, a neural network learns discriminative features from examples labelled by writer [19]. Such a classifier identifies a writer among its trained classes; it does not automatically identify an unfamiliar person.

This project needs an author classifier to select and assess the stored writer styles, while explicit geometry measurements are needed to reconstruct their appearance. Ink-trace width can itself contain writer information [20, p. 162], but the gantry uses its own pen. Stroke normalisation is therefore applied to reduce reliance on that cue and direct the classifier towards shape. Writer identity and stylistic fidelity remain separate measures: recognising an author does not establish that every letter or spacing feature has been reproduced accurately.

1.2 Handwriting synthesis in a writer's style

Before handwritten text can be reproduced through hardware, the desired text must be created in the selected author's style. Four approaches illustrate the trade-offs between source data, learned variation and the form of the output.

Recurrent stroke-level synthesis. A recurrent neural network predicts the next pen movement through small coordinate offsets and an end-of-stroke indicator. It models a distribution of possible movements because no letter is written identically every time [3, pp. 18–20]. Its output is naturally suited to a plotter, but training uses online recordings. This project's offline pages contain no measured stroke order, so this approach would require a different source dataset and an additional generative model.

Adversarial image synthesis. A generative adversarial network trains a generator and discriminator together: one generates handwriting, while the other helps distinguish generated samples from real examples. GANwriting also constrains content and writer style to produce text images [21, pp. 3–7]. Once trained, the generator produces an image in a forward pass. This is a relevant alternative for learning contextual letterforms, but a raster output still needs conversion into the pen paths required by this project.

Diffusion-based synthesis. This method learns to reverse a noise-corruption process and generates handwriting through successive denoising steps. One-DM uses one reference style image at generation time, while its training uses a much larger handwriting corpus [22, pp. 3–5, 9]. A small reference requirement must therefore be distinguished from the data and computation required to train the model. For this project, diffusion would add both a separate training task and image-to-path recovery, rather than directly supplying the motion representation.

Glyph-library synthesis. This method assembles words from shapes or glyphs extracted from existing handwritten pages. Haines et al. store labelled forms and select and join them using information about their shape, context and spacing [23, pp. 4–8]. Their standard data collection uses approximately four A4 pages per writer [23, p. 4]. This suits the project's small personal collections and allows explicit control of size, slant and spacing. The main limitation is that a finite collection of character shapes and joining rules only approximates the way connected handwriting changes with its neighbours.

Generated handwriting can be assessed through geometric comparisons, recognition and human judgement. Haines et al. include human evaluation, while image-generation work also uses text and writer recognition to assess the output [23, pp. 12–14], [21, pp. 8–9]. For the present robot, model scores help choose a candidate before drawing, but final physical output also needs its own read-back and geometric assessment. A classifier used to select the candidate is not an independent human judgement of its style.

1.3 Physical reproduction and pen-path mapping

There are three relevant families of mechanism for reproducing text on paper: Cartesian systems, serial-chain robots and parallel-chain robots. A Cartesian system consists of a gantry with independent x–y movement. Serial-chain robots consist of linkages connected in a chain, like a typical robot arm. Parallel mechanisms use several linked chains to support the moving platform [24, pp. 5–8]. These arrangements differ in their workspace, mechanical structure and coordinate transformations.

A Cartesian gantry is suitable here because the page is flat and the main motion requirement is two-dimensional. Each axis can be commanded and calibrated separately, with a further pen-lift mechanism for travel. This makes the conversion from an image-derived path to physical

coordinates direct, while leaving frame alignment, transmission compliance and missed steps as practical accuracy concerns.

A pen path is stored as a list of points, and a recovered stroke usually has more points than a plotter needs. Polyline simplification reduces this list while retaining the stroke's shape. The Douglas–Peucker method joins the endpoints, finds the point furthest from that line and retains it when its distance exceeds a chosen tolerance, then repeats on the resulting parts [25]. A small tolerance follows the stroke closely; a large tolerance removes more points but can cut corners. When handwriting exists only as an image, the path must first be recovered, for example by thinning the ink to a centreline [26, p. 237].

These operations are needed to turn the project's stored glyphs into manageable G-code trajectories. Simplification controls geometric detail, while the thinning and tracing determine which strokes are available to the synthesiser. Neither recovers the original writing speed or pressure from a photograph, so the controller must define its own drawing feed and pen transitions.

There are multiple methods of linear actuation that could be considered for a Cartesian plotting system, including belt drives, rack-and-pinion drives and screw drives [27]. The project uses timing belts and pulleys to convert motor rotation into movement along supported rails. This choice prioritises a straightforward, affordable page-sized mechanism with sufficient travel speed; its nominal step resolution must still be distinguished from measured positioning accuracy.

1.4 Application of the literature

The literature supports a system built around offline pages, complete-line recognition and explicit glyph geometry. Image conditioning and curved line separation prepare photographed input; convolutional features support separate text and writer outputs; transcript-guided extraction turns observed handwriting into reusable paths. A glyph library then supplies controllable geometry without training an additional handwriting generator. The Cartesian gantry provides the physical output stage. Section 2 explains how these choices were combined and developed, while Section 3 gives their calculations, implementation and verification.

2. Approach

The project was approached as two interacting systems: one interprets photographed handwriting and constructs writer-specific paths, while the other converts those paths into physical pen movement. The proposal's functional analysis provides their responsibilities; the G-code interface allows their implementation and verification to proceed separately.

2.1 Mechanical Hardware Design

Before any computer vision or neural networks were worked on, the hardware to perform the pen plotting was created. This is because the hardware does not rely on any of the coding elements to be complete to be able to be tested, and hardware can experience major delays in production due to order delays, stock shortages or component failure, adding to the reason for it to be constructed first. A Cartesian gantry with two independent axes was chosen over some sort of robotic writing arm, as system requiring only flat, cartesian movement does not need joint-dependent kinematics, and each axis can be calibrated separately and work indepedently from one another. To achieve the linear motion required, GT2 timing belts and pulleys were chosen over a lead-screws due to their lower cost, faster travel and ease of customizstion.

Two NEMA 17 stepper motors were chosen for the X and Y axes, driven by A4988 driver chips. Because a stepper motor always rotates by precisely the number of commanded steps at exact intervals, accuracy could be assured without an encoder. To allow for even more increased accuracy and minimal friction, linear bearings were used within the mechanics of the system to allow parts to move with ease and precision, instead of just having the printed supports sliding over the steel rails of the gantry. The use of microswitches on the axes allowed the system to be able to calibrate to reproduce text more accurately. The gantry was designed to be able to reproduce any G-code file that was provided, allowing for easy integration with the other subsystems in future.

2.2 Page condition and line segmentation

In parallel while working on the gantry and waiting for hardware to be delivered, work was done on the first processing stage of the software pipeline, which was performing image conditioning and segmentation of those pages into a format the future classifiers would accept. A photograph produces a wide range of inconsistencies that need to be dealt with, including uneven lighting, page rotation, printed rules, and strokes from neighbouring lines merging into each other. The image conditioning part of the code was designed to remove these undesired page inconsistencies without ruining the quality of the original writing, using methods like lighting normalization, ink separation, and skew correction.

Segmentation of the page of text was chosen to be done lines at a time instead of per word or character, as it was found that dynamic word separation that worked in all cases was difficult, and line separation has the added advantage of more context to train and classify text. The design chosen to perform the segmentation involves locating line centres first, finds curved boundaries between the lines and its ascenders and descenders, and only then uses the geometry of each ink piece to decide which line actually owns it. Deskewing, removal of image background, and artefact rejection were deliberately ordered to run before line centres are estimated, while

recovery of faint ink was left until after ownership of each piece had already been decided, since doing it earlier risked reattaching noise to the wrong line. The system fomatted each page into the ideal crop sizes to be passed into the text and author classifiers.

2.3 Text and writer classification

Once clean line crops were available, the next stage had two jobs: reading the text and identifying who wrote it. Both depend on the same stroke features, so a single convolutional backbone was built, but then the writer and author classifiers varied after that point. The text classifier uses a CNN–BiLSTM architecture with decoded CTC output, which was heavily based off existing research, and chosen because it reads a whole line without needing individual characters pre-cut, as mentioned in the segmentation section, and cursive text has no reliable character boundary to cut at in the first place.

Training data came from a mix of public handwriting (IAM) and the project's own photographed pages, and both author and text models were trained using hold-out pages or lines to allow the system to be validated against, to prove over-fitting did not occur. The writer network's input was stroke-normalised rather than using varying ink thickness from real pages, specifically because the gantry's own pen produces constant stroke thicknesses, regardless of the author.

2.4 Style extraction and handwriting reproduction

Once the previous subsystems were created, the actual text synthesis and reproduction was made. Four reproduction methods were considered, and the method chosen instead was a stored set of normalised glyph trajectories. The other were not chosen because an online dataset of pen strokes was not available, and the chosen IAM dataset was not extensive enough to train a rpeproduction model, neither was the necessary GPU processing power available for it. Using the glyph trajectory method simply resused data and models the project already had, but at the cost of weaker handling of how letters join and vary with context compared to a true generative model especially for heavy cursive styles. The new text to be created is then assembled from the stored glyphs rather than generated from scratch, giving direct, inspectable control over scale and spacing before anything is sent to the gantry. Extracted features like slant, spacing etc. are also applied to the synthesised text.

2.5 Integration and testing

The integration stage formed a complete pipeline of the system through a web server, with G-code remaining the constant fixed interface between the software reproduction output and the gantry hardware. The web interface was designed to be able to preview and inspect generated G-cocdes before any gantry operation was called. By previewing the gantry before reproduction, the precision of the gantry could be measured, and any errors with gantry movement isolated. Safety features were also included as part of the integration, which included a Emergency abort feature on the web server to manually stop any gantry command, as well as end-stop detection with switches to ensure the motors did not stall if the end of the rail was ever reached.

3. Design and implementation

The design follows the hardware, page processing, classification and reproduction stages, then their integration.

3.1 Design summary

This section summarises the project design tasks and how they were implemented (see Table 1). The tasks follow the deliverables and student design work in the approved Project Proposal. The qualification tests in Results assess numerical compliance.

Deliverable or task	**Implementation**	**Completion and report location**
Investigation of methods	Compared recognition, conditioning and reproduction approaches.	Completed; Literature Review; Sections 3.5, 3.6.
Page capture and image conditioning	The camera supplies pages for illumination correction, ink extraction and orientation correction.	Completed; Sections 3.3.1 and 3.4.
Separation of handwritten text	The segmenter separates lines and assigns their ink. Known transcripts then guide glyph localisation.	Completed; Sections 3.4, 3.6.2.
Text and writer classifiers	Separate networks recognise line text and identify the writer from convolutional features.	Completed; Section 3.5.
Network training implementations	The numerical and framework routes train the same network architectures.	Completed; Sections 3.5.5–3.5.7.
Writer-specific trajectory maps and storage	Aligned glyphs are traced, filtered and stored with each writer's style measurements.	Completed; Sections 3.6.2–3.6.5.
Handwriting synthesis and G-code	The synthesiser lays out glyphs, selects a candidate and converts its paths to machine coordinates.	Completed; Sections 3.6.6–3.6.8.
Gantry, pen mechanism and motion control	The gantry combines a two-axis belt drive, switched pen lift, end stops and calibration.	Completed; Section 3.3.
Processing and motion integration	Server connects capture, processing, stored profiles and execution.	Completed; Section 3.7.
Subsystem verification	Numerical checks and offline image and path experiments verify the stages. Results reports the integrated qualification.	Completed; Sections 3.5.7, 3.4.7; Appendix B.

Table 1. Design summary.

3.2 System block diagram

Figure 3 groups the hardware into five parts. External power (1) supplies the ODROID and motor drivers. External interfaces (2) provide page restraint, camera images and operator access. Stepper control (3) converts GPIO commands into winding currents. The belt-driven gantry (4)

moves the pen in x and y. The pen mechanism (5) switches contact with the page and reports its state. Table 21 in the appendix gives the individual connections.

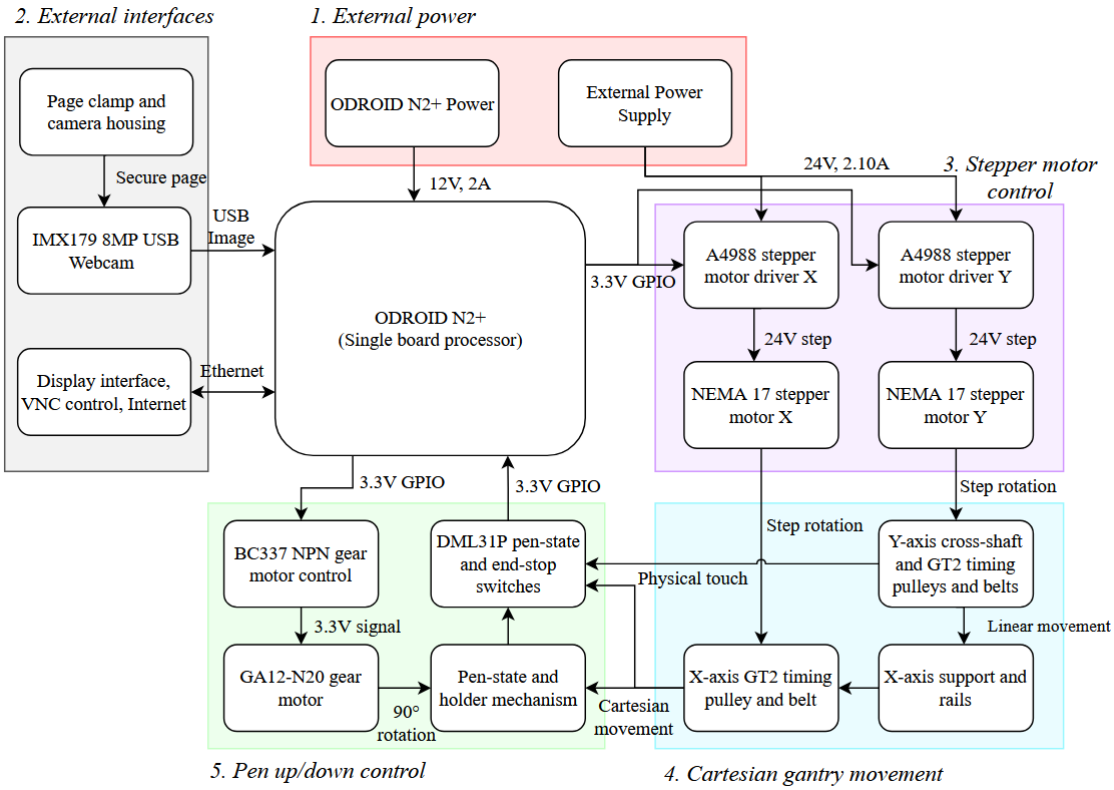


Figure 3. Hardware system block diagram.

Figure 4 shows four software groups. Segmentation (1) conditions a page and returns line crops. Reproduction (2) assembles the selected profile into trajectories and G-code, then executes the path after calibration. Training (3) uses labelled crops to fit the text or writer model and builds stored glyph profiles from labelled handwriting. Classification (4) recognises text or identifies the writer, applying stroke normalisation before writer inference. The diagram groups these functions, while their data dependencies and separate training stages are detailed below.

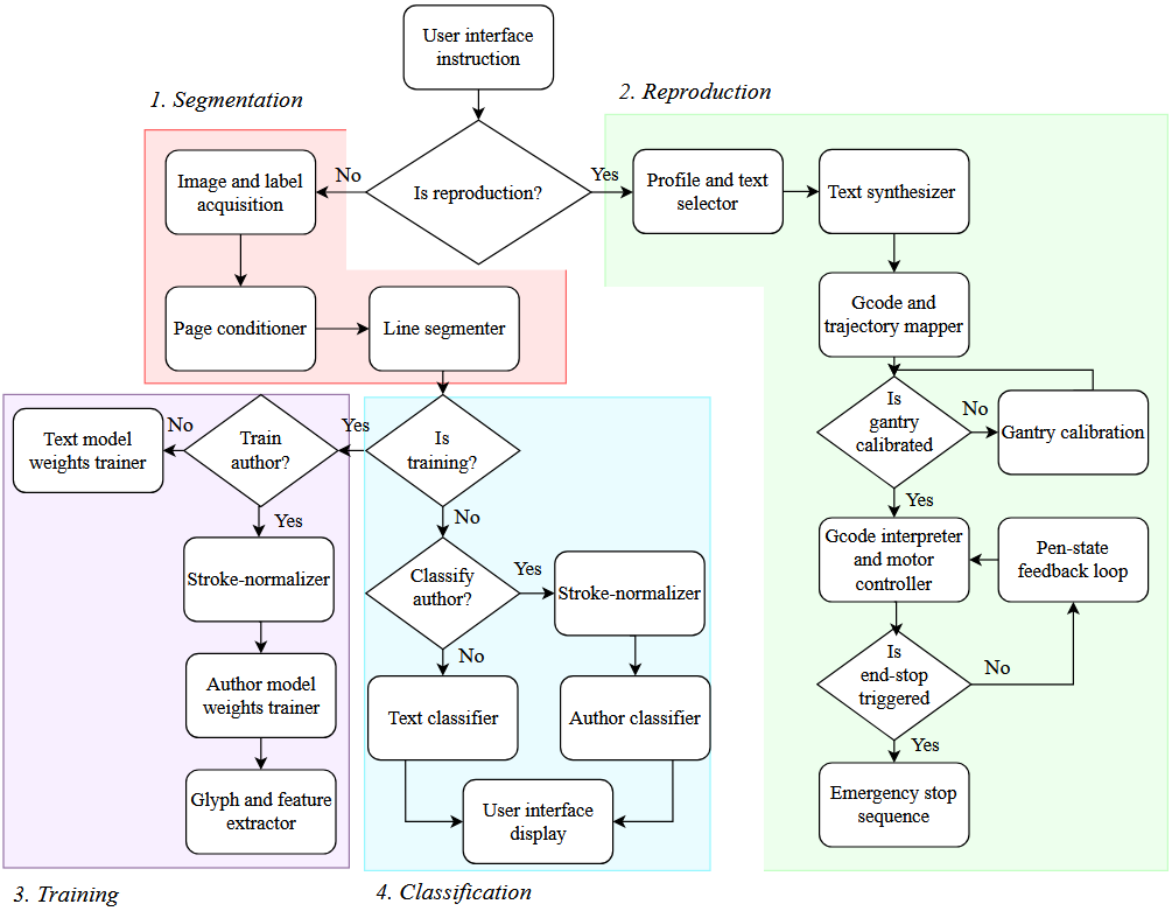


Figure 4. Software system block diagram.

3.3 Hardware Design

3.3.1 Plotter designs and mechanics

The chosen method of text plotting is a Cartesian gantry system with 2 independent axes of movement with a pen up/down mechanicsm. Complex kinematics equations and error dependencies are avoiding by using 2 isolated x and y movement methods for reproduction [24]. Linear GT2 timing belts and pulleys are used to convert rotary motion from stepper motors and axles to the desired cartesian movements. Timing belts are used for their reliability, cost effectiveness, and higher speed travel [27].

A Cartesian gantry consists of a stationary rail that controls movement in one axis, and a movable rail that controls the other axis of movement. For this system and future explanations, the stationary rails control y-axis movement, while the x-axis rail moves along the y-axis. Figure 5 shows the fixed rails guiding *y* movement, the moving *x* rail and the separate pen lift.

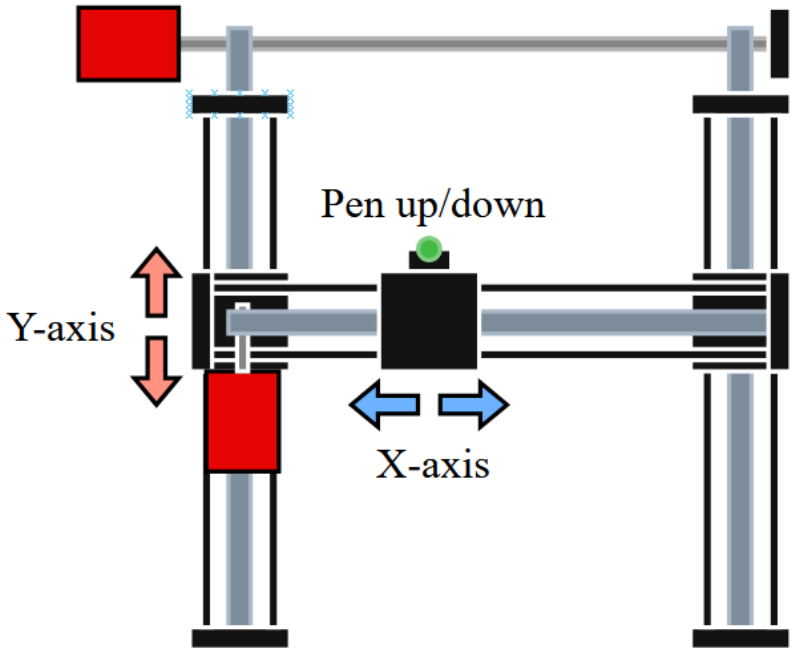


Figure 5. Cartesian gantry axes and pen up/down mechanism.

The x-axis rail needs to move with stability and consistency, which is why it is supported on both ends by dual-rails on the y-axis. A singular stepper motor was chosen to drive both pulleys on both y-axis rails to ensure synchronicity and save costs. 2 independent stepper motors driving one axis creates synchronicity issues because any difference in timing, missed steps or load torques could offset the movement between the two motors. The challenge with using one motor to drive two timing pulleys, is finding a method to extend the shaft of the motor to connect to both timing pulleys without overloading the motor and applying too much tortion or twist to the new axle. A rigid coupler was used to extend the axle of the stepper motor to a chromed steel rod.

To reduce strain on motors, and allow for increased accuracy and precision, linear bearing were fitted around the chromed rod rails to allow all parts to slide with very little friction. To verify the stepper motor could handle the weight of the x-axis rail further, the torque required to move the rail at the desired speed was calculated. It is described in Section 3.3.2 with the descriptions of the stepper motor type chosen and its product specifications.

Figure 6 shows the custom CAD parts. Their shapes locate the pen, drive components and rail bearings while leaving mounting access. Table 2 relates each panel group to its mechanical purpose.

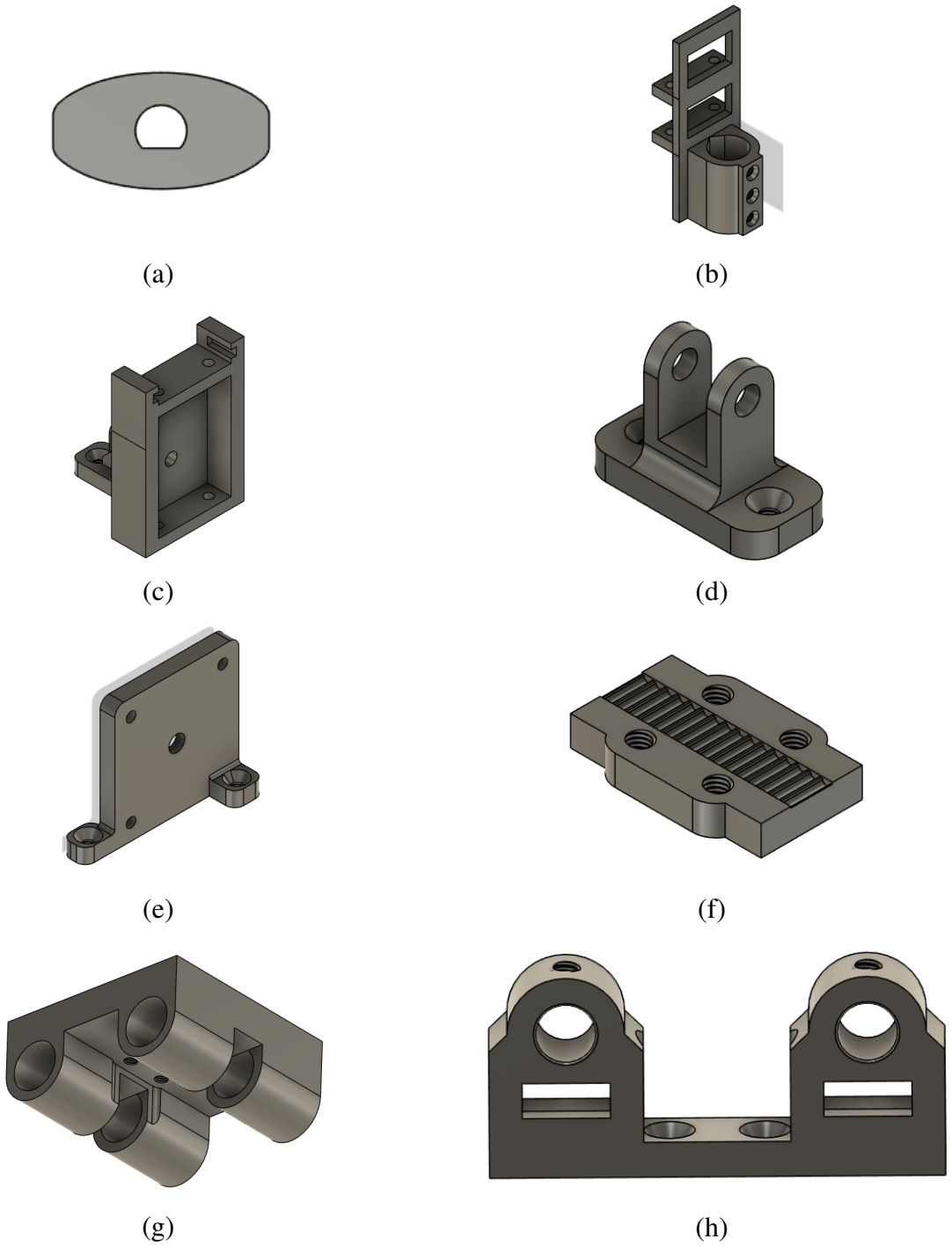


Figure 6. CAD models of the gantry components.

The camera support holds the USB camera over the restrained page so that the image-processing stage receives a consistent view of the writing area. The IMX179 autofocus webcam and page restraint connect to the hardware arrangement shown in the system block diagram.

CAD panels	Mechanical purpose
(a) to (c)	Cam and pen mounting pieces provide lift and hold the pen relative to its actuator.
(d) to (e)	Support brackets locate the rotating drive components and their mounting fasteners.
(f)	The toothed belt grip secures the belt to the moving assembly.
(g) to (h)	Bearing and rod supports locate the rails and guide the moving parts.

Table 2. Functions of the CAD parts.

3.3.2 Stepper motor and driver

A stepper motor does not continuously turn with a PWM input, but rather turns at specific fixed angles each time there is a switch in the windings (the coils of the stepper motor). The A4988 regulates the winding current through a process called chopping, which switches the current supply on the winding until the current rises to a certain point, and then switches off at the target current. This occurs at high frequencies and effectively keeps the current going to the coils within a specific range so as to not overshoot, even when the motor is not actively stepping. A step is created by slighty adjusting the current value between 2 coils just enough to allow the rotors teeth to make a small jump or 'step'. The stepper motor is perfect for the precise movements required for handwriting reproduction with the gantry, as it produces steps at exact, known intervals, with position tracked from commanded steps rather than an encoder. This assumes that the motors do not miss steps.

Two stepper motors are used in the gantry plotter, one for x and one for the y-axis direction. The type of stepper motor chosen was the 42BYGH40-01 Nema 17 bipolar stepper motor, with a resolution of 200 full steps per revolution and a current rating of 1.68A per phase [28]. The following calculations were done to determine if the Nema 17 stepper motor would be strong enough for the use case of the gantry plotter, and if a singular motor could control both y-axis belt drives safely.

Dziedzic et al. examined samples written at 10–60 mm/s [29]. The original sizing check uses the upper value, rounded from 3600 to 4000 mm/min for headroom; this is a design benchmark, not the selected operating feed. Current generated jobs use 900 mm/min for drawing and 2400 mm/min for travel.

For the installed 20-tooth GT2 pulleys with 2 mm belt pitch, the transmission geometry gives [30]

$$\ell_{\text{rev}} = N_t p = 20(2) = 40 \text{ mm/rev}, \qquad r_p = \frac{N_t p}{2\pi} = 6.37 \text{ mm}. \tag{1}$$

Speed-to-torque margin check. The design linear feed is converted to a motor speed and compared with the torque required to move the assembly. The earlier curve readings are retained as a conditional calculation: their applicability to the installed motor and driver settings has not yet been verified.

$$n = \frac{v}{\ell_{\text{rev}}} = \frac{4000 \text{ mm/min}}{40 \text{ mm/rev}} = 100 \text{ RPM} \tag{2}$$

where n is the motor speed in revolutions per minute, $v = 4000$ mm/min is the sizing feed (60 mm/s rounded up for margin), and $\ell_{\text{rev}} = 40$ mm is the belt travel per revolution already established in Equation 1 from the GT2 pulley's pitch ($p = 2$ mm, $N_t = 20$ teeth).

$$T_{\text{po}}(n) = T_1 + \frac{n - n_1}{n_2 - n_1}\left(T_2 - T_1\right) = 7 + \frac{100 - 60}{120 - 60}(21 - 7) \approx 16.3 \text{ Ncm} \tag{3}$$

where $T_{\text{po}}(n)$ is the conditional pull-out torque estimate at speed n, found by linear interpolation between the original two curve readings: $(n_1, T_1) = (60 \text{ RPM}, 7 \text{ Ncm})$ and $(n_2, T_2) = (120 \text{ RPM}, 21 \text{ Ncm})$.

$$T_{\text{req}} = r F_f = r \mu m g = 0.006366 \times 0.004 \times 0.950 \times 9.81 \approx 0.0237 \text{ Ncm} \tag{4}$$

where T_{req} is the torque needed at constant velocity, $r = 6.37$ mm is the pulley pitch radius from Equation 1, $\mu = 0.004$ is the original assumed bearing-friction coefficient, $m = 0.950$ kg is the complete moving assembly and $g = 9.81$ m/s^2. The bearings support the weight, so the motor overcomes horizontal resistance rather than lifting the assembly. The relations $F_f = \mu m g$ and $T = r F_f$ give this steady-speed estimate [30].

$$\frac{T_{\text{po}}}{T_{\text{req}}} = \frac{16.33}{0.02373} \approx 688 \tag{5}$$

Under the assumed friction and original curve values, the estimated torque ratio is about 688. This retains the sizing argument for one motor driving both sides through a shared shaft. It is not a measured operating margin and does not include acceleration, preload or misalignment.

Figure 7 shows the selected motor and its A4988 driver.



Figure 7. NEMA 17 motor and A4988 driver.

To power the two stepper motors, two A4988 stepper motor drivers are used.

3.3.3 Step resolution, microstepping and calibration

Three properties of a stepper drive fix the resolution of this machine. The motors used here take 200 *full steps* per revolution, so one full step is 1.8°; because position is simply the number of steps commanded, no encoder reports where the carriage actually is, which is what *open-loop*

control means, and the design simply trusts the step count. That trust is the assumption on which requirement R5 rests, and it fails silently if a step is ever missed. The driver subdivides each full step by *microstepping*: rather than switching the two windings fully on or off, it holds them at intermediate current ratios, placing the rotor at intermediate angles. At the 1/16 setting used here, one full step becomes 16 microsteps, so a revolution is 3200 microsteps; microstepping buys resolution and smoother motion but not proportionally more accuracy, because the intermediate positions are held by the ratio of two winding currents and are therefore more sensitive to load torque than a full step is.

Equation 6 gives the step resolution from 200 full steps [28], 16 microsteps [31, p. 6] and $\ell_{\text{rev}} = 40\,\text{mm}$,

$$\kappa = \frac{200 \cdot 16}{\ell_{\text{rev}}} = \frac{3200}{40} = 80\ \text{steps/mm}, \qquad \Delta = \frac{1}{\kappa} = 12.5\ \mu\text{m}, \tag{6}$$

which calibration replaces by each axis's measured value. Targets are *absolute* step counts, $\text{round}\big((x+5)\kappa\big)$ with a 5 mm offset, differenced against the tracked position, so rounding cannot accumulate. Quantisation error is at most $\Delta/2 = 6.25\ \mu$m per axis, 8.8 μm combined, 57 times finer than the 0.5 mm of R5 — but that is the *digital* resolution only, and belt stretch, backlash, missed steps, switch repeatability and pen compliance were not measured.

Coil current and its derating. The datasheet relation [31, p. 9] between the trimmer reference, the carrier's two sense resistors and the regulated current gives

$$I_{\text{coil}} = \frac{V_{\text{ref}}}{8 R_S} \quad \Longrightarrow \quad V_{\text{ref}} = 8 R_S I_{\text{coil}} = 8 \cdot 0.1 \cdot 1.68 = 1.34\ \text{V} \tag{7}$$

for the 1.68 A motor rating [28], conditional on the reported R10 sense-resistor marking meaning 0.1 Ω. This gives a rating-based ceiling, rather than the measured adjustment. For comparison, reducing the phase-current limit to 1 A would give $V_{\text{ref}} = 8(0.1)(1) = 0.8$ V. Carrier cooling can require a lower current, but the actual fitted V_{ref} and thermal margin were not recorded.

3.3.4 Pen up/down mechanism

The pen-lift mechanism itself is a design choice justified against the alternative of a third stepper or a servo motor. A servo motor expects a continuous pulse-width-modulated position command and typically needs its own timer peripheral or a precisely timed software loop running concurrently with the gantry's own step timing, competing for the same processor cycles that generate STEP pulses. An N20 geared d.c. motor run in one direction only, with a cam that toggles the pen up or down every 90° of rotation, needs only an on/off drive pulse of roughly known duration and a microswitch to confirm the result; it is cheap, widely available, has high torque at low speed through its internal gearbox for its size, and has a smaller profile than a comparable servo, at the cost of giving no absolute position, only a toggle.

Figure 8 shows the cam holding the pen in its two positions.

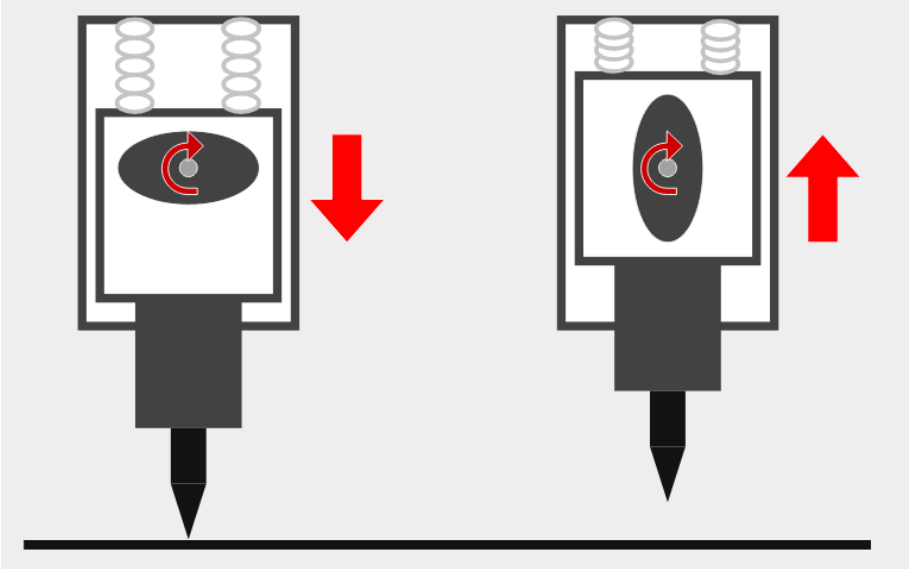


Figure 8. Pen lowered (left) and raised (right).

The implemented lift, planned in the proposal as a stepper, gives only pulses, so the pen state must be tracked in software, and the pen switch is feedback only (high = up). The pen-drive routine of Algorithm 1 must therefore test the switch *before* energising the motor, since the cam turns one way only and an unnecessary pulse would toggle the pen to the wrong state rather than leave it alone. The controller runs the motor until the requested switch state is confirmed or a timeout occurs, with a 40 ms overrun after pen-up confirmation. The generator's separate settling dwell is 350 ms; this is not a measured fixed motor-rotation time.

Pen motor driver and base-resistor selection. The GA12-N20 geared motor was selected to turn the pen cam with a compact, low-speed geared output. The listed 3 V, 150 rpm variant has a specified voltage range of 1.5–5 V, a no-load current of 10 mA, a rated current of 30 mA, and a stall current of 230 mA; these are supplier figures for that variant, rather than measured currents under the installed cam load [32]. The supplied circuit uses the Odroid's 3.3 V GPIO signal to drive the base of a BC337 NPN transistor. The transistor is a low-side switch: motor current flows from the 3.3 V DC motor rail, through the motor and collector-emitter path to the shared ground, while the GPIO supplies only base current [33]. The source symbol in the supplied diagram should therefore be read as a DC supply, not an AC source.

Assuming a base-emitter voltage of approximately 0.7 V, the 1 kΩ base resistor limits the nominal GPIO current to

$$I_B = \frac{V_{\text{GPIO}} - V_{BE}}{R_B} = \frac{3.3 - 0.7}{1000} \approx 2.6\,\text{mA}. \tag{8}$$

The base current is much lower than the motor current. A forced gain of 10, as used in the BC337 saturation test [34], gives a conservative 26 mA collector-current check. This is below the listed 30 mA variant rating, so saturation at startup is not established by the base-resistor calculation alone. The full rated- and stall-current checks are retained in Appendix A.10.

Figure 9 shows the low-side transistor connection. The supplied circuit image does not show a flyback diode. A diode should be connected across the motor, reverse-biased during normal operation, with its cathode toward the positive motor rail and anode at the transistor collector, to provide a path for the inductive current when the transistor switches off [35].

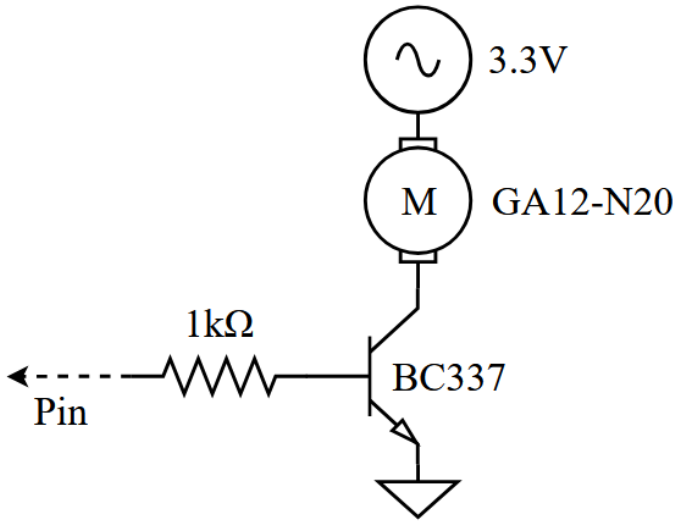


Figure 9. BC337 pen-motor switching circuit.

Algorithm 1 Pen toggle

Require: desired state wantUp
if any end-stop is currently triggered **then**
  refuse the request
else if the pen switch already reads wantUp **then**
  return {the motor must *not* run; the cam turns one way only}
else
  energise the pen motor
  while before the deadline and the pen switch does not yet read wantUp **do**
    wait
  end while
  if the switch now reads up **then**
    wait 40 ms more to seat the cam
  end if
  de-energise the pen motor
end if

During tuning the clicker sometimes over-rotated and the pen did not push down fully before the switch read as triggered, requiring the switch seating and timing to be adjusted experimentally. Differing pen pressure on a book or an exam pad also introduced inconsistent ink, so a rigid clipboard with a single page clamped flat was adopted. The 0.35 s settling dwell contributes to the predicted writing time, but is separate from switch-confirmed actuation. Missed steps cannot be detected from the step counter, so physical accuracy must be assessed from the drawn output rather than nominal resolution alone.

3.3.5 End stop switches and calibration

All four axis end-stop switches are checked before every major-axis step and every G-code line. A potentially triggered switch is accepted only after three consecutive high reads 6 ms apart (Algorithm 2). The pen switch is read separately by the pen-control routine. The confirmation interval is

$$t_{\text{confirm}} = (3-1)(6\,\text{ms}) = 12\,\text{ms}. \tag{9}$$

The stepping loop does not continue pulsing during this check. The interval is therefore not a measured trip-to-stop latency and cannot simply be multiplied by the requested feed to infer stopping distance.

The DML31P was selected as a compact, momentary lever microswitch for detecting gantry end-stop contact; the product listing identifies it as an SPDT switch rated for 1 A at 125 V AC [36]. In this circuit it is used only as a low-current logic contact, not to switch a motor. The diagram uses the common and normally-open contacts so that actuating the switch connects the Odroid N2+'s 3.3 V logic rail to the GPIO input; the source symbol represents DC. The 10 kΩ resistor is a pull-down: it holds the input low while the switch is open, preventing a floating input, and draws a small current when the switch closes. The N2+ GPIO uses 3.3 V logic [33].

With the switch closed, the pull-down current and resistor dissipation are

$$I_{\text{closed}} = \frac{V_{\text{rail}}}{R_{\text{PD}}} = \frac{3.3\,\text{V}}{10\,\text{k}\Omega} = 0.33\,\text{mA}, \qquad P_{\text{PD}} = \frac{V_{\text{rail}}^2}{R_{\text{PD}}} = \frac{(3.3\,\text{V})^2}{10\,\text{k}\Omega} \approx 1.1\,\text{mW}. \tag{10}$$

The 0.33 mA flows from the 3.3 V rail through the switch and resistor to ground; only the GPIO input leakage flows into the pin. When the switch opens, the resistor pulls the input to ground; when it closes, the input rises to approximately 3.3 V. This provides defined logic levels while keeping the closed-contact current low. The GPIO must be configured as an input before this circuit is connected and kept in input mode during operation. If the pin were configured as an output-low, closing the switch would connect the 3.3 V rail directly to the GPIO output driver; the $10k\Omega$ pull-down does not limit that fault current.

Figure 10 shows the switch contact and pull-down that define the GPIO input state.

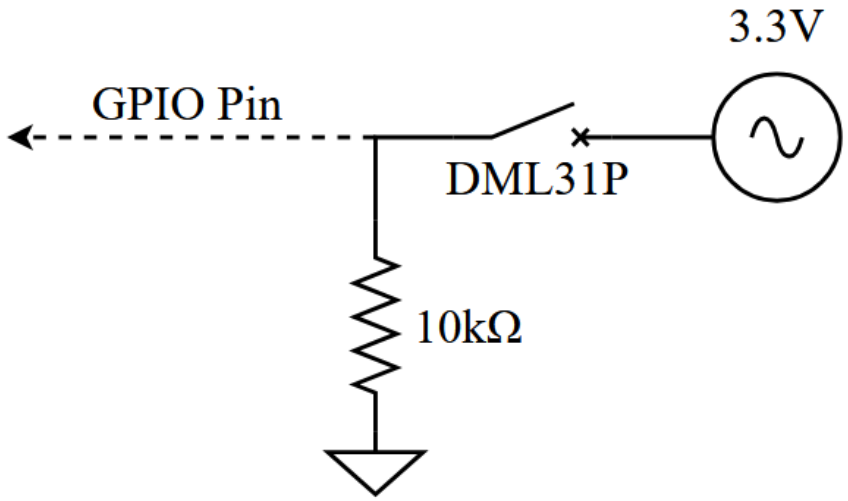


Figure 10. End-stop input and pull-down resistor.

Algorithm 2 Debounced end-stop read, called before every major-axis pulse

Require: set of end-stops to ignore (already triggered on this axis)
for each end-stop not in the ignore set **do**
  if it does not read closed **then**
    continue to the next end-stop
  end if
  re-read it twice more, 6 ms apart {three reads in total, debounce of Equation 9}
  if it read closed on all three reads **then**
    return that end-stop as triggered
  end if
end for
return none triggered

On a trip the program latches a stop, sets both STEP pins low, retracts off every triggered switch and aborts the job; targets are clamped to the calibrated area and the latch clears only by explicit command, refused while a switch is still triggered. Protection is *software only*: no hardware stop that cuts motor power is documented, the switches are polled only during a job, the web stop request can interrupt a running job, but it is not an independent hardware power cut.

Equation 6 rests on assumed transmission values and the position after power-up is unknown, so each session begins with a direction-agnostic calibration, written so that it never needs to know which direction leads to which switch. It drives one way at 1.6 ms per step until *either* switch of that axis triggers, reverses, and counts the steps N to the *other*. With the hand-measured travel L, the resolution, the initial step counter and the position in millimetres are

$$\kappa_{\text{axis}} = \frac{N}{L}, \qquad n_0 = N \text{ if the second switch was MAX else } 0, \qquad x_{\text{mm}} = \frac{n}{\kappa} - 5, \qquad (11)$$

so the zero of Equation 11 lies 5 mm inside the minimum switch, the usable area is 190.6×251.3 mm, and each axis retracts 5 mm off its last switch. Using the nominal 80 microsteps/mm, a 1.6 ms homing-step period corresponds to 7.8125 mm/s. The counted switch-

to-switch traverses alone would then need $(200.6 + 261.3)/7.8125 \approx 59.1$ s; this excludes the initial searches, retractions and parking, and is not a measured calibration duration. The result is deliberately not stored between sessions, since a bump between sessions would silently invalidate it, and a failed calibration leaves movement unavailable until calibration succeeds. Algorithm 3 states the procedure.

Algorithm 3 Direction-agnostic calibration of one axis

drive one way at 1.6 ms per step until *either* switch of this axis triggers
reverse; count steps N until the *other* switch triggers
$\kappa_{\text{axis}} \leftarrow N/L$ {L the hand-measured travel, Equation 11}
if the second switch to trigger was the maximum-side one **then**
    $n_0 \leftarrow N$
else
    $n_0 \leftarrow 0$
end if
retract 5 mm off the switch that is still closed
if an already-triggered switch, an unexpected switch on the other axis, or 200 000 steps pass with no trigger **then**
    abort calibration; refuse all jobs until it is re-run
end if

Travel measurement error also affects the calibrated scale. The numerical allocation of the positional-error allowance is retained in Appendix A.10.

A mechanical switch does not change state cleanly: its contacts strike, separate and strike again over a few milliseconds, so a single read taken during that interval may return either level. Treating one high read as a trip would stop the machine at random, and treating the first read as final would miss a genuine trip. The standard remedy is to *debounce* the input, requiring the same level on several reads spaced far enough apart to outlast the bouncing.

3.3.6 Gcode generation and motion mapping

G-code stores movement and pen-state instructions in a text file. The hardware controller reads each command and its parameters, then converts the target coordinates to motor pulses. The instructions used by this plotter are explained below before the square-and-triangle commissioning example.

Interpolating a straight line. Stepping the two axes independently would draw an L-shaped path, so the program interpolates with Bresenham's integer line algorithm [37, p. 28]: an error term e is carried along the major axis of $a_x = |\Delta x|$ and $a_y = |\Delta y|$ steps, the major axis is pulsed every iteration and the minor axis whenever e would go negative, and the end-stops are read before every major step. Keeping e in $[0, a_x)$ gives the minor-axis count after k major steps as

$$m_k = \left\lceil \frac{k\,a_y - \lfloor a_x/2 \rfloor}{a_x} \right\rceil, \tag{12}$$

which is within half a microstep of the ideal $k\,a_y/a_x$, at most 6.25 µm at the nominal 80 microsteps/mm, using integer arithmetic only. Equation 12 is the reason no floating-point division appears in the inner loop: the deviation from the true line is bounded by construction rather than by rounding behaviour. Algorithm 4 states the loop.

Algorithm 4 Coordinated line movement with an integer error accumulator

Require: signed target changes in microsteps

set axis directions; choose major count N_M and minor count N_m
$e \leftarrow \lfloor N_M/2 \rfloor$
for each of the N_M major-axis steps **do**
  if abort requested or an end-stop is confirmed **then**
    return interrupted
  end if
  pulse major axis; update its count; $e \leftarrow e - N_m$
  if $e < 0$ **then**
    pulse minor axis; update its count; $e \leftarrow e + N_M$
  end if
end for

Figure 11 illustrates the commanded microsteps for a separate generated move of $(+6, -28)$ microsteps. Its maximum deviation from the ideal line is 0.5 microsteps, or $0.5/80 = 0.00625$ mm at the nominal scale. This checks command-space interpolation; it does not measure the pen's physical path.

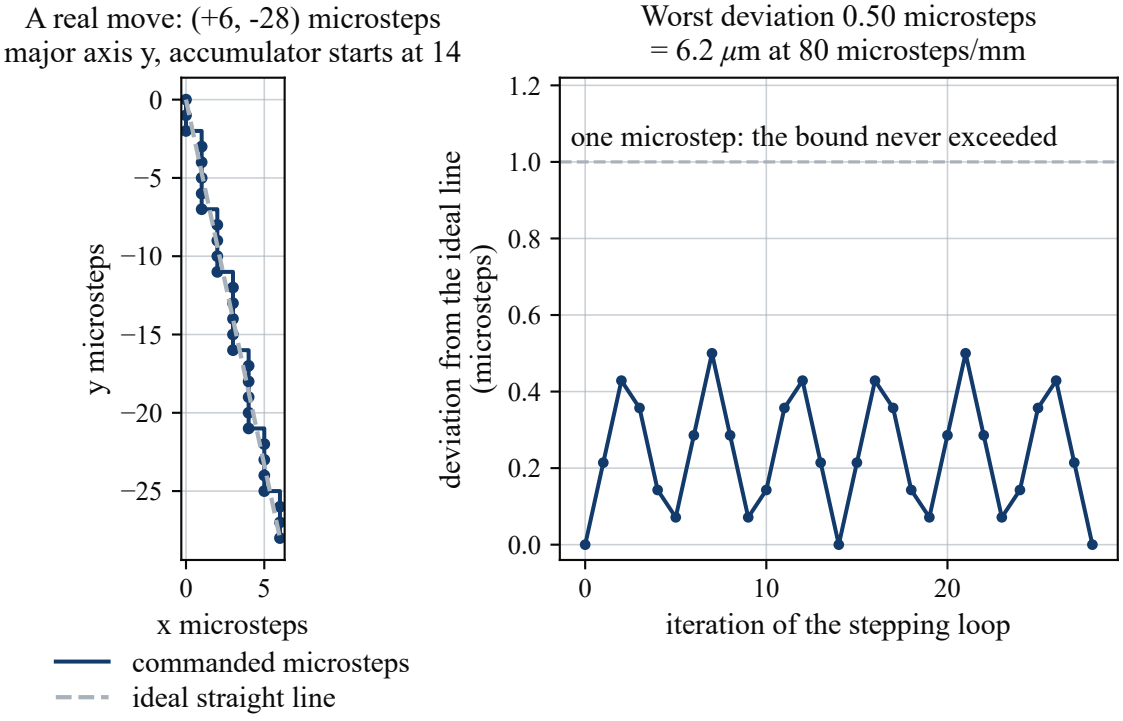


Figure 11. Integer axis stepping and command-space deviation for a generated move.

Equation 13 gives the delay of every pulse for a move of length d at feed v_f over $N_{\text{maj}} = \max(|n_x|, |n_y|, 1)$ major-axis steps,

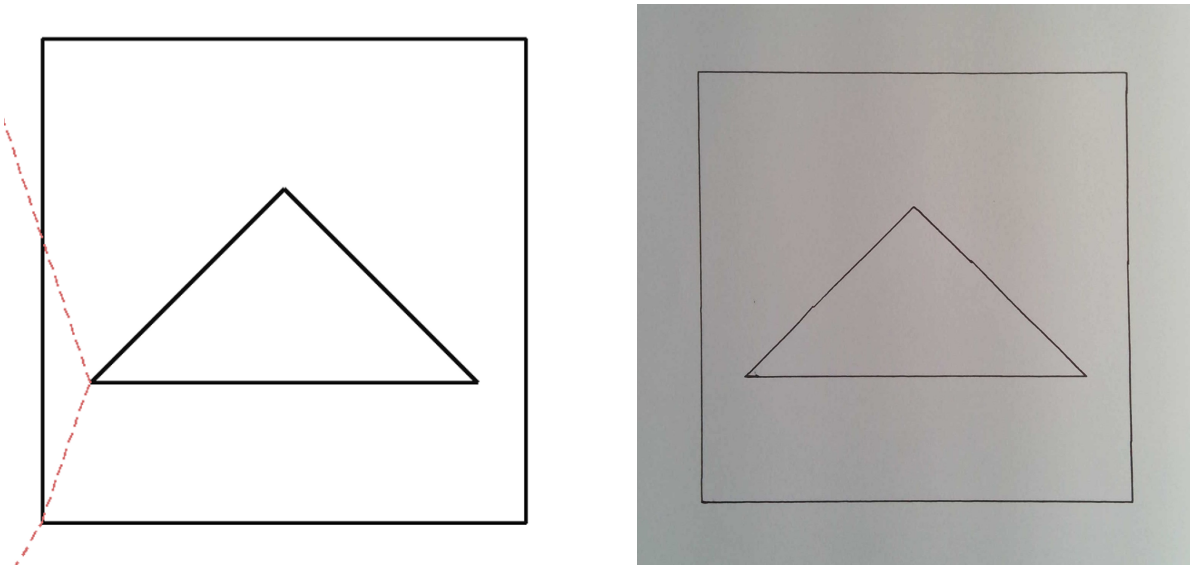
$$\tau = \max\left(\frac{d/v_f}{N_{\text{maj}}},\ 2\,\mu\text{s}\right), \qquad v_f = \max(F/60,\ 0.1), \tag{13}$$

with F in mm/min: 900 gives $\tau = 833$ μs (1200 steps/s), 2400 gives 312.5 µs (3200 steps/s, 60 rpm), homing 625 steps/s or 7.8 mm/s. Because *every* pulse of both axes sleeps for τ, a segment takes $(n_{\text{maj}} + n_{\text{min}})\tau$ and Equation 14 gives the effective feed,

$$v_{\text{eff}} = v_f \frac{n_{\text{maj}}}{n_{\text{maj}} + n_{\text{min}}} \in \left[\frac{v_f}{2}, v_f\right], \tag{14}$$

unity axis-parallel, $\frac{1}{2}$ at 45°. There are no controlled acceleration ramps; the steady-speed torque estimate alone does not establish a safe starting rate [30].

Figure 12 compares the generated path preview with the corresponding physical plot. The test file commands a 100 mm square and an 80 mm-base triangle, providing both horizontal and diagonal movements for visual comparison.



(a) G-code path preview (b) Reproduced drawing

Figure 12. Comparison of the G-code preview and the physical square-and-triangle plot.

The following extract shows the commands used to draw the square; the full file then continues with the triangle before its final `M2`. Coordinates are in millimetres and use absolute positioning; the pen is lowered before the drawing moves and raised after the square is complete.

```
G21
G90
G0 F2400
G0 X45.000 Y76.000
M3
G4 P0.350
G1 F900
G1 X145.000 Y76.000
G1 X145.000 Y176.000
G1 X45.000 Y176.000
G1 X45.000 Y76.000
M5
G4 P0.350
```

Table 3 describes the commands and parameters used in this file. Travel commands do not themselves raise the pen, and the travel feed is restated after drawing because `F` remains active until changed. The meanings of `M3` and `M5` are specific to this plotter firmware, where they operate the pen-lift mechanism rather than a spindle.

Command or parameter	Meaning in this plotter
`G21`	Select millimetres as the units for coordinates and feed rates.
`G90`	Use absolute positioning, so each X/Y coordinate is measured from the machine origin.
`G0`	Move rapidly without drawing, used for positioning and travel with the pen raised.
`G1`	Make a straight, feed-controlled move; with the pen down, this draws a line.
`G4 P`	Dwell for the duration specified by P in seconds; used here to let the pen-lift motor finish moving.
`M3`	Lower the pen using the plotter's one-way pen-lift motor.
`M5`	Raise the pen using the plotter's one-way pen-lift motor.
`M2`	End the G-code program.
`X`, `Y`	Target coordinates in millimetres when used with the active positioning mode.
`F`	Feed rate for movement, in millimetres per minute with `G21` active.
`P`	Dwell duration in seconds when supplied to `G4`.

Table 3. G-code commands and parameters used in the square-and-triangle test file.

3.4 Page conditioning and line segmentation

The recogniser needs one line of text at a time. A photographed page can contain shadows, rotation and strokes extending between lines. This stage prepares the page and returns ordered line crops while preserving the shape and spacing of the writing.

3.4.1 Lighting normalisation and ink extraction

A single brightness threshold can mistake a shadow for ink or lose pale strokes. The design compares each pixel with its local background, following the local comparison used in adaptive thresholding [7, pp. 2–3]. Analysis uses a half-resolution grey image formed by averaging 2×2 blocks. The full-resolution page is retained for the final crops.

Estimating the local background. The background B_r is the mean of a square window of radius r around each grey pixel. The implemented radius is $r = \max(8, \min(40, \operatorname{round}(H/41)))$, where H is working-image height. A summed-area table S gives each window sum from four accesses [7, p. 2].

$$\Sigma_{[y_0,y_1)\times[x_0,x_1)} = S(y_1,x_1) - S(y_0,x_1) - S(y_1,x_0) + S(y_0,x_0). \tag{15}$$

The window area divides this sum to give B_r. Edge replication avoids darkening the image boundary with artificial zero padding. This calculation is independent of window size once the prefix table is available.

Separating ink from paper. Dividing grey intensity G by its local background gives ratio R. Ink has a lower ratio than paper. The binary mask M retains pixels below threshold t^*.

$$R(y,x) = \frac{G(y,x)}{\max[B_r(y,x), 1]}, \qquad M(y,x) = \mathbf{1}[R(y,x) < t^*]. \tag{16}$$

The denominator floor avoids division by zero. Otsu's criterion selects the threshold by maximising the separation between two histogram classes [6]. The class masses are ω_0, ω_1, with mean bin indices μ_0, μ_1.

$$t^* = \arg\max_t \ \omega_0(t)\omega_1(t)\big[\mu_0(t) - \mu_1(t)\big]^2. \tag{17}$$

The software uses 256 bins over the clipped ratio range 0 to 1.2. Applying the threshold to a brightness ratio makes the decision less sensitive to gradual shadows than applying it directly to the grey image.

Calculation on an actual page. The recorded Owen01 example has pixel intensity 100 and local background 161.43, giving $R = 0.62$. Its selected threshold is 0.80, so the pixel is retained as ink. Owen01 provides one traceable example throughout this explanation, rather than an average or a selected worst case. The prefix values and threshold calculation are retained in Appendix A.10.

3.4.2 Page orientation and skew correction

A quarter turn changes the line axis, while a small rotation changes the slope of lines on that axis. The software first compares the fraction of nearly empty rows before and after a 90° rotation. A turn requires a score improvement by a factor of 1.35, avoiding unnecessary rotation when the evidence is weak.

Fine deskewing then tests seventeen angles from −4° to +4° in 0.5° steps. For each angle, $p_\theta(y)$ is the projected row ink count. Concentrating ink into distinct rows increases the squared-profile objective [8].

$$\theta^* = \arg\max_\theta \sum_y p_\theta(y)^2, \qquad p_\theta(y) = \sum_x M_\theta(y,x). \tag{18}$$

The search maps ink coordinates onto candidate rows in a further reduced mask. Only the selected rotation is applied to the full and working grey images, after which the mask is rebuilt. The Owen01 trace selects +1.0°, with the trial scores retained in Appendix A.10.

3.4.3 Line detection and separation

Line centres locate the writing, while curved boundaries separate overlapping ascenders and descenders. A straight horizontal cut can split these strokes even when they belong to different lines.

Removing marks and locating line centres. A preliminary line pitch P, meaning the estimated spacing between text lines, sets the scale for filtering page surrounds and printed rules. Horizontal and vertical run limits are 1.25 and 1.4 pitches. The pitch is estimated again after these marks and artifact components have been removed. A long handwritten underline can also meet a rule-removal condition.

The final smoothed row profile supplies candidate peaks above 6% of its maximum. Median peak spacing gives the preferred pitch, with a lower bound based on component height. Autocorrelation and component height provide fallbacks. Centres are selected by peak strength with at least $0.6P$ separation. Optional shallow-valley merging is disabled by default.

Finding a boundary between lines. A boundary follows a low-cost path across each gap. The local cost combines a strong penalty for crossing ink with a weaker attraction towards midpoint y_m, scaled by half-gap height h_g.

$$C(y,x) = 4\widetilde{M}(y,x) + 0.012\left(\frac{y - y_m}{h_g}\right)^2. \tag{19}$$

Here $\widetilde{M}$ is the mask after morphological closing, a dilation followed by erosion [38, p. 541]. Closing with radius $\max(2, \lfloor 0.07P \rfloor)$ fills narrow breaks that could otherwise let the boundary enter a stroke. The weights are project choices. Dynamic programming then finds the path using accumulated cost D [10, p. 3].

$$D(y,x) = C(y,x) + \min_{|d| \le s}\{D(y+d, x-1) + 0.02|d|\}. \tag{20}$$

The allowed row change is $s = 7$ per column. The $0.02|d|$ term discourages unnecessary vertical movement. Backtracking from the lowest final cost returns the seam. The worked recurrence and six-gap coefficient check are in Appendix A.10 and Appendix B.6.

Figure 13 shows one gap from Abhinav01. It was selected for the largest reduction in crossed ink on that page. The seam crosses one working-image ink pixel, compared with 64 for the straight midpoint cut. The lower panel shows the cost field that makes the seam bend around the writing. The second example and complete original figure are in Appendix B.11.

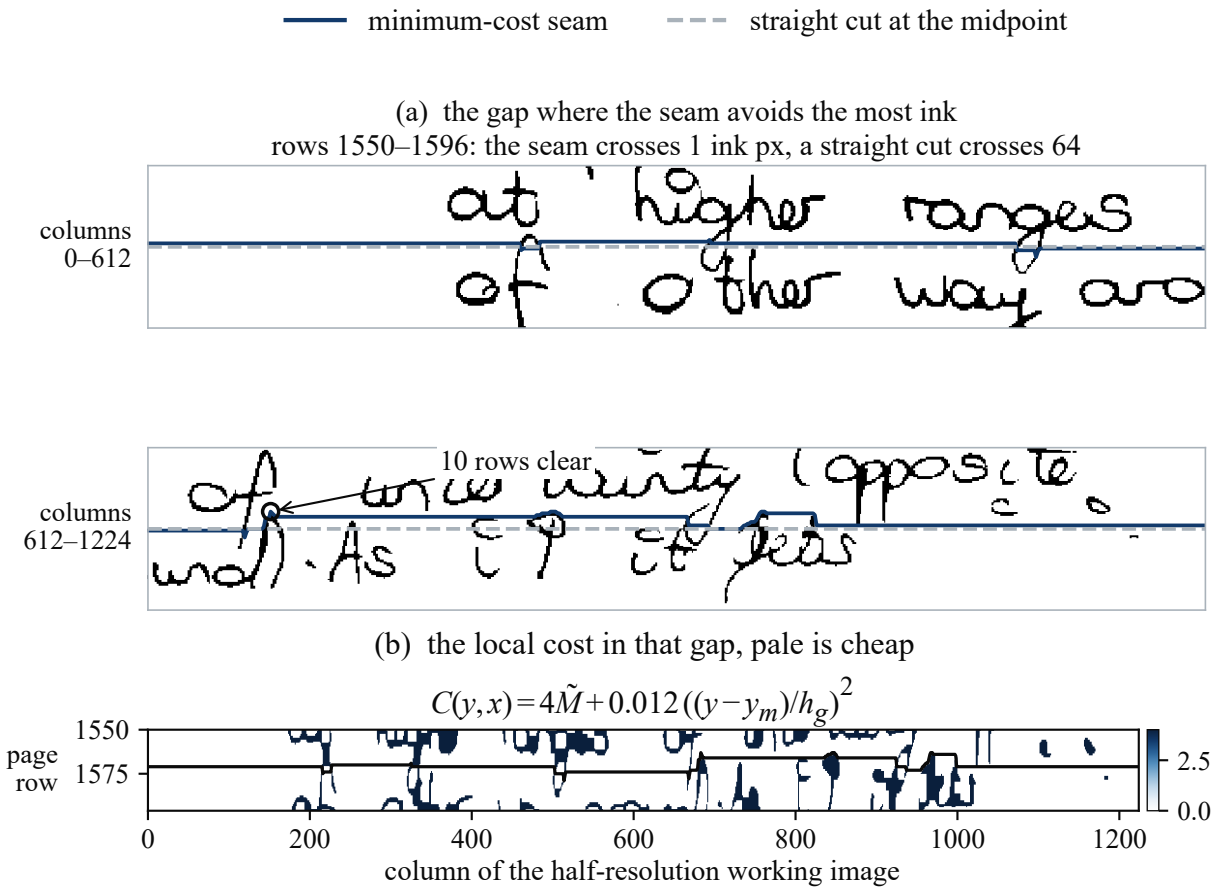


Figure 13. Seam path and its cost field.

3.4.4 Assigning ink to the correct line

A seam divides the page into bands, but a dot or descender can still fall into its neighbour's band. Eight-connected components, found using row runs and union operations, provide a second check [39, pp. 476–479].

For each component, a band containing at least 15% of its in-band ink is substantial. The uppermost substantial band supplies the reference owner for eligible small-piece corrections. If none qualifies, the largest overlap is used. Large components retain their seam assignments except for the narrow-tail and small-piece rules.

For example, an `i` dot may lie above the seam while its stem supports assignment to the lower line. Raw components are revisited because closing may join the dot to an unrelated stroke. Enclosed-ring and residual-fragment checks handle further detached pieces. Algorithm 1 gives this order, with O storing each pixel's line index.

Figure 14 shows the three largest corrections on Abhinav01. The boxes mark the affected pieces and colour shows the assigned line. The complete page has 462 reassigned pixels. This count shows where the rules act, without treating reassignment alone as proof of correct ownership.

Algorithm 1 Refining seam bands into ink ownership

Require: Ink mask M, components $\mathcal{C}$, seams, centres and pitch P.

for each component $C \in \mathcal{C}$ **do**
  Count overlap $a_{C,n}$ with each seam band n
  $S \leftarrow \{n : a_{C,n} \geq 0.15 \sum_j a_{C,j}\}$
  Choose topmost S as reference owner, or maximum overlap if $S = \varnothing$
  Correct eligible narrow tails and small pieces; reject remote specks
  Associate compact dots with plausible stems using pitch-scaled guards
end for
$O \leftarrow$ corrected ownership, preserving large-piece seam assignments
for each raw component near a neighbouring line **do**
  if detached-dot or enclosed-ring guards pass **then**
    Transfer eligible pixels to the supported stem's line; protect correction
  end if
end for
Reassign residual fragments lacking their own body-band support
return O; excluded pixels retain no line index

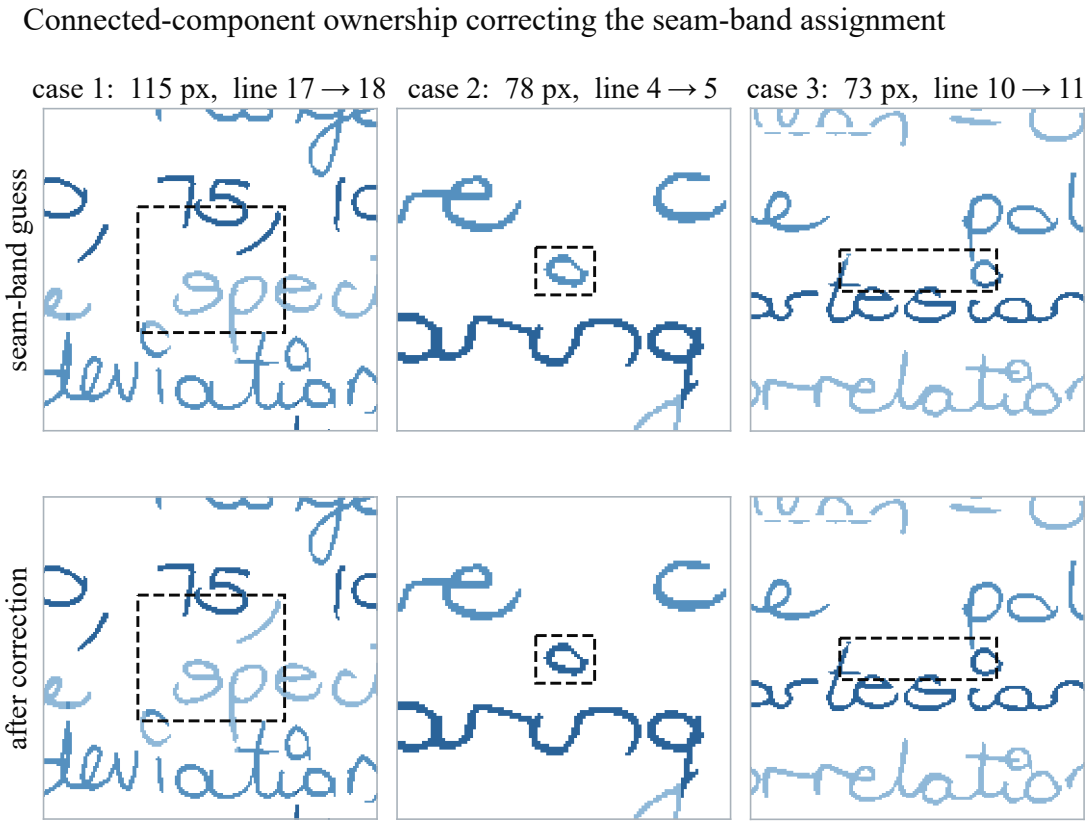


Figure 14. Ink ownership before and after correction.

3.4.5 Preparing the classifier crops

Crops are built from the full-resolution image after ownership is settled. A soft ink mask avoids jagged edges, while a separate nearest-neighbour ownership exclusion keeps neighbouring ink out. Contrast stretching and a darkness exponent of 0.55 strengthen faint retained strokes.

The solid-crop guard rejects a line only when its dark-pixel fraction exceeds both 15% and three times the page median. If every crop would be rejected, the input list is retained. The guard therefore reduces solid-shadow crops without establishing that every remaining crop is valid.

The classifier canvas is 64×640 pixels. A common scale preserves the proportions of a crop of height h and width w.

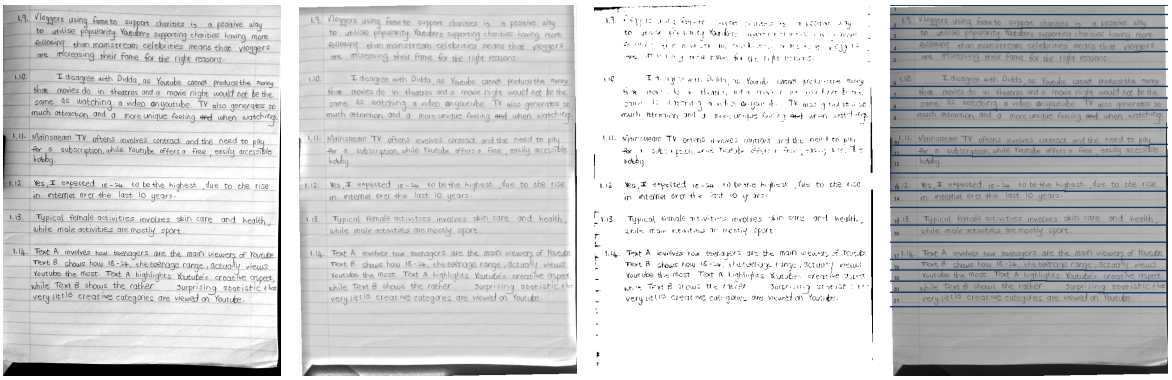
$$a = \min\left(\frac{64}{h}, \frac{640}{w}\right), \qquad (h', w') \simeq (ah, aw). \tag{21}$$

For the archived 156×2170 Owen line, $a \simeq 0.295$ gives a 46×640 image. It is left-aligned, vertically centred and padded white. A very long line loses vertical detail when the width limit controls this scale.

3.4.6 Complete processing sequence and output

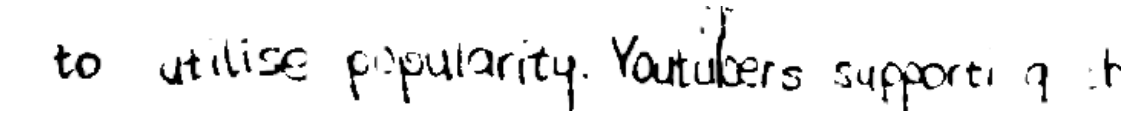
Figure 15 follows the page from lighting correction to ink extraction, line boundaries and final crops. These views show where a stroke was removed or assigned to another line. Algorithm 2 gives the full sequence after the individual operations described above.

(a) Original **(b) Brightness ratio** **(c) Thresholded ink** **(d) Seams found**



Actual pipeline on Owen01: 19 crops; pitch 25 analysis pixels; deskew +1.0 deg.

Output line 2



Output line 3



Figure 15. Page preparation and line extraction.

In the algorithm, G_f, G_h are the full and half-resolution grey images, R is the brightness ratio and M is the ink mask. Pitch P, centres c and seams b define the lines. The output $\mathcal{L}$ contains the ordered crops, classifier views and bounding boxes.

After cropping, the reader compares up to three sampled lines in both directions to distinguish upright from upside-down writing. It chooses the higher mean frame log score, reuses those readings and reverses line order when needed. This heuristic can still fail when the reader is confidently wrong.

3.4.7 Segmentation verification and limitations

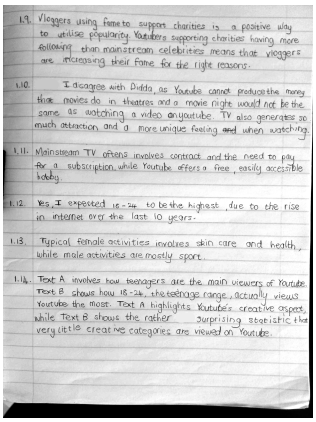
The native Owen01 page produces 20 crops for 20 labelled lines. Figure 16 tests a harder sparse page photographed three times. The striped-shadow image gives three crops, the diffuse-shadow image four, and the changed-angle image 28. In the last case, gap scores of 0.379 and 0.400 give only a 1.054 ratio, below the 1.35 turn threshold. Fine deskewing cannot repair the missed quarter turn.

Algorithm 2 Photographed page to ordered line crops

Require: Grey page on its chosen line axis.
Ensure: Ordered full-resolution crops, coordinates and diagnostics.

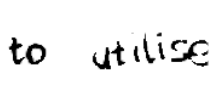
$(G_f, G_h, M, R, \theta) \leftarrow \text{Deskew}(G)$
$M \leftarrow \text{RemovePrintedStructure}(M, G_h)$
$(M, \mathcal{C}) \leftarrow \text{DropArtifactPieces}(M, R)$
$(P, c) \leftarrow \text{LineCentres}(M)$
$b \leftarrow \text{SeamCuts}(M, c, P)$
$O \leftarrow \text{AssignOwnership}(M, \mathcal{C}, b, c, P)$
$\mathcal{L} \leftarrow \varnothing$; normalise full-resolution brightness
for each line index n **do**
    $C_n \leftarrow \text{BuildLineCrop}(n, O, G_f, P)$
    if C_n has sufficient ink and height **then**
        Append C_n, its fixed-canvas view and bounding box to $\mathcal{L}$
    end if
end for
$\mathcal{L} \leftarrow \text{DropSolidCrops}(\mathcal{L})$; renumber retained crops
return $\mathcal{L}$, seams, ratio image and diagnostics

Original photograph

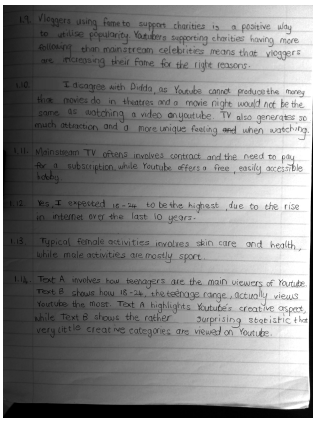


19 crops; deskew +1.0 deg

Start of recovered line 2

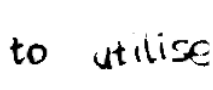


Synthetic gradient

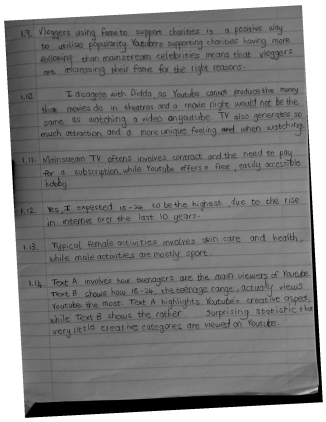


20 crops; deskew +1.0 deg

Start of recovered line 2



Synthetic +3 deg rotation



11 crops; deskew -2.0 deg

Start of recovered line 2



Digital perturbations of one photograph; no measured illumination claim.

Figure 16. Three captures of the same sparse page.

These captures show failure conditions rather than a measured lighting range. Irregular spacing weakens pitch estimation, touching strokes can defeat ownership, and sparse pages provide little

orientation evidence. A fixed mount and restrained page reduce perspective and curvature that a global rotation cannot correct. Appendix B retains the capture records and extended checks.

3.5 Neural network design, training and classification

Two networks identify the text and writer of each line. Both extract stroke features with the same convolutional architecture, but their trained weights and output heads are separate. The text reader preserves horizontal order using a CNN and BiLSTM sequence model [12, pp. 4–5]. The writer head combines features across the line.

3.5.1 Training data, labels and evaluation separation

IAM lines [4] and photographed samples provide transcripts and writer labels. The evaluated set contains six IAM writers (150, 152, 153, 384, 552 and 588) and five personal contributors. The personal pages were written on paper, photographed and paired with line transcripts. The eleven-writer set defines the reported classifier and stored style profiles.

The writer corpus contains 517 IAM and 669 personal lines. One IAM page per writer is held out, while a seeded 15% split selects personal validation lines. This gives 1031 training and 155 validation lines. The final common evaluation contains 156 re-extracted lines, so it is reported separately from training validation. The text reader uses a joint corpus of Teklia pre-cut IAM and personal lines. Its complete historical training split is not retained.

Blank or unusable lines and printed prompts are excluded before pairing crops with labels. Augmentation stays within the training partition. Personal line holdout can share a page or capture session with training. The reader corpus and glyph-profile pages are also separate inputs, so a line held out from one model is not automatically excluded from all project data.

3.5.2 Shared convolutional feature extractor

The input is a batch of grey 64×640 line crops, shaped $(B, 1, 64, 640)$ for batch size B. Intensity scaling to $[-1, 1]$ makes dark ink positive and white padding negative. A learned kernel W and bias b calculate local responses from input X [11, pp. 6–8].

$$z_{o,y,x} = b_o + \sum_c \sum_{i,j} W_{o,c,i,j} X_{c,y+i,x+j}. \tag{22}$$

The sums cover input channels and kernel positions. Input patches are rearranged into a matrix for multiplication by flattened kernels, avoiding a Python loop for every pixel operation [40].

Batch normalisation uses channel mean μ_B and variance σ_B^2 during training, followed by learned scale γ and offset β [15, pp. 4–5].

$$\widehat{z} = \frac{z - \mu_B}{\sqrt{\sigma_B^2 + \varepsilon}}, \qquad y = \gamma \widehat{z} + \beta. \tag{23}$$

The constant ε stabilises division. Inference uses stored running statistics. ReLU applies $\max(0, y)$, retaining positive responses with a simple nonlinear operation. Max pooling retains the strongest response in each local window. Table 4 gives the resulting feature dimensions.

Stage	Operation	Output excluding batch
Input	Aspect-preserving line fit and intensity scaling	$1 \times 64 \times 640$
1	Two 3×3 convolutions, each with batch normalisation and ReLU; 2×2 max pool	$32 \times 32 \times 320$
2	Four convolution/normalisation/ReLU blocks; 2×2 max pool	$64 \times 16 \times 160$
3	Six convolution/normalisation/ReLU blocks	$128 \times 16 \times 160$
Height reduction	Maximum over the remaining height	$128 \times 1 \times 160$

Table 4. Convolutional stages and feature dimensions.

Two spatial pools reduce the width to 160 columns. The final maximum over height leaves 128 features per column. Figure 17 shows the input Owen line and three strong feature channels before the first spatial pool. Their scales are independent, so darkness cannot be compared as an absolute value between channels. The complete layer sequence is retained in Figure 39 in the appendix.

The Owen line on the network canvas and the three strongest stage 1 channels

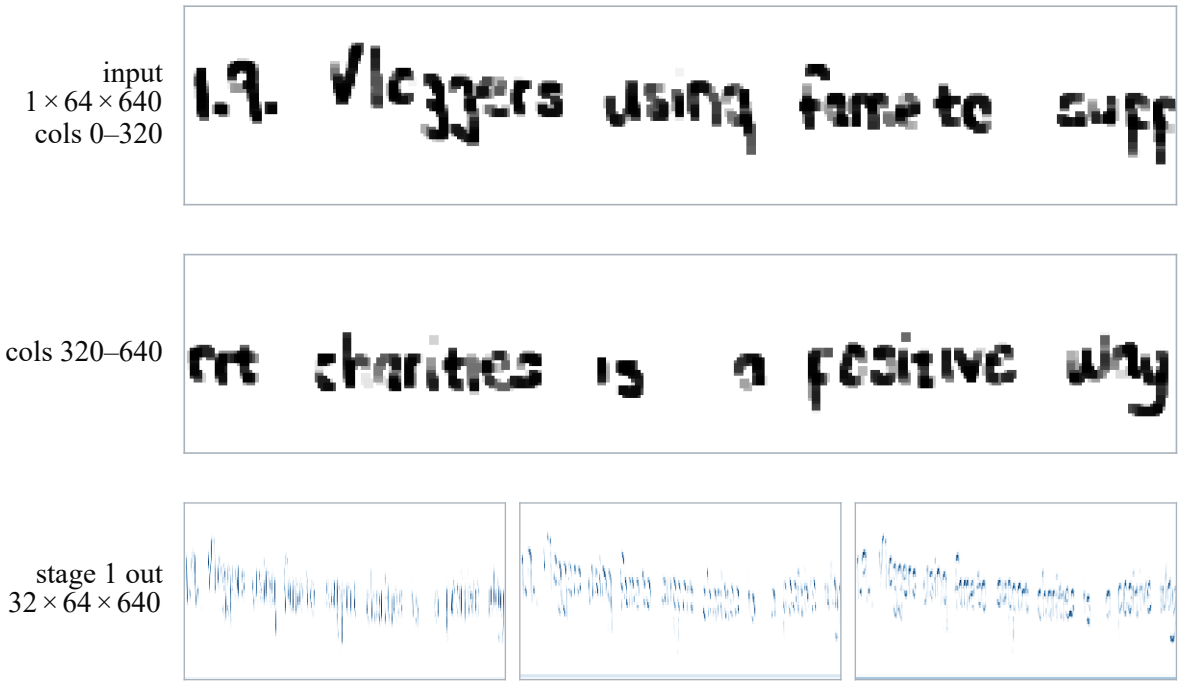


Figure 17. Input line and first convolutional features.

In the worked convolution, the weighted sum and bias give $z \simeq -1.15$. Batch normalisation gives -8.54, which ReLU sets to zero. The patch, learned values and full substitution are retained in Appendix A.10.

3.5.3 *Text recognition using sequence modelling and CTC*

Figure 18 shows the text-reader path. Two bidirectional LSTM layers, each with 256 hidden units per direction, process the 160 feature columns. The first receives 128 features per frame. The second receives 512 after both directions are concatenated. A linear layer produces scores for 74 characters and blank. The writer classifier does not use these recurrent layers.

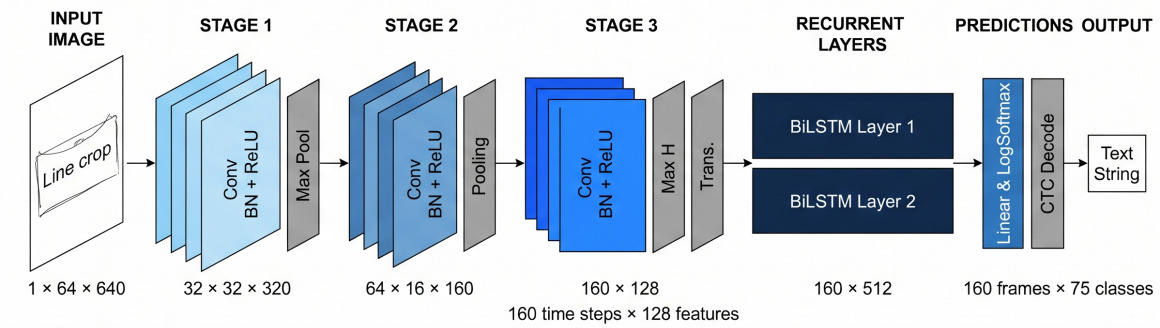


Figure 18. Text-reader architecture.

The LSTM cell state c_t carries memory between frames. Input, forget and output gates are i_t, f_t, o_t, with candidate update g_t and hidden output h_t [3, p. 5].

$$c_t = f_t \odot c_{t-1} + i_t \odot g_t, \qquad h_t = o_t \odot \tanh(c_t). \tag{24}$$

The gates depend on the current feature vector and previous hidden state. Sigmoid $\sigma(a) = 1/(1+e^{-a})$ bounds the gates between zero and one, while tanh bounds the candidate update. The reverse direction supplies context from the other end of the line. The implementation omits cell-to-gate peephole connections.

CTC trains from a line transcript without labelled character positions [14, pp. 2–3]. It sums frame paths π whose collapse $\mathcal{B}$ produces transcript ℓ, using probabilities p_t over $T = 160$ frames.

$$\mathcal{L}_{\mathrm{CTC}} = -\log \sum_{\pi:\mathcal{B}(\pi)=\ell} \prod_{t=1}^{T} p_t(\pi_t). \tag{25}$$

Collapse merges consecutive repeats before removing blanks. Thus $a, a, \varnothing, b, b$ becomes “ab”, while $a, \varnothing, a$ gives “aa”. The dynamic programme uses blank-interleaved target states and log probabilities. For L labels and r adjacent repeated pairs, at least $L + r$ frames are required.

Greedy inference selects the most probable class at each frame and applies that collapse rule. No language model or lexicon is used. Figure 19 compares greedy decoding with alignment constrained to the known Owen transcript. This line is a worked example containing adjacent gs, which require separated emission runs. The greedy result matches its transcript and blank carries about 46% of mean frame probability.

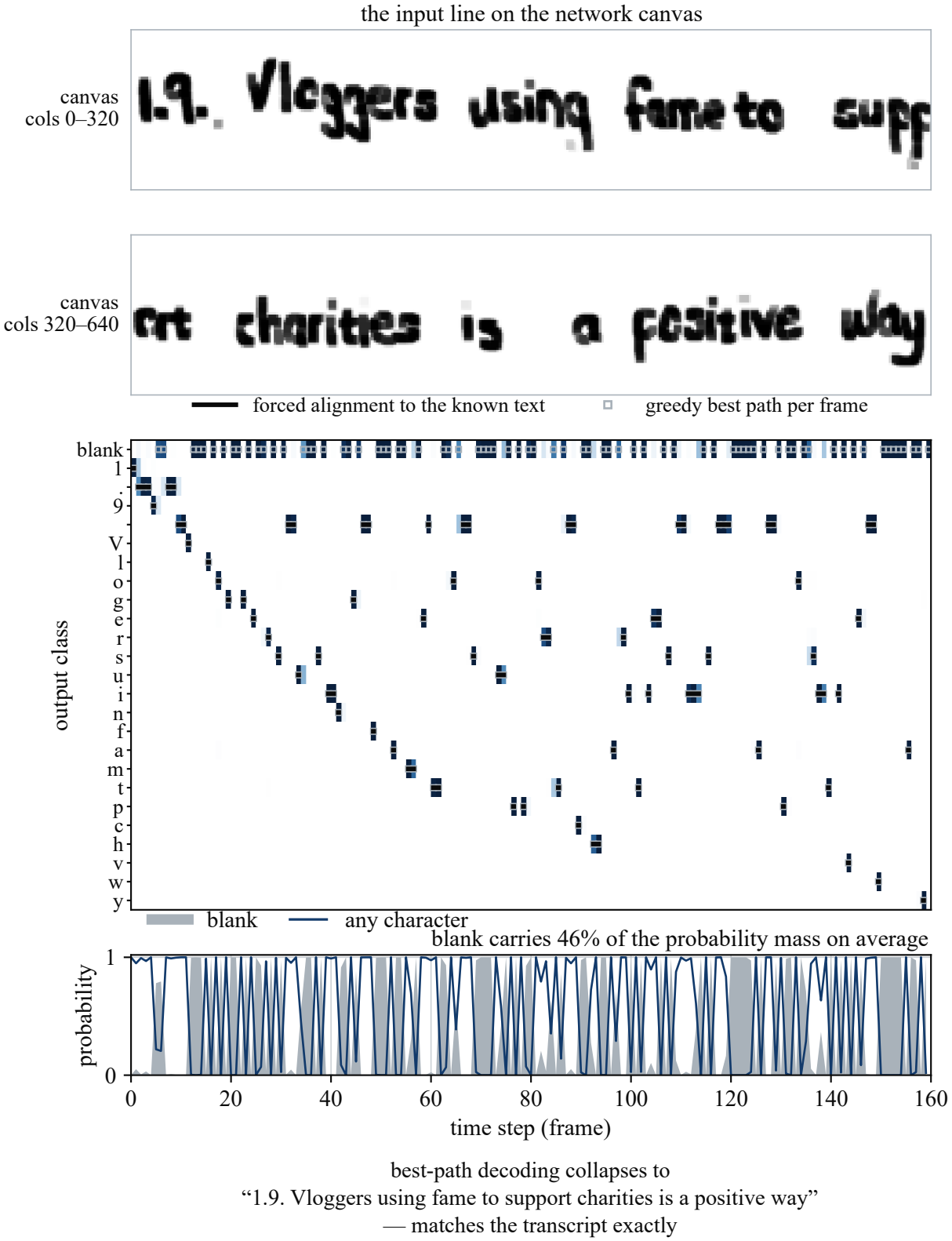


Figure 19. CTC decoding and forced alignment.

The numerical backward pass uses target-state occupancy $q_{t,k}$, total path probability Z and logits $z_{t,k}$. Forward score α includes the current emission, while backward score β includes only later emissions [14, pp. 4–6].

$$q_{t,k} = \sum_{s:\ell'_s=k} \exp(\alpha_{t,s} + \beta_{t,s} - \log Z), \qquad \frac{\partial \mathcal{L}}{\partial z_{t,k}} = p_t(k) - q_{t,k}. \tag{26}$$

The batch average is applied without target-length division in the numerical trainer. Appendix A.5 gives the full recurrence, and Appendix A.10 retains the recorded state and derivative calculations.

3.5.4 Writer identification and stroke normalisation

The writer head averages the 160 feature columns into 128 values. A 96-unit hidden layer, ReLU and a K-class output identify the writer. Dropout probability 0.2 is active only during training [16, pp. 1931, 1934].

The first classifier used the photographed ink directly. Ink width is a useful writer cue [20, p. 162], but the gantry uses its own pen. The later classifier therefore uses normalised strokes so that original and reproduced writing are compared more through their geometry. Binarisation, Zhang–Suen thinning and square dilation form that input [26, p. 237], [38, p. 533].

$$I_s = 255\left[1 - \delta_k\big(\text{thin}(M)\big)\right], \quad w = \text{clip}\big(\text{round}(\rho h_x), 2, 9\big), \quad k = 2\lfloor w/2 \rfloor + 1. \tag{27}$$

Here h_x is estimated x-height, ρ is the width fraction and k is an odd dilation-kernel size. The selected fraction is 0.17. Figure 20 shows Owen and Yeukita to illustrate different letter shapes under the same preparation. The original ink is reduced to a centreline before a controlled width is restored. Thresholding or thinning can still damage faint and touching strokes.

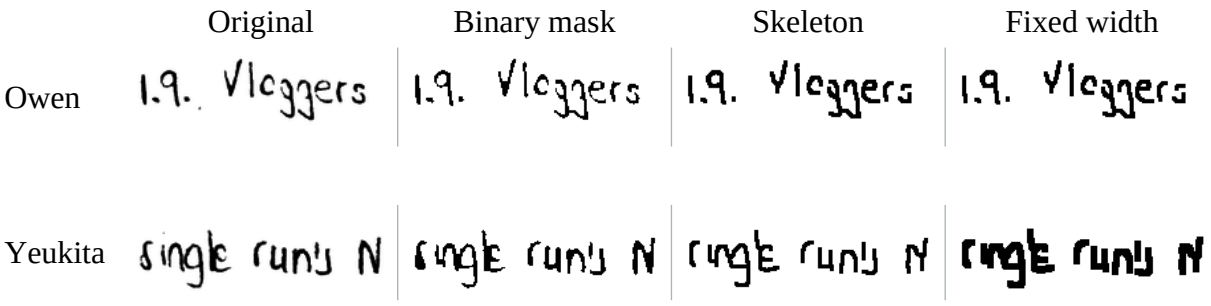


Figure 20. Stroke normalisation on two writers.

The supplied sweep uses the same eleven writers and held-out partition, with a 50-epoch budget for each width. Table 5 gives the best saved epoch and validation score.

The best observed width was 0.17. Preparation and inference use that stored setting rather than the 0.14 module default. Odd integer kernels mean that nearby fractions can give the same image on one crop, as shown in Appendix B.9. The sweep selects preprocessing and is kept separate from final qualification.

Width fraction ρ	Best saved epoch	Held-out writer accuracy
0.06	38	98.06%
0.09	43	97.42%
0.11	39	98.06%
0.17	43	99.35%

Table 5. Stroke-width selection using the same eleven-writer population and held-out partition.

3.5.5 Training procedure and model selection

The backbone and task head are updated together. Training uses variance-scaled convolution weights and initial normalisation scales of one with zero offsets. Shear, small rotations, perspective changes and elastic distortion vary the training images while preserving labels. Validation uses a fixed preparation path without augmentation or dropout.

The reader minimises CTC loss. The writer minimises cross-entropy $-\log p_y$ for true writer y, with logit derivative $p_k - \mathbf{1}[k = y]$ [3, p. 6]. Table 6 records implemented numerical defaults, rather than settings recovered for every historical run.

Setting	Reader	Writer classifier
Objective	CTC loss	Cross-entropy
Optimiser	RMSprop	AdamW
Initial learning rate	10^{-3} default	10^{-3} default
Weight decay	10^{-5}	10^{-4}
Learning-rate schedule	Plateau reduction, factor 0.5, patience 5, minimum 10^{-5}	Cosine annealing over the configured run
Gradient-norm clipping	Not applied by the current NumPy text loop	Maximum norm 5
Best-checkpoint criterion	Lowest validation loss	Highest validation accuracy

Table 6. Training settings implemented in the first-principles trainer. The table describes implemented defaults rather than a recovered record of every completed training run.

Joint reader training supports controlled personal-sample representation, with default fraction 0.20. Reader checkpoints are selected by validation loss and writer checkpoints by validation accuracy. Forward caches retain the quantities needed for differentiation. Algorithm 6 in Appendix A.5 gives the update procedure. The supplied first-principles training records are retained in Appendix C.13.

3.5.6 First principles and framework implementations

The numerical route implements forward operations, backward operations, losses and optimiser updates using NumPy array arithmetic. The framework route expresses the corresponding architecture through PyTorch layers and automatic differentiation. Both use the project's preparation and named-array format, which the numerical inference path reads.

Table 7 compares trained models on common original validation material. The reader scores differ by 0.33 percentage points and writer scores by 2.57 points. These comparisons support the first-principles implementation without treating separately trained models as identical. Training logs and checkpoint identities are retained in the appendix.

Common validation material	Library route / %	First principles / %
Reader, public IAM lines	94.12	94.39
Reader, photographed lines	93.33	94.42
Reader, pooled character accuracy	94.06	94.39
Writer, 156 original lines	98.08	95.51

Table 7. Training-route comparison on common data.

The two CTC training reductions also differ. The numerical route averages sequence losses across the batch. The framework default divides by target length before averaging and zeroes impossible-alignment losses [41]. Controlled training comparisons must match these choices. Table 18 records the operations and data interfaces.

3.5.7 Network verification and design limitations

A numerical comparison uses identical inputs, parameters and precision, with random masks matched or disabled. A learning comparison instead evaluates trained predictions. Table 8 gives the archived float64 check on two random images, including the reader and an earlier thirteen-output writer head.

Recorded comparison metric	Reader	Writer classifier
Maximum logit difference / maximum magnitude	2.12×10^{-14}	3.04×10^{-15}
Absolute loss difference	1.14×10^{-13}	1.78×10^{-15}
Worst ranked gradient relative L_2 difference	2.80×10^{-13}	6.01×10^{-14}

Table 8. Archived float64 operation comparison.

The logit metric uses the maximum absolute difference divided by the larger maximum magnitude, with a 10^{-8} floor. Gradient comparisons use relative L_2 difference and exclude near-zero tensors from the worst-ranked summary. The check supports the tested operations, rather than every possible training run.

The archive also contains a 500-line, 15-epoch learning comparison and an exhaustive 1024-path CTC check. The latter gives loss discrepancy 4.44×10^{-16} and maximum derivative discrepancy 3.63×10^{-11}. Their fixtures and gradient checks are retained in Appendices B and B.8.

Long-line resizing reduces fine detail, personal line splits share capture conditions, and the closed writer head always selects among its trained identities. These limits guide the examples and held-out evaluation. Final recognition and writer results are reported in Section 4.

3.6 Style extraction and handwriting reproduction

Recognition returns text, but the gantry needs pen paths. The reproduction stage extracts reusable glyphs from labelled handwriting, stores each writer's shape and spacing measurements, and assembles them into new text.

3.6.1 Choice of reproduction representation

A glyph profile reuses the few labelled pages available for each writer. Haines et al. used approximately four A4 pages in their standard glyph-based experiments [23, p. 4]. The explicit paths can be inspected, resized and mapped to the gantry without training another generator.

The photographs contain no measured pen order, speed or pressure, unlike the trajectories required by recurrent synthesis [3, p. 18]. Image generators can use few style references, but still need a separate training corpus. One-DM's IAM experiment used 339 writers and four RTX 3090 GPUs [22, p. 9]. The chosen glyph approach fits the available data and compute, with weaker handling of letter shapes that change with their neighbours.

3.6.2 Character alignment and glyph extraction

A source line has a known transcript but no character-position labels. The reader supplies frame probabilities, and forced alignment selects the highest-scoring path consistent with that transcript. It uses the CTC transition rules, replacing path summation with maximisation [14, pp. 4–5].

$$V(t,s) = \log p_t(\ell'_s) + \max_{r \in \mathcal{P}(s)} V(t-1,r). \tag{28}$$

Here ℓ' is the blank-interleaved transcript and $\mathcal{P}(s)$ is the permitted predecessor set for state s. Backtracking gives each character's frame interval. A frame edge t maps approximately to source position $(640t/T)/a$, using the inverse canvas scale a. Several plausible boundaries can still give the correct transcript.

The horizontal ink profile estimates the body band. Its first and last rows above 45% of the maximum define the band top and baseline b, with difference h_x giving estimated x-height. Bands below three pixels are rejected. Initial character cuts are refined towards thin connectors or blank columns using the project-specific cost

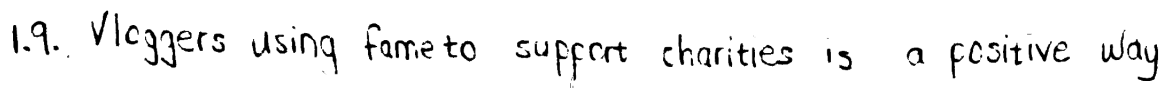
$$C_x = \frac{\overline{p}_x}{q_{0.60}(\{\overline{p}_j : \overline{p}_j > 0\})} + 1.15\,\mathrm{clip}\left(\frac{b - y_{\mathrm{top}}(x)}{h_x}, 0, 3\right). \tag{29}$$

where $\overline{p}_x$ is smoothed column ink, the denominator is its positive-count 60th percentile, and $y_{\mathrm{top}}(x)$ is the highest ink row. The height term favours shallow connectors near the baseline. A true blank column has zero cost. Cuts remain ordered, with minimum cell width $\max(1, 0.08h_x)$, so neighbouring boundaries cannot collapse onto the same gap.

Figure 21 follows the archived first Owen line because its crop and transcript can be traced through recognition and extraction. It shows the initial frame locations, refined cuts and resulting glyphs. The worked path and cut-cost substitutions are in Appendix A.10.

Known transcript: 1.9. Vloggers using fame to support charities is a positive way

Original line crop



Refined shared boundaries (blue)





Figure 21. Character alignment and refined glyph cuts.

3.6.3 Skeleton tracing and pen path construction

Thinning converts ink area into a one-pixel centreline [26, p. 237]. The complete line is thinned before cutting it into cells, avoiding an artificial thinning boundary at every letter. A graph tracer splits paths at junctions, prunes short spurs and rejoins compatible ends below a 75° continuation angle. Douglas–Peucker simplification removes unnecessary vertices within a bounded deviation [25].

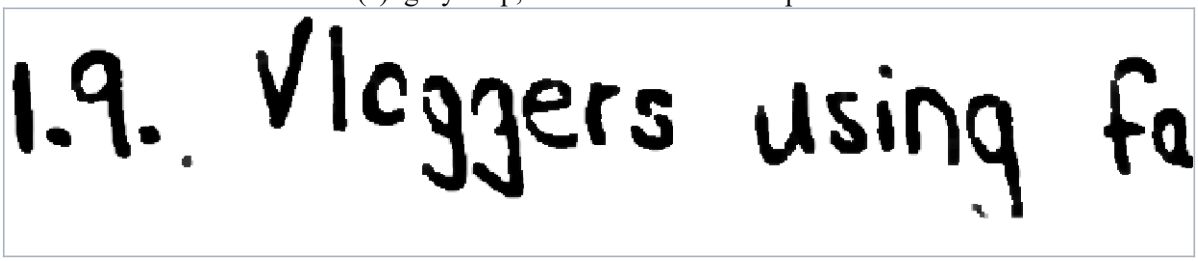
Isolated dots are recovered as short paths. Eligible detached crossbars are assigned to their letter rather than split across cells or counted as cursive joins. The resulting points are stored relative to left cut x_L, baseline b, x-height h_x and slant ϕ, using a shear transform [9].

$$v = \frac{b - y}{h_x}, \qquad u_0 = \frac{x - x_L}{h_x} - v \tan \phi, \qquad u = u_0 - \delta. \tag{30}$$

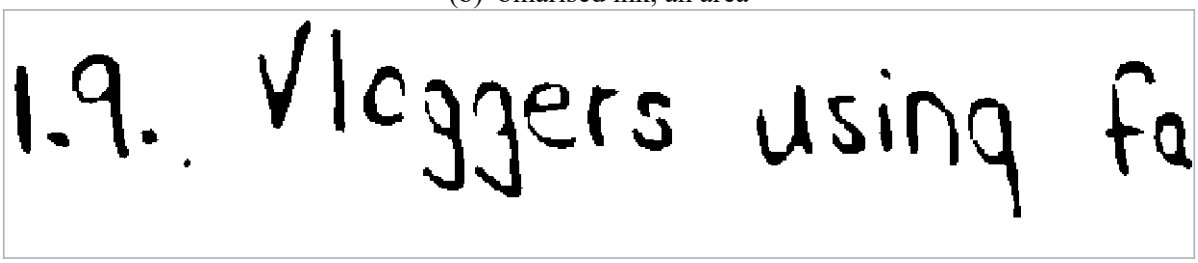
The vertical coordinate increases upwards. The trim shift δ is removed from the stored horizontal path and retained as lead-in spacing. Slant is removed for storage and reapplied at placement. This separates the letter shape from its source size and cell spacing.

Figure 22 shows the Owen fragment before and after thinning and tracing. Across its full line, simplification reduces 3738 vertices to 974 at 0.6 source-pixel tolerance. The graph gives geometric paths, while the writer's original stroke order remains unknown. The coordinate substitution is retained in Appendix A.10.

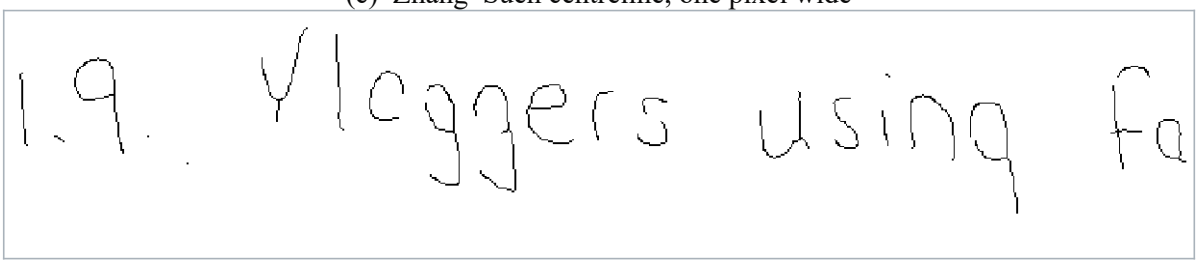
(a) grey crop, contrast stretched for print



(b) binarised ink, an area



(c) Zhang–Suen centreline, one pixel wide



(d) 85 traced paths over the whole line, simplified from 3738 to 974 vertices at ε=0.6 px
a dot marks each path start, and a change of shade marks a new path

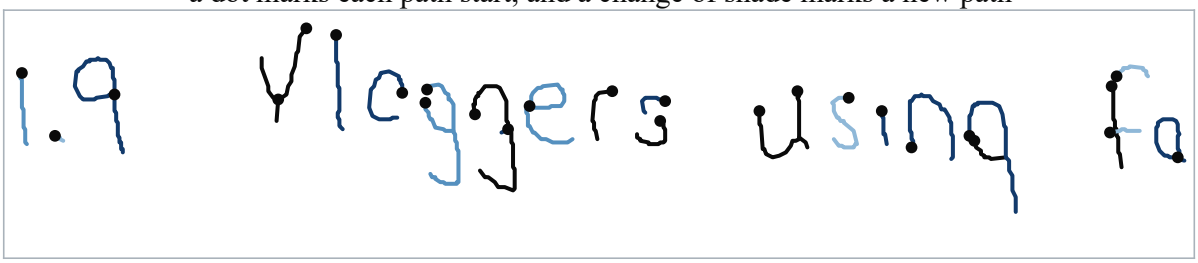


Figure 22. Ink, skeleton and simplified pen paths.

3.6.4 Glyph selection and library quality

Alignment can place a cut inside a letter even when the text is correct. The profile therefore filters unreliable lines and glyphs using width, height, stroke count, path length and allocated character advance. Ordinary widths are limited to 2.1 x-heights and wide letters to 2.9. Character-specific distributions provide further checks without forcing every letter to the same size.

A normalised 12×12 occupancy descriptor compares shape. A pooled character reference identifies malformed cuts, while pairwise distances within a writer's pool identify a representative

real variant, called the medoid. A separate denoised prototype combines compatible strokes. The ordinary pool holds up to twelve variants per character. Algorithm 3 shows where these checks enter profile construction. The descriptor distance is $d(g, c) = 1 - v_g^T r_c$ for unit-normalised glyph descriptor v_g and class reference r_c. The medoid minimises summed within-class distances.

Algorithm 3 Labelled handwriting to a persistent writer profile

Require: Labelled original lines $\mathcal{L}$, trained reader and writer identifier.

$\mathcal{G} \leftarrow \varnothing$; $\mathcal{S} \leftarrow \varnothing$
for each $(G, \ell) \in \mathcal{L}$ **do**
    $A \leftarrow \text{ForcedAlign}(G, \ell)$; estimate baseline, x-height and slant
    if alignment or body band is unusable **then continue**
    Refine shared increasing cuts; thin full line; trace skeleton paths
    Split paths at cuts; assign crossbars; recover untraced dots
    Normalise glyph coordinates; record advance, lead and boundary crossings
    Append glyphs to $\mathcal{G}$ and line measurements to $\mathcal{S}$
end for
Cache raw extraction; reject low-confidence lines relative to this writer
for each character's glyph pool $\mathcal{G}_c$ **do**
    Apply size, height-class, path-length and cell-consistency gates
    Refine size range; apply cross-writer prior and within-writer consensus
    Rank and smooth retained variants; construct medoid and eligible prototype
end for
return persistent profile containing $\mathcal{G}$ and robust summaries of $\mathcal{S}$

Figure 23 compares the nine stored `a` variants for Owen, Dylan and Yeukita. The cloud shows their variation, the medoid is an actual example, and the prototype is a combined shape. The distance bars explain which variant is most typical within each writer. The exact descriptor and medoid calculations are retained in Appendix A.10.

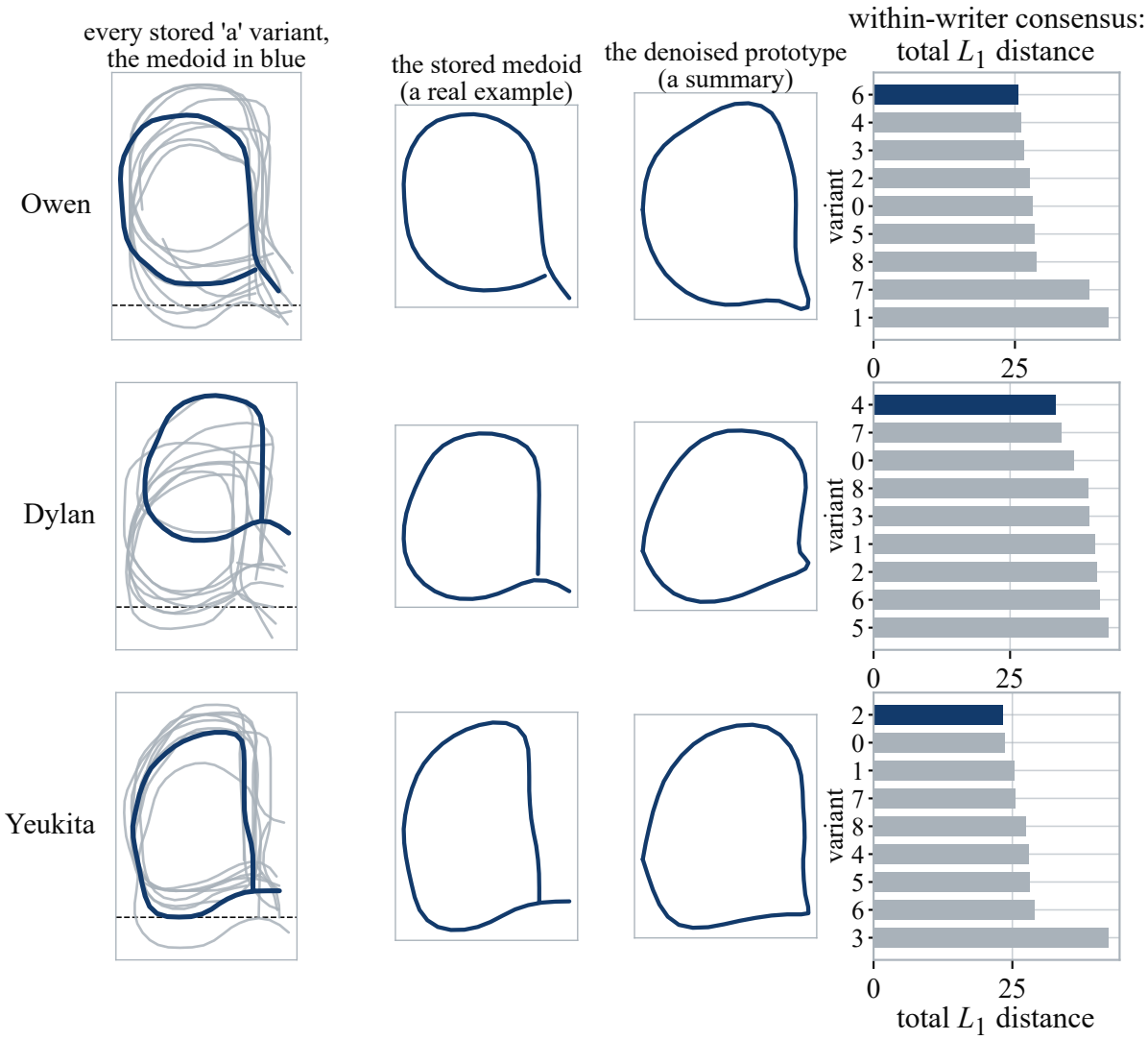


Figure 23. Glyph variants and representatives for three writers.

Missing characters can use case-related shapes, similar forms or fallback glyphs. Source tags distinguish these choices from observed personal letterforms. They allow text generation to continue, but a fallback does not establish that the character was captured in that writer's style.

3.6.5 Writer style measurement and storage

The profile stores slant, x-height, ascender and descender extent, character advance, word gap and connection fraction. Slant is estimated by finding the shear that concentrates ink into columns. Dimensions are normalised by x-height to reduce dependence on photograph resolution.

Connectedness is the fraction of eligible adjacent non-space boundaries crossed by a traced centreline, motivated by contour-based writer features [18, p. 701].

$$c = \frac{N_{\text{cross}}}{\max(1, N_{\text{adj}})}. \tag{31}$$

The Owen example has 3 crossings among 39 eligible boundaries, giving 0.077. This is a property of the source ink, not a measured pen-lift sequence. Glyph paths and style measurements are stored together under the writer identity for later requests.

3.6.6 Text layout and connected handwriting synthesis

The synthesiser selects glyph variants, applies the stored style and places words within the available line width. Character advance and word gap come from the profile. Words wrap to the next line when the remaining width is insufficient.

For stored point (u, v) and lead l, physical x-height h_x^{mm} converts the glyph to millimetres. Slant shear s and baseline drift d are applied about placement origin (p_x, p_y).

$$x = p_x + h_x^{\mathrm{mm}}(l + u + sv), \qquad y = p_y + h_x^{\mathrm{mm}}(v + d). \tag{32}$$

Small translations, rotations and scale changes provide variation. Drift changes gradually with persistence 0.85 and is bounded to ±0.25 x-heights. A nominal 0.05 x-height clearance limits collisions. Better-ranked variants are sampled more often, with compatible entry heights preferred for joining.

The automatic anchoring strength λ depends on median stored prior distance $\widetilde{d}$.

$$\lambda = \mathrm{clip}\left(\frac{\widetilde{d} - 0.24}{0.32 - 0.24}, 0, 1\right). \tag{33}$$

Lower-distance pools retain more personal variants. Larger distances increase reliance on an acceptable representative and then a generic form if needed. This is a heuristic, with source tags recording the resulting glyph choice.

Joining uses the writer's measured connection fraction, but is suppressed below 0.45. A join also requires a compatible source and gap no greater than 0.80 x-heights. A shallow connector joins the body strokes without attaching to dots or crossbars. A rendered probe applies bounded slant and extent corrections before synthesis. The placement and anchoring examples are retained in Appendix A.10.

3.6.7 Candidate selection for legibility and writer identity

Different variant choices can change readability and writer identity. Each candidate is rendered at constant ink width and scored by the reader and stroke-normalised writer classifier. Character score A and requested-writer probability P form joint score J.

$$A = \max\left(0, 1 - \frac{d_{\mathrm{edit}}(\widehat{t}, t)}{|t|}\right), \qquad J = \begin{cases} 2AP/(A+P), & A + P > 0, \\ 0, & \text{otherwise.} \end{cases} \tag{34}$$

The harmonic mean penalises a candidate weak in either score. The recorded Owen candidate has $A = 18/19 = 0.95$ and $P = 0.58$, giving $J = 0.72$. Algorithm 4 gives the bounded search and repair sequence. A repair is retained only when the complete candidate score improves.

Figure 24 compares the three recorded trials of "Writing with style." using the same profile and settings. Candidate 1 is selected because its writer score is stronger while retaining the same character score as candidate 3. The full values are in Appendix A.10.

Algorithm 4 Joint candidate search and bounded legibility repair

Require: Text, writer profile, reader, shape classifier, seed s and trial budget K.

$J^* \leftarrow -1$; $R \leftarrow \varnothing$; determine anchoring strength
for $k \leftarrow 0$ **to** $\max(1, K) - 1$ **do**
  $T \leftarrow \text{SynthesizeText}(\text{text}, \text{profile}, s + 977k)$
  $I \leftarrow \text{RenderUniformInk}(T)$; $(A, P, J) \leftarrow \text{Score}(I)$
  if $J > J^*$ **then** $(T^*, I^*, J^*) \leftarrow (T, I, J)$
  if $J^* \geq 0.99$ **then break**
end for
for at most three configured repair rounds **do**
  if $J^* \geq 0.995$ **then break**
  $W \leftarrow$ up to two weak aligned positions not already in R
  if $W = \varnothing$ **then break**
  $R' \leftarrow R \cup W$; regenerate with stronger anchors at R' and seed s
  Render and rescore the complete trial trajectory
  if $J > J^*$ **then** commit R' and the new best trajectory
end for
return T^* with selection and accepted-repair metadata

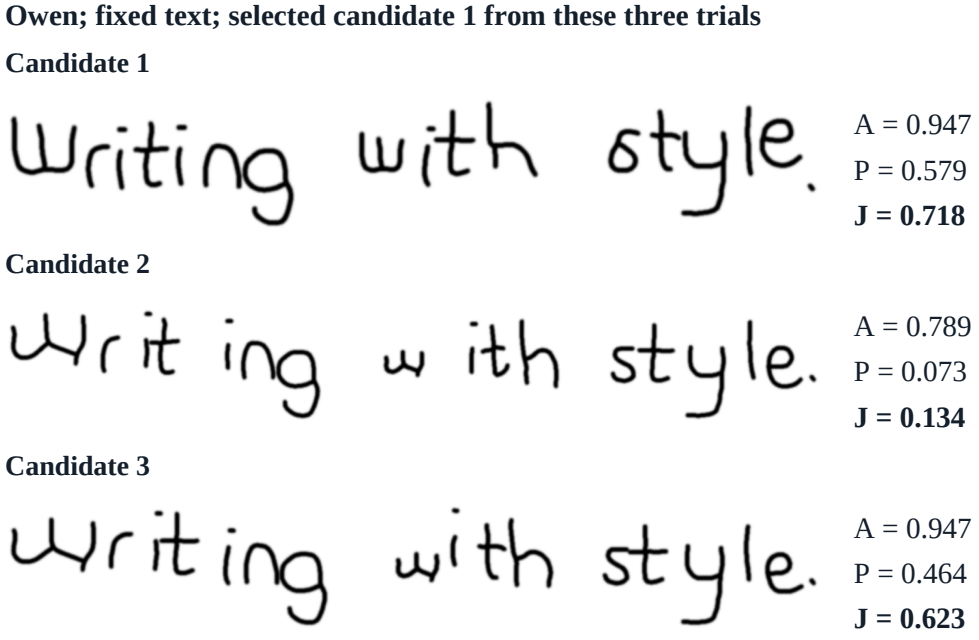


Figure 24. Candidate paths and their selection scores.

The scoring models guide this choice, so their confidence does not independently prove physical style similarity. More trials also add inference time and cannot recover a missing source letterform.

3.6.8 Machine coordinate mapping and G-code generation

The selected trajectory is fitted inside the calibrated workspace. Ruled placement adjusts each line to the chosen ruling, while free placement retains its synthesised pitch. Character size and line spacing remain separate so that a tall descender need not change both.

Trajectory bounds (x_0, y_0, x_1, y_1), origin (o_x, o_y), margins m_x, m_y and scale a define the mapping. Offset d_y selects a later starting line and Δ_y adjusts line spacing.

$$x_m = o_x + m_x + a(x - x_0), \qquad y_m = o_y - m_y - d_y - a(y_1 - y) + \Delta_y. \tag{35}$$

Both stored and machine coordinates use upward-increasing y. Clamping is a final travel limit and can distort a letter. Robust extent fitting and margins should keep the generated text inside the bed before execution. Nearby path segments may be simplified and compatible strokes joined, using a default join distance of 0.6 mm.

Figure 25 gives the opening commands of the generated Owen example. It selects millimetres, absolute coordinates, a raised-pen travel and the first drawing movements. The full point mapping and file interpretation are extended in Appendix A.10.

```
G21
G90
G0 F2400
G0 F2400 X15.000 Y240.557
M3
G4 P0.350
G1 F900
G1 X15.001 Y240.207
G1 X15.002 Y239.857
G1 X15.003 Y239.506
G1 X15.005 Y239.156
G1 X15.006 Y238.806
G1 X15.007 Y238.456
```

Figure 25. Opening commands of the Owen example.

Travel feed is restated after drawing because feed is modal. Pen dwell adds 0.35 s per transition to the switch-confirmed actuation. For drawing and travel distances D_d, D_u, feeds F_d, F_u and N_p alternating pen transitions, the nominal budget is

$$t_{\text{nom}} = \frac{60 D_d}{F_d} + \frac{60 D_u}{F_u} + \frac{N_p (t_{\text{up}} + t_{\text{down}})}{2}. \tag{36}$$

The earlier traced file gives 32.82 s, including 19.60 s dwell. Figure 26 shows a separate four-profile calculation for the same sentence. Dwell contributes about 61 to 65% of these nominal budgets. This identifies a motion-side improvement to test, while actual completion also includes synthesis, actuation and pulse-loop overhead.

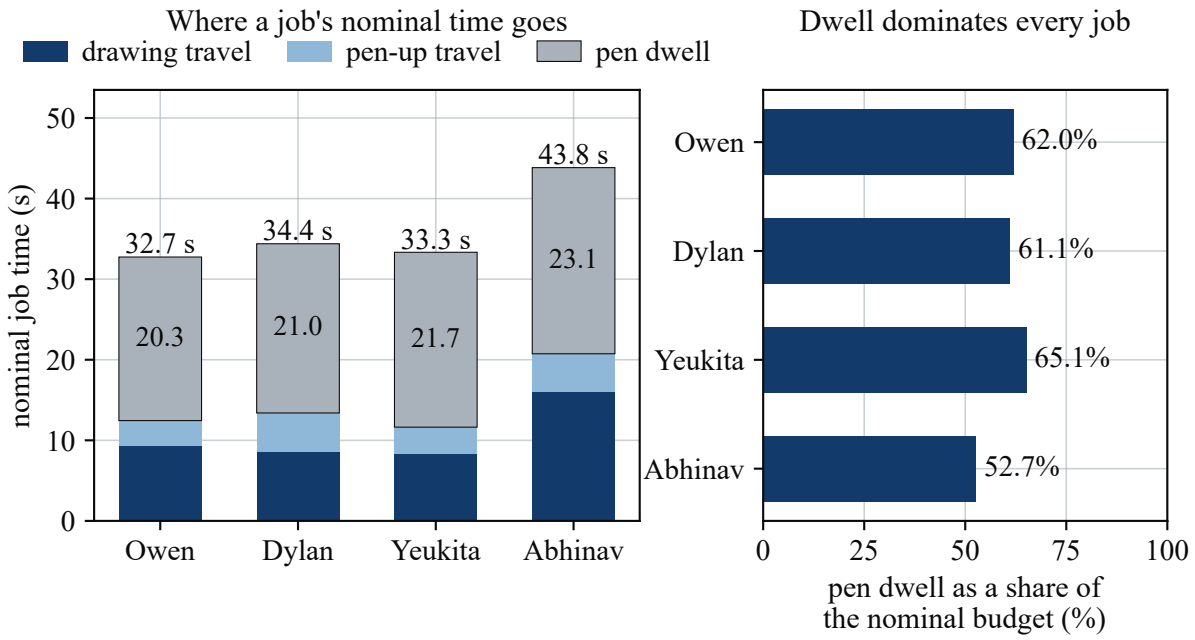


Figure 26. Drawing, travel and pen-dwell budgets.

3.6.9 Reproduction verification and design limitations

Checks follow source ink, stored glyphs, assembled paths and emitted G-code. The independent file parser in Appendix B recovers the generated paths. Nine controlled fixtures preserve vertex counts with maximum coordinate discrepancy 0.0004 mm, consistent with three-decimal coordinate emission. Physical drawing tests then check the mechanism rather than file conversion alone.

The explicit glyph representation makes source errors and scaling inspectable. Its main limits are context-dependent letterforms, missing characters and the absence of natural pen dynamics. Multi-character units and recorded trajectories could address these losses, but need additional reliable data. Physical qualification remains in Results.

3.7 System integration and embedded operation

The server connects page processing, models, profiles and motion control to the browser.

3.7.1 Classification and reproduction workflows

Captured and uploaded pages use the same preparation and recognition path. For each writer k, page pooling combines the line probabilities $p_{i,k}$ with ink weights w_i.

$$\overline{p}_k = \frac{\sum_i w_i p_{i,k}}{\sum_i w_i}, \qquad \widehat{k} = \arg\max_k \overline{p}_k. \tag{37}$$

Each weight counts pixels below grey value 128. Empty weights fall back to equal weighting. Crop rejection precedes pooling so that a dark artifact does not gain excessive influence. The two-line Owen calculation is retained in Appendix A.10.

Each generated job stores paths, preview, G-code and metadata. Drawing requires connection and calibration, allowing preview inspection first. Table 20 records the data interfaces.

3.7.2 Embedded execution, interfaces and job control

The ODROID runs inference and motion control. NumPy supplies array operations, Flask handles requests, image libraries and OpenCV load or capture images, and gpiod controls GPIO.

The pulse loop checks aborts and confirmed end stops. A separate request thread accepts stops during drawing. End-stop faults trigger the configured recovery. Job records distinguish generation, refusal, completion and interruption. Software stops do not independently disconnect motor power.

3.7.3 Integration verification and remaining constraints

Interface checks cover input consistency, profile mapping, G-code bounds and interruptions. Diagnostic retracing bypasses recognition and synthesis. Comparing its preview with the drawing separates extraction faults from execution faults.

Stage timings separate processing, inference, generation and drawing. Reproduction totals include generation. Results evaluates accuracy and embedded runtime.

4. Results

The qualification tests follow the six requirements of the Project Proposal in order of importance. The physical reproduction, photographed-page analysis and embedded timing records are kept identifiable so that each result can be traced to the part of the system that produced it. Section 5 interprets the engineering consequences. this section states the procedures and measurements.

4.1 Summary of results achieved

Table 9 compares the measured outcomes with the Project Proposal. The qualification tests below give the procedures and evidence.

Requirement	Measured outcome	Assessment
R1. At least 85% character accuracy and style correlation	Physical character accuracy 84.41%. Seven of eleven writers exceed 85%. Shape correlation 0.697.	Below pooled targets
R2. At least 95% text and writer accuracy	Reader 94.06%. Writer identification 153/156, or 98.08%.	Writer target met
R3. At most 4 s per character for reproduction	All 25 completion-confirmed ODROID jobs pass. Worst 3.875 s per character.	Met in tested jobs
R4. Classification within $2 + 0.025x$ s	Two ODROID requests take 46.26 and 55.49 s, against 14.50 and 26.55 s allowed.	Not met
R5. At most 0.5 mm positional error	Seven measured feature lengths have a maximum absolute error of 0.37 mm. Absolute placement and repeatability need further measurements.	Dimensional check met
R6. At least ten stored styles	Eleven profiles contain glyph mappings and are exercised in physical reproduction.	Met
Field conditions. Well-lit, stable bench	Indoor captures and drawings demonstrate the fixed setup. No calibrated lux or vibration series was recorded.	Observed setup only

Table 9. Results against the proposal.

4.2 Qualification tests

4.2.1 Test material, configuration and measurement definitions

The evaluation uses six IAM writers (150, 152, 153, 384, 552 and 588) and five personal contributors (A, B, O, T and Y). Fourteen physical pages provide 184 transcript-paired lines. A separate model comparison includes one additional unpaired crop and therefore has 185 crops.

The gantry produced the pages under ODROID control. Saved images were scored on the development host. Embedded timings come from the preserved ODROID log. The selected joint reader and eleven-writer classifier use the configuration described in Design, with writer stroke width 0.17. Appendix C identifies the measured checkpoints and source records.

Character accuracy uses the summed Levenshtein distance d_{edit} between recognised text $\hat{t}_i$ and reference t_i, following the character-error convention [14, p. 2].

$$A_c = 100\left(1 - \frac{\sum_i d_{\text{edit}}(\hat{t}_i, t_i)}{\sum_i |t_i|}\right). \tag{38}$$

The physical result is $100(1 - 853/5471) = 84.41\%$. Writer accuracy is the proportion of correctly identified lines. Page pooling is reported separately. Word match counts equal words at corresponding positions, so it differs from word edit accuracy. Full page and line records are retained in Appendix C.10.

Wilson intervals describe line-classification uncertainty [42, pp. 209–212]. Lines share writers and capture conditions, so these are descriptive intervals. Resampling whole writer groups provides a separate sensitivity check on the physical character and style scores.

Qualification test 1: Reproduction of digital text in a trained writer's style (R1)

Objectives of the test or experiment

Measure legibility and writer-style retention of new digital text after physical reproduction. Compare character accuracy and geometric style correlation with the two 85% targets.

Equipment used

The calibrated gantry, ODROID N2+, IMX179 camera, stored profiles and selected models. Saved photographs are 3264×2448 pixels.

Test setup and experimental parameters

Each writer reproduces a common paragraph, a pangram and available source text. Transcripts follow the photographed line breaks. The final stored glyph mappings, greedy CTC decoding and 0.17 writer normalisation are used.

Steps followed in the test or experiment

Generate and draw each job, photograph it, pair the segmented lines with transcripts, then save reader edits and writer predictions. Sum the counts by page, text block and writer. Extract the same geometric descriptors from original and reproduced lines for the style comparison.

Results or measurements

Table 10 gives the 184 paired lines by text block. Figure 27 exposes the variation across writers. the full values are in Table 26. Figure 28 provides examples of actual robot output alongside both strong and more difficult cases. Strip labels give each writer's aggregate character accuracy. The dashed 85% line in Figure 27 marks the character target, with identity reported separately.

Text block	Lines	Characters	Character / %	Writer / %
Paragraph	119	4041	84.88	91.60
Pangram	21	463	78.40	76.19
Source text	44	967	85.32	77.27
All	184	5471	84.41	86.41

Table 10. Physical reproduction scores by text block.

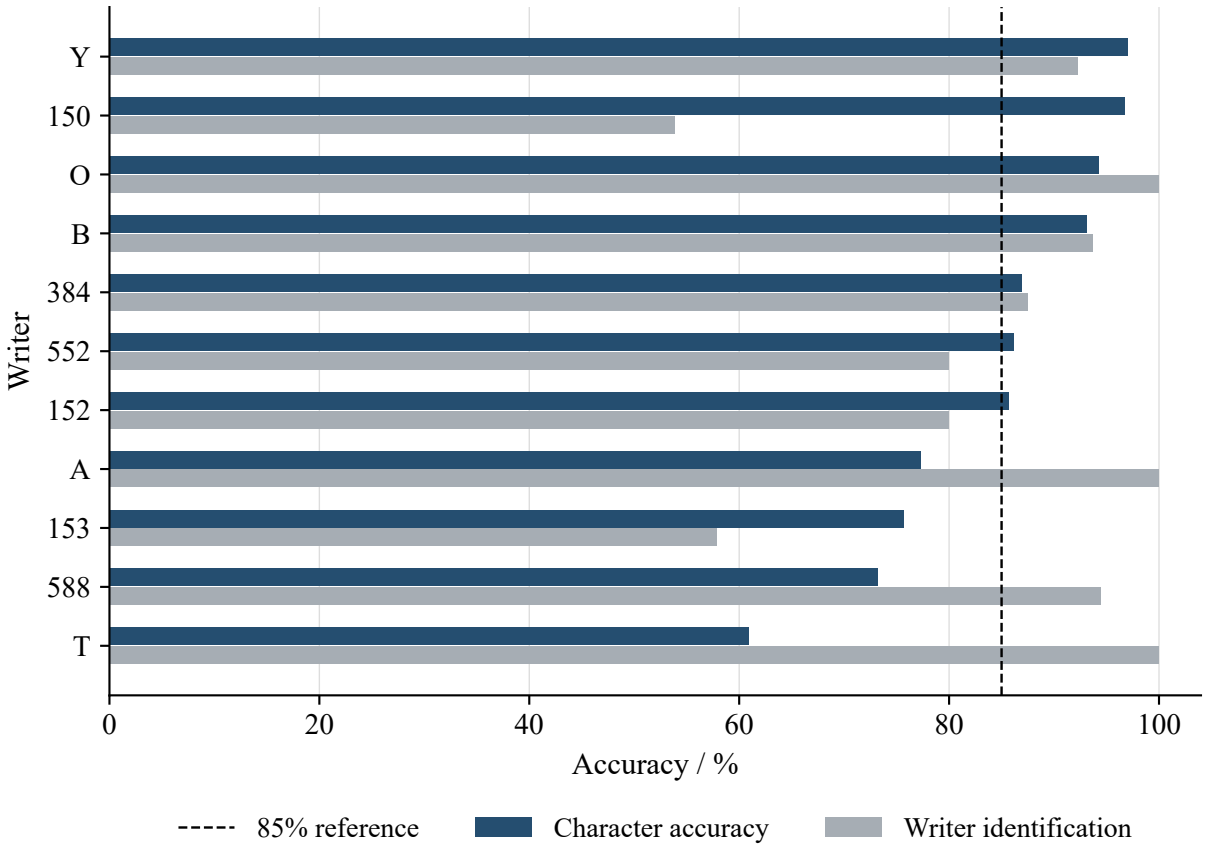


Figure 27. Physical reproduction scores by writer.

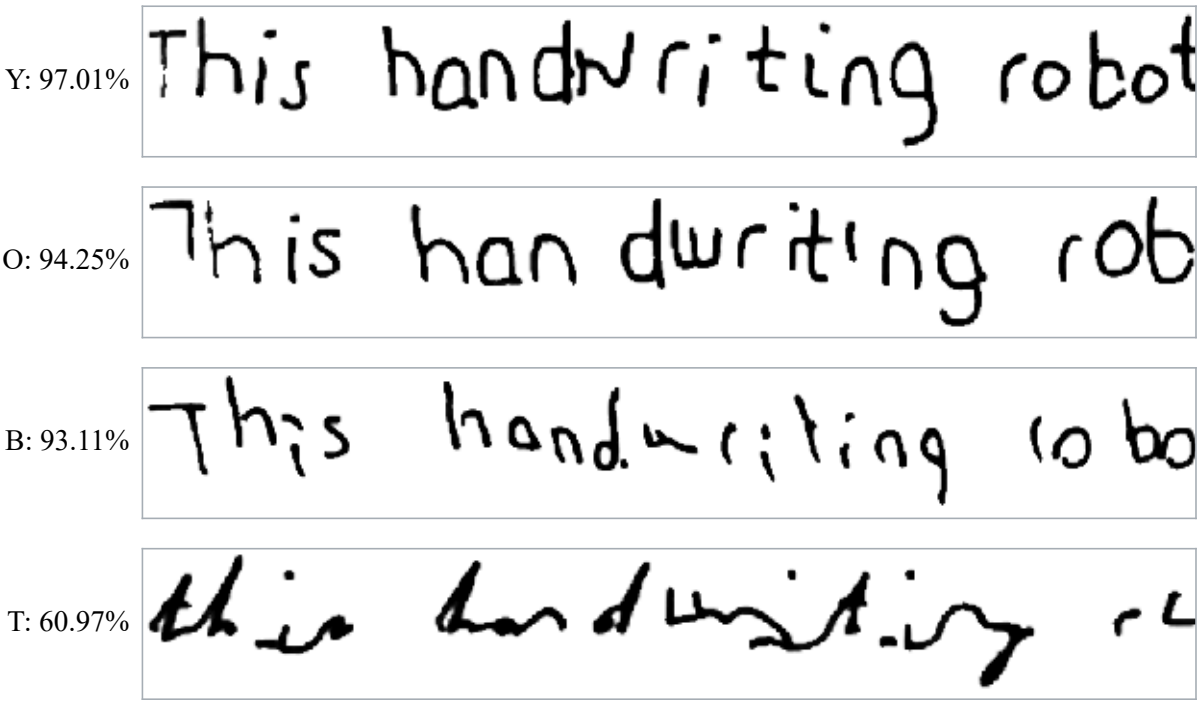


Figure 28. Examples from four physical writing styles.

Overall character accuracy is 84.41%, positional word match is 45.97% and writer identification is 159/184 (86.41%). All fourteen recorded page pools are correctly attributed. Seven writers exceed 85% character accuracy. The complete Owen page in Figure 29 connects the line examples to a physical sheet. The full fourteen-page set and page scores are in Appendix C.10.

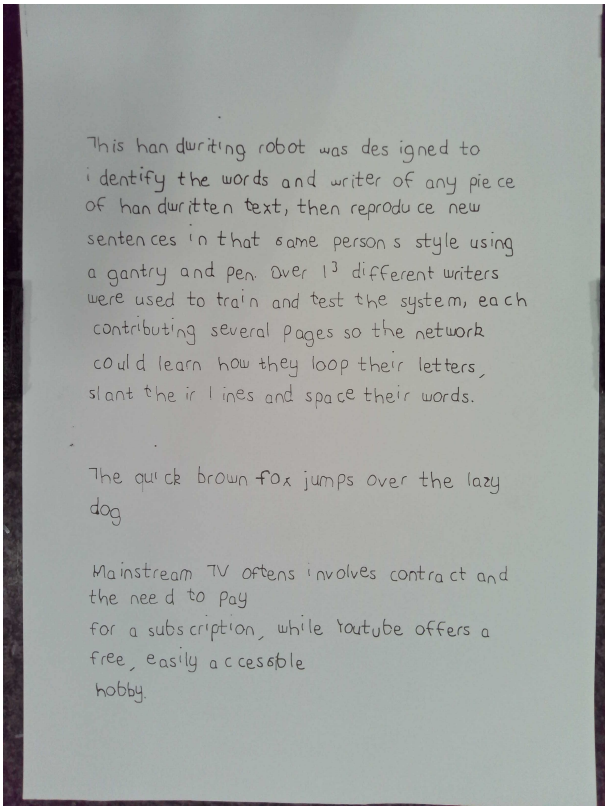


Figure 29. Owen's reproduced page.

Figure 30 compares the physical-output writer predictions. Each row sums to 100%, giving each writer equal visual weight despite different line counts. The row labels retain the actual sample sizes. This normalisation changes the display, not the measured 159 correct predictions.

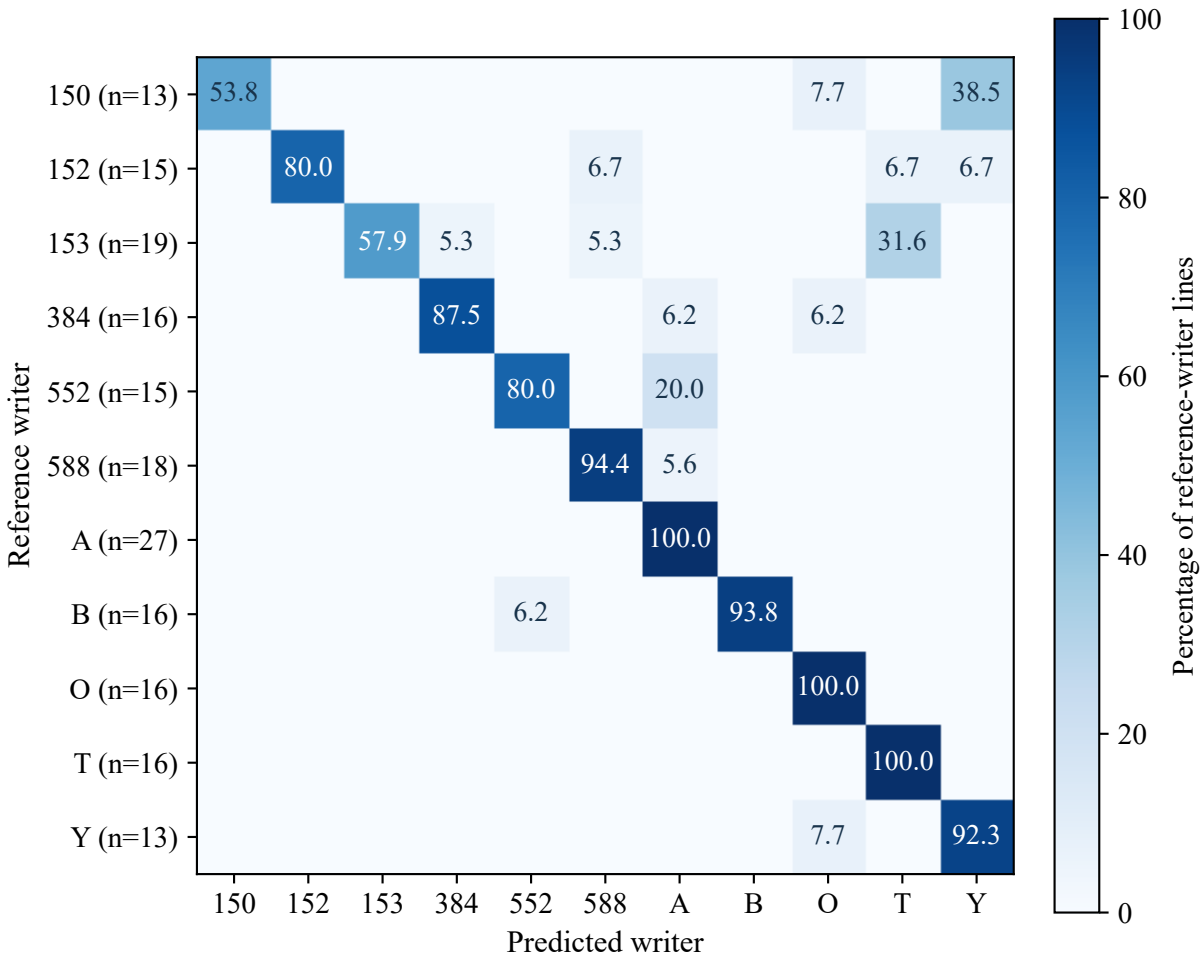


Figure 30. Writer identification on reproduced lines, row percentages.

The geometric comparison uses 156 original and 184 physical lines across all eleven writers. Nine descriptors include ink density and stroke width. Seven shape descriptors omit these pen-dependent quantities. Table 11 gives the correlations. Original and reproduced sentences are not matched, so this measures writer-level geometry across the available samples.

Feature set	Features	Separate domains	Absolute variant
All geometric descriptors	9	0.713	0.633
Shape descriptors (ink omitted)	7	0.697	0.611

Table 11. Physical-to-original geometric correlations.

Slant correlation is 0.991, ascender extent 0.817, character width 0.798 and word gap 0.486. Separately standardising the seven features in each domain gives pooled shape correlation 0.697.

Using original-domain scaling for both gives 0.611. Appendix C.4 retains the definitions and full descriptor values.

Observations

The pooled character score is 0.59 percentage points below target. Writer scores range from 60.97% to 97.01%. Legibility and identity measure different properties. T is identified on every line but has the lowest character score, while 150 scores 96.75% character accuracy with 53.85% identification. More separated original handwriting is associated with better reproduced character accuracy across these writers (Spearman $\rho = 0.845$), without isolating the cause of the difference.

Statistical analysis

The Wilson interval for 159/184 identifications is 80.7% to 90.6%. Resampling eleven whole writers 10 000 times gives a central 95% range of 77.82% to 90.27% for character accuracy and 0.555 to 0.827 for shape correlation. These describe sensitivity to the observed writer set. Neither pooled target is met by the observed point estimate.

Qualification test 2: Reading handwritten content and identifying its writer (R2)

Objectives of the test or experiment

Evaluate reading and writer identification separately against 95%, using the recorded validation partitions.

Equipment used

Saved IAM and personal line images, reference transcripts, selected checkpoints and the scoring tools. This experiment does not require gantry movement.

Test setup and experimental parameters

The reader population has 976 public validation lines (42 072 characters) and 100 photographed lines (3 675 characters). The writer population has 156 lines, using held-out IAM pages and a seeded personal-line split. Personal validation can share page backgrounds with training. These are validation results because checkpoint selection may have used the same split.

Steps followed in the test or experiment

Check the corpus partitions, read each line with greedy CTC decoding, classify the normalised writer crop and retain the edits, reference counts and predictions. Aggregate the reader populations and writer confusion counts separately.

Results or measurements

Table 12 reports 94.06% pooled character accuracy for the selected joint reader. The scratch comparison reaches 94.39% on the same 1 076 lines. The training-route comparison is in Table 7. Table 25 retains the individual personal-writer scores. The writer model correctly identifies 153/156 lines (98.08%) and all eleven pooled writer groups. Figure 31 shows the row-normalised predictions, with actual line counts beside each writer. The only three errors are predictions of Y.

Population	Lines	Characters	Joint / %	Scratch / %
IAM validation	976	42072	94.12	94.39
Personal validation	100	3675	93.33	94.42
Pooled	1076	45747	94.06	94.39

Table 12. Reader character accuracy on validation lines.

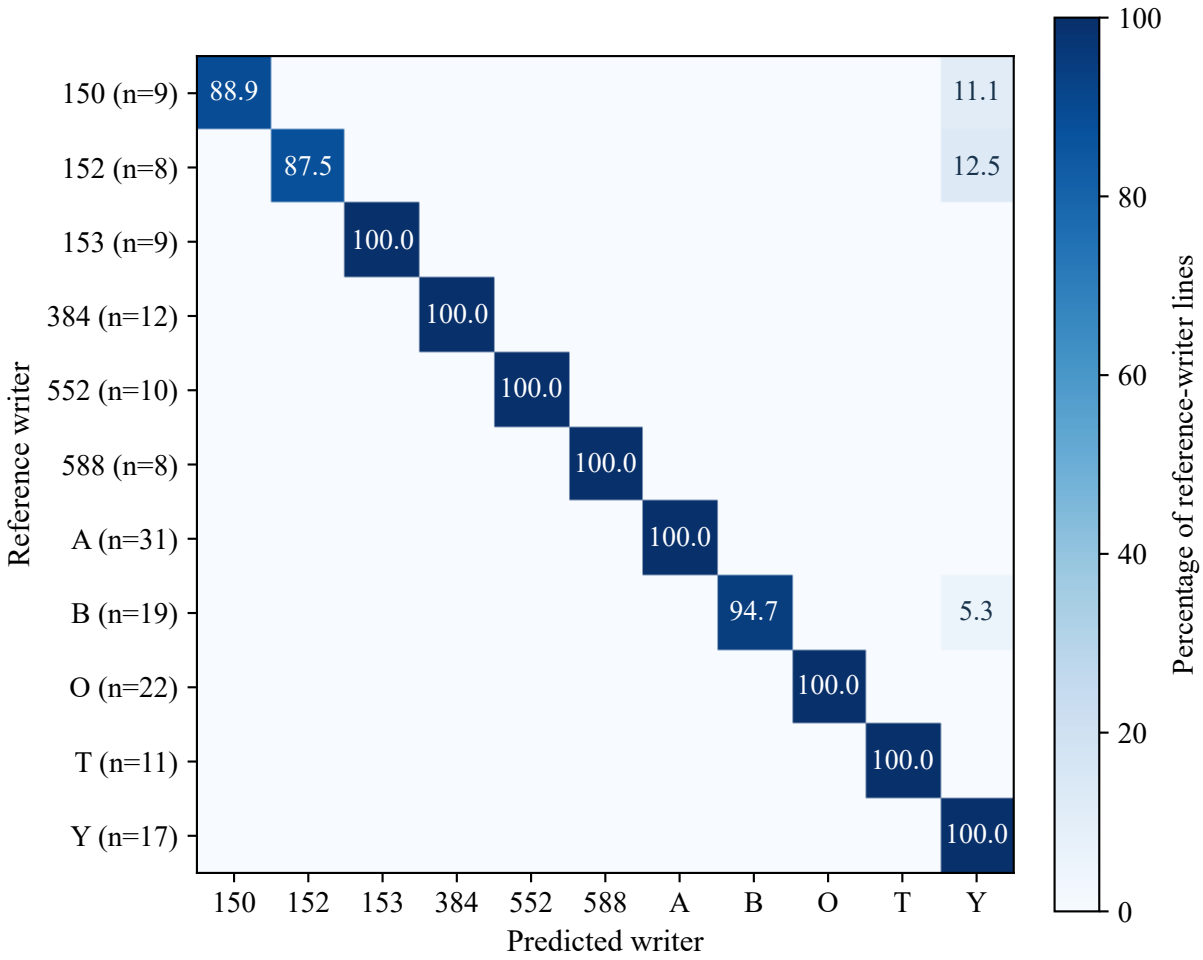


Figure 31. Writer validation predictions, row percentages.

Observations

The writer point estimate exceeds target by 3.08 percentage points. The selected reader is 0.94 points below target and recovers 221 complete lines without edits. One incorrect writer prediction has 83.8% confidence, showing that a high posterior alone cannot guarantee identity.

Statistical analysis

The writer Wilson interval is 94.5% to 99.3%. The result demonstrates a point estimate above 95%, with a stronger pooled decision across the available lines. Character accuracy is an edit-distance ratio, so a binomial interval on individual characters is not assigned.

Qualification test 3: Complete reproduction time on the embedded platform (R3)

Objectives of the test or experiment

Compare total generation and drawing time with the proposal allowance of $4x$ seconds for x characters.

Equipment used

ODROID N2+, gantry and the server timer. The preserved log supplies the measured totals and completion status.

Test setup and experimental parameters

Retain positive-character jobs with a drawing stage and an explicit program-end marker. Exclude stopped jobs, commissioning shapes and entries without confirmed completion. Generation time is carried into the recorded drawing-request total.

Steps followed in the test or experiment

Associate each timing block with its terminal status, retain completed jobs, calculate t/x and compare the recorded total t with $4x$. Group the retained records by text length.

Results or measurements

Table 13 summarises 25 completed requests. All pass. The worst is $166.61/43 = 3.875$ s per character, leaving 5.39 s. At 378 characters, nine totals range from 507.57 to 1085.01 s against 1512 s allowed. Figure 41 and the complete records are retained in Appendix C.6.

Characters	Records	Allowance / s	Worst total / s	Worst / s per char
43	9	172	166.61	3.875
75	1	300	188.60	2.515
82	3	328	260.33	3.175
91	2	364	222.63	2.446
132	1	528	277.90	2.105
378	9	1512	1085.01	2.870

Table 13. Completed ODROID reproduction jobs by text length.

Observations

The shortest tested length has a minimum margin of 3.1%, compared with 28.2% for the longest. Timing therefore depends on both fixed generation work and the drawn path. Completion and timing do not replace the separate fidelity checks.

Statistical analysis

All 25 observations pass, with a descriptive Wilson interval of 86.7% to 100%. The records cover six lengths from 43 to 378 characters, with one to nine observations at each length. They do not establish a timing envelope for shorter requests.

Qualification test 4: Time to decode a photographed page and identify its writer (R4)

Objectives of the test or experiment

Compare the complete embedded classification request with $2 + 0.025x$ seconds and identify the stages responsible for the difference.

Equipment used

ODROID server log and stage timer. Supporting acceleration, resolution and per-line rate experiments use saved pages on the development host, running the same deployed code and checkpoints on eight logical cores.

Test setup and experimental parameters

The two ODROID requests log 500 characters over 16 lines and 982 over 22 lines. Totals include loading, conditioning, segmentation, cropping and both networks. Allowances use those logged counts. Camera acquisition outside the timer is not included.

Steps followed in the test or experiment

Read the totals and component stages, calculate the allowances and compare them. In separate host experiments, compare sequential and batched network execution on identical saved pages, then check the consequences of reducing image resolution.

Results or measurements

Table 14 gives the ODROID measurements. Neither request meets R4. Networks account for 67.6% of their combined time. Figure 42 and the supporting comparisons are in Appendix C.6. Expressed per line instead of per character, the reader takes 1.189 and 1.121 s and the writer namer 0.693 and 0.637 s. The per-character rates differ by 41.0% and 43.4% between the two requests, while the per-line rates differ by 5.9% and 8.4%: both networks take one fixed 64×640 canvas per line whatever that line carries, so cost follows the line and the per-character figure only tracks how full a line is. Table 15 sets these rates beside the host measurement. The host network comparison takes 359.8 s sequentially and 130.3 s with equivalent batched modules, a factor of 2.76. Reader text agrees on every evaluated line. This changes both backend and batching. Halving image resolution is faster but loses lines and produces only 33.8% mean text agreement, so it is unsuitable as an equivalent-output replacement.

Observations

The measured totals are 3.19 and 2.09 times their allowances. Even projecting the host network factor onto these requests, with unchanged image processing, gives approximately 27.05 and

Recorded quantity	500 characters	982 characters
Loading / s	0.63	0.53
Conditioning / s	6.61	6.04
Segmentation / s	4.29	5.59
Cropping / s	4.61	4.65
Image stages / s	16.14	16.81
Reading / s	19.02	24.66
Writer classification / s	11.09	14.01
Network stages / s	30.12	38.67
Total / s	46.26	55.49
R4 allowance / s	14.50	26.55
Overrun / s	31.76	28.94

Table 14. Recorded ODROID classification-stage timings.

	ODROID, measured		Host, measured
Recorded quantity	500 char	982 char	10 pages
Lines in the request	16	22	221
Characters per line	31.2	44.6	44.9
Reader / s per line	1.189	1.121	0.371
Writer namer / s per line	0.693	0.637	0.398
Image stages / s per line	0.969	0.740	0.440

Table 15. Classification stage rates per line. The host column is a median over ten photographed pages and 221 lines.

30.82 s. Both remain above target. The projection identifies further work rather than an embedded pass.

The two requests also fix a board-to-host rate ratio per stage: 3.20 and 3.02 for the reader, 1.74 and 1.60 for the namer, and 2.20 and 1.68 for the image stages. The reader loses most, consistent with its sequential recurrent layer, and the namer, which has no recurrence, loses least. At the mean ratios a projected board cost is 2.67 s per line, 1.15 s of it reading and 0.66 s naming, so 30 s is reached at about eleven lines; projected onto the ten host pages every one exceeds its allowance, by 1.83 to 3.34 times. These are projections from host measurements and the two logged ratios, not embedded measurements, and they size the shortfall for pages the board did not run.

Statistical analysis

Two requests show two observed overruns. They do not establish a general timing distribution. The fourteen-page host comparison supports an optimisation route, but its embedded gain requires a direct ODROID measurement. Fitting the three stage ratios on one request and predicting the other request's measured total gives +14.2% and −12.5%, so a projected page total is accurate to roughly ±15% across these two points. With two observations this bounds nothing in general, and no projected figure replaces a board measurement.

Qualification test 5: Gantry dimensional accuracy and geometric consistency (R5)

Objectives of the test or experiment

Check the calibrated geometry against the 0.5 mm dimensional limit and distinguish feature lengths from absolute placement and repeatability.

Equipment used

Calibrated gantry, pen, paper and a metal non-digital vernier caliper. Its model and least count are not recorded.

Test setup and experimental parameters

One 100 mm square and one triangle of 80 mm base and nominal 40 mm height were drawn after calibration. Four square sides and three triangle sides were measured. Pen width and caliper reading affect the measurement.

Steps followed in the test or experiment

Execute the geometric jobs, measure each side and subtract its commanded length. Compare the errors with 0.5 mm and inspect the unequal triangle sides for geometric asymmetry.

Results or measurements

Table 16 retains all seven readings. Mean absolute square-side error is $(0.30 + 0.20 + 0.10 + 0.10)/4 = 0.175$ mm. The largest difference is $|56.20 - \sqrt{40^2 + 40^2}| = 0.369$ mm. Figure 12 shows the actual specimen beside its commanded geometry. Figure 43 and the triangle diagnostic are retained in Appendix C.9.

Shape	Feature	Nominal / mm	Measured / mm	Error / mm
Square	Bottom X	100.00	100.30	+0.30
Square	Top X	100.00	100.20	+0.20
Square	Left Y	100.00	100.10	+0.10
Square	Right Y	100.00	99.90	-0.10
Triangle	Base	80.00	80.30	+0.30
Triangle	Left slope	56.57	56.20	-0.37
Triangle	Right slope	56.57	56.90	+0.33

Table 16. Seven measured feature lengths after calibration.

Observations

Every measured feature length is within 0.5 mm of nominal. The unequal triangle slopes suggest geometric asymmetry, but these measurements do not separate rail alignment, scale, pen width and reading uncertainty.

Statistical analysis

There is one specimen of each shape, so variation between sides is not repeated-draw variability. Length checks leave translation undetected. These results support dimensional accuracy, while absolute bed coordinates and repeated-word overlays remain separate checks.

Qualification test 6: Storage and use of multiple trained styles (R6)

Objectives of the test or experiment

Confirm at least ten represented writers, stored glyph mappings and the ability to use those mappings for reproduction.

Equipment used

The evaluated eleven-class writer checkpoint, stored style profiles and glyph libraries, saved generated jobs and fourteen physical output pages.

Test setup and experimental parameters

The required count is ten. The evaluated set is 150, 152, 153, 384, 552, 588, A, B, O, T and Y. Both class identities and stored trajectory libraries are checked.

Steps followed in the test or experiment

Compare the classifier mapping with the stored profiles, then confirm that every identity occurs in the writer-validation records, physical pages and generated jobs.

Results or measurements

Eleven identities have classifier entries and stored profiles. All have physical samples in Figure 27 and correct pooled identities in QT 2, exceeding the required count by one.

Observations

The result demonstrates concurrent use of multiple stored styles. QT 1 measures their different reproduction quality.

Statistical analysis

This is an inventory count, so a sampling interval is inapplicable. R6 is met by the evaluated eleven-style set.

4.2.2 Qualification under the proposal's field conditions

The indoor captures and physical drawings demonstrate the fixed bench arrangement. The capture variations in Section 3.4.7 support operation under different photographed conditions. A calibrated illuminance and vibration series is not present, so no numerical coverage of the full field-condition range is assigned.

5. Discussion

5.1 Critical evaluation of the design

The evaluation relates the measured performance to the Design choices. The main trade-offs concern variation between writing styles and the time needed for embedded classification.

5.1.1 Interpretation of results

Useful reproduction is demonstrated, with a clear operating range. Seven writers exceed the 85% character target, with four above 93%. The pooled 84.41% score shows that the main challenge is consistency across styles. Figure 28 makes the difference visible in actual output. The association between separated original characters and better reproduction ($\rho = 0.845$) supports investigating connected handwriting, but does not isolate joining from input quality or glyph coverage.

Writer identity survives on 86.41% of physical lines and all fourteen page pools. Geometric shape correlation is lower, at 0.697. Slant is strongly retained ($r = 0.991$), while word spacing is weaker ($r = 0.486$). These results favour improving context and layout while preserving the useful glyph geometry. The writer classifier also helps select candidates, so its score alone is not an independent judgement of style. The geometric comparison adds a separate measure, while human reading and style assessment would extend both automated tests.

The classifier results support the chosen architecture. The joint reader's 94.06% character accuracy is 0.94 percentage points below target. Its public and personal subsets score 94.12% and 93.33%. This supports line recognition across both inputs, although the experiment does not isolate errors introduced by segmentation.

Writer identification reaches 98.08%, with every writer group correct after pooling. Stroke normalisation makes the test less dependent on original pen thickness. However, an error at 83.8% confidence shows that confidence is not a reliable identity guarantee. The personal line split also shares capture conditions with training. A new-session evaluation would test deployment more strongly.

The physical timing result is a practical success. All 25 completed ODROID jobs meet 4 s per character, including generation and drawing. This is stronger evidence than nominal feed calculations because it includes pen changes and execution overhead. The worst short request leaves 5.39 s of margin. The measured range starts at 43 characters, so shorter requests need their own test. Repeated 378-character jobs also vary substantially, consistent with different path lengths and pen transitions.

The classification-time gap is measurable and actionable. The two classification requests exceed their allowances by 31.76 and 28.94 s. Networks contribute about two thirds of the combined time. The host comparison demonstrates a 2.76-fold network improvement with unchanged reader text, but transferring that factor still leaves both projected totals above target. Network acceleration and image-stage optimisation therefore need to be addressed together.

Halving image resolution is unsuitable because it changes the detected lines and decoded text. A useful optimisation must retain the reference output as well as reduce runtime.

Mechanical accuracy is promising, with a precise distinction in what was measured. All seven measured lengths are within 0.5 mm of nominal, supporting the calibrated belt drive. The unequal triangle slopes suggest a possible alignment effect. The apparent 0.706° diagnostic in Appendix C.9 is conditional on the assumed height and does not independently measure rail squareness.

A translated drawing can retain correct side lengths. Repeated bed-coordinate points and word overlays would therefore extend the present dimensional check to absolute placement and repeatability. This builds on the functioning motion system rather than requiring an unsupported redesign.

Relationship to published approaches. The reader follows the CNN, BiLSTM and CTC approach discussed by Kızılırmak and Yanıkoğlu [12, pp. 4–5]. Its 5.94% character error supports the architecture, but different partitions and decoding prevent a controlled ranking against published scores.

Haines et al. support example-based synthesis from a small collection of pages [23, p. 4]. Graves's sequence generator instead requires ordered pen coordinates [3, p. 18], which are absent from the project's photographs. One-DM uses a single reference at generation time, but still requires a separate training corpus [22, p. 9]. The glyph approach suits the available offline data and makes geometry directly controllable, with limited contextual variation as its main compromise.

5.1.2 Critical evaluation

Separating the implementation contribution from checkpoint selection. The first-principles implementation covers the forward passes, derivatives, CTC and optimiser updates. Table 7 compares its trained models with the reference route on common validation data. The first-principles writer reaches 95.51% and the reader reaches 94.39%, supporting the numerical implementation independently of the selected system configuration.

The supplied logs in Appendix C.13 record approximately 19 to 23 minutes per writer epoch and 111 to 122 minutes per reader epoch. Float32 was chosen to limit training cost, but no controlled experiment attributes the model-score differences to precision. The 155-line training validation and 156-line re-extraction are also kept separate.

Line recognition addresses a real weakness in character isolation. The proposal described classifying separated characters or connected groups. The final reader recognises whole lines, then uses known-transcript CTC alignment to recover character boundaries for profiles. This avoids making uncertain character cuts a prerequisite for reading [14, pp. 2–3]. The compromise is a larger sequence model and continued dependence on correct line ownership.

The glyph library fits the data and exposes its own limitations. Stored glyphs allow size, slant, spacing and machine coordinates to be inspected and changed directly. Eleven profiles demonstrate the approach on real output. A finite library, however, cannot reproduce every change caused by neighbouring characters. Multi-character units or context-specific variants

would address this more directly than simply increasing candidate count. Their extraction would need enough reliable examples.

Off-device training and embedded inference keep model preparation separate from operation. Batched numerical kernels or compiled hot loops are plausible speed improvements compatible with the numerical contribution. Their benefit and full timing compliance still require embedded measurements.

Economic feasibility. The current build estimate is approximately R4 000. The ODROID was borrowed at no purchase cost and will be returned. A repeat build must include its replacement cost. Later purchases and estimated training electricity total R750, but may overlap the approximate estimate, so they are not added as an exact revised total. The software speed experiment gives a measured improvement opportunity before committing to a more expensive computer.

5.1.3 Unsolved problems

The main reproduction problem is variation between writers. A controlled comparison of current glyphs and multi-character units, using fixed texts and models, would test the context explanation. Matching original and reproduced sentences would also strengthen the style comparison.

The timing priority is to preserve the reference output while profiling and optimising ODROID hot loops. Mechanical work should concentrate on repeated coordinates, frame alignment and pen-holder stiffness. A shared pulse scheduler and controlled acceleration are reasonable extensions, with their value assessed on completed drawings.

5.1.4 Strong points of the design

The complete chain captures handwriting, reads text, identifies writers, stores paths and produces physical writing. Eleven styles are exercised, seven exceed the reproduction character target and every validation writer group is identified correctly.

A path can be followed from source crop through glyph extraction, placement and G-code. This makes faults easier to locate and allows individual stages to be improved. Numerical operation checks and the trained-model comparison support the first-principles contribution alongside the physical prototype.

5.1.5 Expected failure conditions

Strong cursive connections, missing character variants and text beyond the available alphabet can weaken reproduction. Large layouts can exceed the bed, making preview inspection important before drawing.

Touching lines, page curvature and perspective can defeat preparation even when the reader works on clean crops. The writer classifier always selects a known identity, so an unfamiliar writer can receive a confident label. Page pooling adds evidence but cannot recover information missing from every line.

The gantry tracks open-loop steps. A missed step changes the real position without changing its counter. Limit switches and abort handling reduce operational risk, but depend on working electronics and software. The missing pen-motor flyback diode remains a circuit limitation described in Design. Requests outside the tested lengths have no demonstrated timing envelope.

5.2 Considerations in the design

5.2.1 Ergonomics

The browser combines capture review, text entry, writer selection and preview. The operator positions the paper and pen, then checks restraint, calibration and clearance. Separate generation and execution states make job progress clear.

5.2.2 Health and safety

Belts, pulleys and the carriage create pinch points, so the working area must remain clear. End stops and aborts assist operation. Independent motor-power removal would protect against an unresponsive controller. Driver temperature and restrained wiring also require attention.

The 24 V motor and 3.3 V logic rails are separated from the purchased supplies' mains inputs. A reverse-biased diode across the pen motor would provide an inductive-current path when switching off [35]. It is a recommended improvement, not a fitted component.

5.2.3 Environmental impact

The borrowed ODROID will be returned. Motors, rails and drivers can be reused, and printed parts replaced individually. Digital previews reduce wasted paper, while physical tests remain necessary. The printed material and training energy were not measured, so no specific recycling or environmental-saving figure is claimed.

5.2.4 Social and legal impact

Personalised writing could assist someone who has difficulty writing, although user benefit is not evaluated here. Style reproduction also permits impersonation. Permission and clear identification of machine-produced material are therefore important.

Signed forms are retained securely for all five contributors. Handwriting and profiles require collection, retention and access controls under the Protection of Personal Information Act [43, pp. 8–9, 11]. IAM data must also be used under its applicable terms. The prototype is intended for a controlled local network.

5.2.5 Ethics clearance

Participant consent is retained. Institutional clearance or exemption remains to be confirmed.

6. Conclusion

6.1 Summary of the work completed

A handwriting robot was designed and built to recognise photographed text, identify a known writer and reproduce new digital text in that writer's style. The system combines page conditioning, line segmentation, neural recognition, stored glyph trajectories and a calibrated Cartesian gantry. Eleven writer profiles are stored and exercised in physical reproduction.

6.2 Summary of observations and findings

The selected reader achieves 94.06% character accuracy and writer identification reaches 98.08% on the recorded validation set. Physical reproduction gives 84.41% read-back character accuracy, with seven writers above 85%. Writer identity is retained on 86.41% of reproduced lines and all fourteen page pools. The measured shape correlation is 0.697, showing that recognisable identity does not imply complete geometric similarity.

All 25 completion-confirmed ODROID reproduction jobs satisfy 4 s per character. The two measured classification requests exceed their time allowances. Every measured square and triangle length is within 0.5 mm of nominal, with a maximum difference of 0.37 mm. These results establish dimensional performance on the specimens, while absolute placement and repeatability need further qualification.

6.3 Contribution

The project demonstrates a complete path from offline handwriting samples to physical multi-writer output. Its first-principles network implementation is supported by numerical checks and trained models with scores close to the reference route. Explicit glyphs and G-code make the relationship between image features and pen movement visible, allowing each stage to be inspected and improved.

6.4 Suggestions for future work

The first priority is faster embedded classification without changing the detected lines or decoded text. Batching and compiled numerical kernels should be measured on the ODROID. Context-specific glyphs and multi-character units should then be tested on the more connected writers. Repeated bed-coordinate measurements, word overlays and matched original-to-reproduced sentences would strengthen the mechanical and stylistic evaluation. Improvements to motor protection and pulse scheduling would complete the next hardware revision.

7. References

[1] A. Chen, “Handwritten text classification based on convolutional neural network,” *Highlights in Science, Engineering and Technology*, vol. 34, pp. 39–44, 2023.

[2] J. Ma, “A study on a hybrid CNN-RNN model for handwritten recognition based on deep learning,” in *Proc. 1st Int. Conf. Data Analysis and Machine Learning (DAML)*, 2023, pp. 268–273.

[3] A. Graves, “Generating sequences with recurrent neural networks,” arXiv preprint arXiv:1308.0850, 2013. [Online]. Available: https://arxiv.org/pdf/1308.0850

[4] U.-V. Marti and H. Bunke, “The IAM-database: an English sentence database for offline handwriting recognition,” *Int. J. Document Analysis and Recognition*, vol. 5, no. 1, pp. 39–46, 2002.

[5] G. R. Hemanth, M. Jayasree, S. K. Venii, P. Akshaya, and R. Saranya, “CNN-RNN based handwritten text recognition,” *ICTACT Journal on Soft Computing*, vol. 12, no. 1, pp. 2457–2463, Oct 2021.

[6] N. Otsu, “A threshold selection method from gray-level histograms,” *IEEE Transactions on Systems, Man, and Cybernetics*, vol. 9, no. 1, pp. 62–66, 1979.

[7] D. Bradley and G. Roth, “Adaptive thresholding using the integral image,” *Journal of Graphics Tools*, vol. 12, no. 2, pp. 13–21, 2007. [Online]. Available: https://people.scs.carleton.ca/~roth/iit-publications-iti/docs/gerh-50002.pdf

[8] W. Postl, “Detection of linear oblique structures and skew scan in digitized documents,” in *Proc. 8th Int. Conf. Pattern Recognition (ICPR)*, 1986.

[9] A. Vinciarelli and J. Luettin, “A new normalization technique for cursive handwritten words,” *Pattern Recognition Letters*, vol. 22, no. 9, 2001.

[10] S. Avidan and A. Shamir, “Seam carving for content-aware image resizing,” *ACM Transactions on Graphics*, vol. 26, no. 3, pp. 10:1–10:9, 2007. [Online]. Available: https://www.cs.jhu.edu/~misha/ReadingSeminar/Papers/Avidan07.pdf

[11] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, “Gradient-based learning applied to document recognition,” *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998. [Online]. Available: https://leon.bottou.org/papers/lecun-98h

[12] F. Kizilirmak and B. Yanikoglu, “CNN-BiLSTM model for English handwriting recognition: comprehensive evaluation on the IAM dataset,” arXiv preprint arXiv:2307.00664, 2023. [Online]. Available: https://arxiv.org/pdf/2307.00664

[13] B. Shi, X. Bai, and C. Yao, “An end-to-end trainable neural network for image-based sequence recognition and its application to scene text recognition,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 39, no. 11, pp. 2298–2304, 2017.

[14] A. Graves, S. Fernández, F. Gomez, and J. Schmidhuber, “Connectionist temporal classification: labelling unsegmented sequence data with recurrent neural networks,” in *Proc. 23rd Int. Conf. Machine Learning (ICML)*, 2006, pp. 369–376. [Online]. Available: https://www.cs.toronto.edu/~graves/icml_2006.pdf

[15] S. Ioffe and C. Szegedy, “Batch normalization: accelerating deep network training by reducing internal covariate shift,” in *Proc. 32nd Int. Conf. Machine Learning (ICML)*, 2015, pp. 448–456. [Online]. Available: https://proceedings.mlr.press/v37/ioffe15.pdf

[16] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, “Dropout: a simple way to prevent neural networks from overfitting,” *Journal of Machine Learning Research*, vol. 15, no. 56, pp. 1929–1958, 2014. [Online]. Available: https://jmlr.org/papers/volume15/srivastava14a/srivastava14a.pdf

[17] C. Wigington, S. Stewart, B. Davis, B. Barrett, B. Price, and S. Cohen, “Data augmentation for recognition of handwritten words and lines using a CNN-LSTM network,” in *Proc. 14th IAPR Int. Conf. Document Analysis and Recognition (ICDAR)*, vol. 1, 2017, pp. 639–645.

[18] M. Bulacu and L. Schomaker, “Text-independent writer identification and verification using textural and allographic features,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 29, no. 4, pp. 701–717, 2007.

[19] S. He and L. Schomaker, “FragNet: writer identification using deep fragment networks,” *IEEE Transactions on Information Forensics and Security*, vol. 15, pp. 3013–3022, 2020.

[20] A. A. Brink, J. Smit, M. L. Bulacu, and L. R. B. Schomaker, “Writer identification using directional ink-trace width measurements,” *Pattern Recognition*, vol. 45, no. 1, pp. 162–171, 2012. [Online]. Available: https://www.ai.rug.nl/~mbulacu/brink-quill-pr2012.pdf

[21] L. Kang, P. Riba, Y. Wang, M. Rusiñol, A. Fornés, and M. Villegas, “GANwriting: content-conditioned generation of styled handwritten word images,” in *Computer Vision – ECCV 2020*. Springer, 2020, pp. 273–289.

[22] G. Dai, Y. Zhang, Q. Ke, Q. Guo, and S. Huang, “One-DM: one-shot diffusion mimicker for handwritten text generation,” in *Computer Vision – ECCV 2024*, ser. Lecture Notes in Computer Science. Springer, 2024. [Online]. Available: https://www.ecva.net/papers/eccv_2024/papers_ECCV/papers/07598.pdf

[23] T. S. F. Haines, O. Mac Aodha, and G. J. Brostow, “My text in your handwriting,” *ACM Transactions on Graphics*, vol. 35, no. 3, pp. 26:1–26:18, 2016. [Online]. Available: https://reality.cs.ucl.ac.uk/projects/haines/haines16handwriting.pdf

[24] O. Teikari, “Reproducing handwriting with position-controlled robots,” Master’s thesis, Tampere University of Technology, Tampere, Finland, 2016.

[25] D. H. Douglas and T. K. Peucker, “Algorithms for the reduction of the number of points required to represent a digitized line or its caricature,” *Cartographica*, vol. 10, no. 2, pp. 112–122, 1973.

[26] T. Y. Zhang and C. Y. Suen, “A fast parallel algorithm for thinning digital patterns,” *Communications of the ACM*, vol. 27, no. 3, pp. 236–239, 1984. [Online]. Available: https://agcggs680.pbworks.com/f/Zhan-Suen_algorithm.pdf

[27] IQS Directory, “Types of linear actuators,” IQS Directory Technical Library, 2024, [Online]. Available: https://www.iqsdirectory.com/articles/linear-actuator/types-of-linear-actuators.html.

[28] "NEMA 17 bipolar stepper motor, 1.68 A per phase, 200 full steps per revolution," Manufacturer datasheet.

[29] T. Dziedzic, A. Sen Yilmaz, and A. Szafrańska, "How fast is fluent handwriting?" *Forensic Science International*, vol. 383, p. 112896, Jun 2026.

[30] P. P. Acarnley, *Stepping Motors: A Guide to Theory and Practice*, 4th ed. London, U.K.: Institution of Electrical Engineers, 2002.

[31] Allegro MicroSystems, "A4988 DMOS microstepping driver with translator and overcurrent protection," Datasheet, rev. 8, 2022, [Online]. Available: https://www.allegromicro.com/-/media/files/datasheets/a4988-datasheet.pdf.

[32] Phipps Electronics, "GA12-N20-30 3 V, 150 rpm metal geared motor specifications," Product specification, 2024, [Online]. Available: https://www.phippselectronics.com/support/ga12-n20-metal-geared-motor-specifications/?seq_no=2 [Accessed: Oct. 31, 2026].

[33] Hardkernel, "ODROID-N2+ single-board computer," Hardware technical reference, [Online]. Available: https://www.hardkernel.com/shop/odroid-n2-with-4gbyte-ram-2/ [Accessed: Oct. 31, 2026].

[34] ON Semiconductor, "BC337/BC337-40 NPN general-purpose amplifier transistor," Datasheet, [Online]. Available: https://www.onsemi.com/download/data-sheet/pdf/bc337-fsc-d.pdf.

[35] ROHM Semiconductor, "NPN transistor: Low-side switch fundamentals," TechWeb, Transistors Basic, 2026, 27 February 2026; accessed 1 November 2026. [Online]. Available: https://techweb.rohm.com/product/transistors-diodes/transistors/27354/

[36] Communica, "DML31P sub-miniature microswitch," Product listing, [Online]. Available: https://www.communica.co.za/products/dml31p [Accessed: Oct. 31, 2026].

[37] J. E. Bresenham, "Algorithm for computer control of a digital plotter," *IBM Systems Journal*, vol. 4, no. 1, pp. 25–30, 1965. [Online]. Available: https://janmr.com/files/papers/bresenham65.pdf

[38] R. M. Haralick, S. R. Sternberg, and X. Zhuang, "Image analysis using mathematical morphology," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 9, no. 4, pp. 532–550, 1987. [Online]. Available: https://www.haralick.org/journals/04767941.pdf

[39] A. Rosenfeld and J. L. Pfaltz, "Sequential operations in digital picture processing," *Journal of the ACM*, vol. 13, no. 4, pp. 471–494, 1966. [Online]. Available: https://www.cs.virginia.edu/~jlp/66.sequential.op.pdf

[40] K. Chellapilla, S. Puri, and P. Simard, "High performance convolutional neural networks for document processing," in *Proc. 10th Int. Workshop on Frontiers in Handwriting Recognition (ICFHR)*, 2006.

[41] PyTorch contributors, "CTCLoss: reduction and alignment-loss conventions," PyTorch documentation, accessed 1 November 2026. [Online]. Available: https://docs.pytorch.org/docs/2.15/generated/torch.nn.CTCLoss.html

[42] E. B. Wilson, “Probable inference, the law of succession, and statistical inference,” *Journal of the American Statistical Association*, vol. 22, no. 158, pp. 209–212, 1927.

[43] Republic of South Africa, “Protection of personal information act 4 of 2013,” 2013, official consolidated text. [Online]. Available: https://www.justice.gov.za/legislation/acts/2013-004.pdf [Accessed: 1 November 2026].

[44] T. Tieleman and G. Hinton, “Lecture 6.5—RMSprop: divide the gradient by a running average of its recent magnitude,” COURSERA: Neural Networks for Machine Learning, 2012. [Online]. Available: https://www.cs.toronto.edu/~tijmen/csc321/slides/lecture_slides_lec6.pdf

[45] D. P. Kingma and J. Ba, “Adam: a method for stochastic optimization,” in *Proc. 3rd Int. Conf. Learning Representations (ICLR)*, 2015. [Online]. Available: https://arxiv.org/pdf/1412.6980

[46] I. Loshchilov and F. Hutter, “Decoupled weight decay regularization,” in *Proc. 7th Int. Conf. Learning Representations (ICLR)*, 2019. [Online]. Available: https://arxiv.org/pdf/1711.05101

[47] R. Pascanu, T. Mikolov, and Y. Bengio, “On the difficulty of training recurrent neural networks,” in *Proc. 30th Int. Conf. Machine Learning (ICML)*, 2013, pp. 1310–1318.

[48] Joint Committee for Guides in Metrology, “Evaluation of measurement data—guide to the expression of uncertainty in measurement,” JCGM 100:2008, 2008.

A. Supporting implementation calculations

The main Design section explains the engineering decisions and representative operations. This appendix records the derivative conventions, image operations and parameter meanings needed to inspect the numerical implementation in more detail. Algorithms in the main text are explanatory pseudocode; the separately submitted source remains the executable implementation.

A.1 Numerical network derivatives

Applying the backpropagation chain rule [11, pp. 3–4] to a linear layer $Y = XW^T + b$ with incoming derivative G, the implemented derivatives are $\partial\mathcal{L}/\partial X = GW$, $\partial\mathcal{L}/\partial W = G^T X$ and the sum of G over samples for the bias. Convolution applies the corresponding matrix calculation to overlapping image patches as described in Equation 45. Contributions to the same input pixel are summed.

For each batch-normalisation channel, $N = BHW$ values contribute to the batch statistics. The training forward pass uses population variance, while the running variance update uses its $N/(N-1)$ corrected estimate. The running update momentum is 0.1. With $\widehat{x} = (x-\mu)/\sqrt{\sigma^2+\varepsilon}$ and $g = \partial\mathcal{L}/\partial y$, Equation 39 gives the input derivative, obtained by simplifying the batch-normalisation chain-rule expressions [15, p. 4].

$$\frac{\partial\mathcal{L}}{\partial x_i} = \frac{\gamma}{N\sqrt{\sigma^2+\varepsilon}}\left[Ng_i - \sum_j g_j - \widehat{x}_i \sum_j g_j \widehat{x}_j\right]. \tag{39}$$

The parameter derivatives accompanying Equation 39 are $\sum_i g_i \widehat{x}_i$ for γ and $\sum_i g_i$ for β. Inference uses stored statistics and requires no batch-dependent update.

ReLU passes an incoming derivative only where its cached input was positive. Spatial max pooling scatters it to the selected maximum in each window. Height-max pooling does the same along height; width averaging distributes the derivative equally over the pooled columns. The implementation uses inverted dropout: it multiplies by the cached Bernoulli mask divided by $1-p$ during training and is the identity at inference. This moves the expected-output scaling from test time to training time relative to the original presentation [16, pp. 1931, 1934]. Reusing the same mask in the backward calculation is essential.

The LSTM backward pass differentiates the implemented recurrence in Equation 24; full back-propagation through time for this cell form is described by Graves [3, pp. 4–5]. It carries both the hidden derivative from the next step and a separate cell derivative. Writing the combined hidden derivative as dh_t, Equation 40 shows the dependencies before gate-activation derivatives are applied.

$$\begin{aligned} dc_t &= dc_t^{\text{next}} + dh_t \odot o_t \odot [1 - \tanh^2(c_t)], \\ do_t &= dh_t \odot \tanh(c_t), & df_t &= dc_t \odot c_{t-1}, \\ di_t &= dc_t \odot g_t, & dg_t &= dc_t \odot i_t, \\ dc_{t-1} &= dc_t \odot f_t. \end{aligned} \tag{40}$$

Sigmoid gates multiply these derivatives by $a(1-a)$; the candidate multiplies by $1-g_t^2$. The concatenated gate derivatives accumulate outer products with x_t and h_{t-1} into the two weight matrices. Multiplication by W_h provides the next reverse-time hidden derivative in Equation 40.

Each bidirectional layer reverses the appropriate sequence and sums the two input derivatives; stacked layers are differentiated in reverse layer order.

The numerical CTC forward score uses the recurrence in Equation 41, written in log space from the CTC path recursion [14, p. 4].

$$\alpha_{t,s} = \log p_t(\ell'_s) + \mathrm{LSE}_{r\in\mathcal{P}(s)}\,\alpha_{t-1,r}. \tag{41}$$

For frame 11 and the V state in the real Owen line, the predecessor log-forward scores are −12.8882643, −8.9807719 and −0.1088294, and the log emission is −0.01988985. Equation 41 gives $\alpha = -0.01988985 + \log(e^{-12.8882643} + e^{-8.9807719} + e^{-0.1088294}) \simeq -0.12857619$. This sums the admissible predecessor mass, whereas the main-text forced alignment selects one predecessor.

The predecessor set in Equation 41 contains the current state, the preceding state and an admissible two-state skip. A skip is excluded when its destination is blank or repeats the label two states earlier. With log scores a, b, the stable pairwise reduction is $m + \log(\exp(a-m) + \exp(b-m))$, where $m = \max(a,b)$; the all-unreachable case remains $-\infty$. The backward recurrence uses successor emissions at $t+1$, yielding the occupancy convention in Equation 26.

A.2 Optimisers and gradient scaling

The reader uses an uncentred RMSprop accumulator, following the moving-square-gradient principle [44, p. 29]. The code selects its own averaging coefficient and adds the stated decay term. For parameter θ, raw gradient g, learning rate η and weight-decay coefficient λ_w, the implemented update is Equation 42.

$$g' = g + \lambda_w\theta, \quad v_t = 0.99v_{t-1} + 0.01(g')^2, \quad \theta_t = \theta_{t-1} - \eta\,\frac{g'}{\sqrt{v_t} + 10^{-8}}. \tag{42}$$

Decay in Equation 42 enters the gradient before accumulation. The writer's AdamW instead uses separate parameter decay, as shown in Equation 43 [45, p. 2], [46, p. 3].

$$\begin{aligned} m_t &= 0.9m_{t-1} + 0.1g, & v_t &= 0.999v_{t-1} + 0.001g^2, \\ \widehat{m}_t &= m_t/(1-0.9^t), & \widehat{v}_t &= v_t/(1-0.999^t), \\ \theta_t &= \theta_{t-1} - \eta\left(\frac{\widehat{m}_t}{\sqrt{\widehat{v}_t} + 10^{-8}} + \lambda_w\theta_{t-1}\right). \end{aligned} \tag{43}$$

Following global gradient-norm clipping [47], before the writer update all gradient tensors are scaled together by $\min(1, 5/\|g\|_2)$, with a numerical denominator guard. Clipping individual tensor norms independently would be a different operation. The numerical reader loop does not apply that clipping. Equations 42 and 43 describe code operations, not a claim that their default learning rates were used in every completed run.

A.3 Morphology, thinning and path simplification

For binary mask M and neighbourhood K, dilation is the maximum of translated mask values and erosion is their minimum. Closing applies dilation followed by erosion with the same footprint [38, pp. 533, 541]. For a binary mask and symmetric flat footprint, the set operations correspond to neighbourhood maximum and minimum. The implementation obtains rectangular neighbourhood counts through prefix sums; binary comparisons then implement these operations.

Connectivity analysis uses horizontal runs and union operations with diagonal overlap admitted, rather than a library component-labelling call.

The thinning conditions follow Zhang and Suen [26, p. 237]. The method inspects the eight neighbours $P_2, \ldots, P_9$ clockwise from north. A removable foreground pixel has two to six foreground neighbours and exactly one background-to-foreground transition around that ring. The two alternating subpasses additionally require, respectively,

$$\begin{aligned} P_2P_4P_6 = 0,\ P_4P_6P_8 = 0 \quad &\text{(first subpass)},\\ P_2P_4P_8 = 0,\ P_2P_6P_8 = 0 \quad &\text{(second subpass)}. \end{aligned} \tag{44}$$

During thinning of the actual Owen crop, the first recorded deletion in subpass one was pixel $(y,x) = (12,190)$ with neighbours $(P_2, \ldots, P_9) = (0,0,1,1,1,1,0,0)$. Its neighbour count is four and its circular transition count is one. Both first-subpass products are zero because $P_2 = 0$ and $P_8 = 0$, so the pixel is deleted when the subpass mask is applied. The deletion is applied with the other eligible pixels in that subpass.

Candidates in Equation 44 are deleted together in each subpass, and the pair of subpasses repeats until unchanged. This reduces thickness while limiting topology changes; it cannot restore ink already lost during thresholding. The same thinning primitive supports stroke normalisation, glyph tracing and the diagnostic retracing route.

The default tracer begins at graph nodes whose degree differs from two, follows unused pixel edges, and then recovers remaining closed loops. Short dangling spurs are pruned, and nearby chain ends are greedily rejoined when their continuation angle is below 75°. This remains a geometric estimate rather than a verified natural handwriting order. Polyline simplification compares interior points with the line through the endpoints, retaining a point and splitting recursively when the deviation exceeds tolerance. The code adapts the polyline reduction method [25]. Near-closed paths are first split at a distant point so that a zero-length endpoint chord does not collapse the loop. The extraction tolerance is 0.6 source pixels before normalisation; final path smoothing and machine decimation occur in their own coordinate units.

A.4 Profile and trajectory quantities

Table 17 records the fields most likely to be confused during inspection. Full arrays and every tuning constant remain in the source rather than expanding the main text into a variable dictionary.

Raw alignment confidence is a path score used for relative filtering, not a calibrated probability of a correct cut. The cross-writer prior uses a median occupancy descriptor, then unit normalisation; its minimum pooled sample count is twelve. Writer consensus retains approximately 75% of a sufficiently populated pool, with at least three variants. Compatible prototypes group equal stroke counts, resample strokes to 44 points, align them and take robust pointwise summaries. Admission to the variant pool requires prior distance below 0.32 and improvement of more than 0.02 over the median raw distance. These gates reduce malformed geometry but do not prove that a distinctive form is incorrect.

Machine conversion maintains image, glyph, trajectory and machine coordinate frames separately. The line-centre shifts in Equation 35 are zero when no respacing is requested. Workspace fitting and clipping are distinct: fitting scales the whole trajectory uniformly, while clipping changes individual points and can deform letters. Final file parsing provides a useful conversion check

Quantity	Meaning and unit
`xHeightPx`	Reference source x-height in pixels; not a physical pen height
`strokes`	Ordered point lists in glyph x-height units; *y* increases upwards
`width`, `lead`, `advance`	Ink width, lead-in and cell/placement extent; distinct x-height quantities
`top`, `bot`	Vertical extrema relative to the baseline, in x-heights
`connL`, `connR`	Observed boundary crossings during source extraction
`priorD`	Descriptor distance from the pooled character reference; not writer confidence
`idealMedoid`, `idealGlyphs`	Actual representative variant and compatible median-stroke prototype
`letterAdvance`	Per-character median source advance; sampled variants retain their own placement geometry
`lineStepMm`	Synthesised baseline spacing before optional machine respacing
`jointScore`	Internal harmonic selection score; not independent stylistic correlation
`clipped`	Count of points clamped by the motion-file conversion; not an acceptable layout target

Table 17. Selected implementation fields and their interpretation.

because it reconstructs movements from emitted commands instead of directly displaying the input trajectory again.

A.5 Detailed training procedures

The main chapter retains the model, loss, design choices and representative checks. The following pseudocode records the detailed recurrences and update sequence. Algorithm 5 adapts the CTC forward–backward method [14, pp. 4–6]; Algorithm 6 assembles the project's training operations.

Algorithm 5 Numerical CTC loss and logit gradient

Require: Logits $z \in \mathbb{R}^{T\times B\times C}$ and encoded targets.

$\log p \leftarrow \mathrm{LogSoftmax}(z)$; $L \leftarrow 0$; $g \leftarrow 0$
for each batch item b **do**
  $\ell' \leftarrow$ target interleaved with blanks
  Initialise α, β to $-\infty$; admit first blank and first label
  for $t \leftarrow 1$ **to** $T-1$ **do**
    for each state s **do**
      $\alpha_{t,s} \leftarrow \log p_t(\ell'_s) + \mathrm{LSE}_{r\in\mathcal{P}(s)}\, \alpha_{t-1,r}$
    end for
  end for
  $\log Z \leftarrow$ LSE of the admissible final states
  Initialise final β to zero; recurse backward with next-frame emissions
  $q_{t,k} \leftarrow \sum_{s:\ell'_s=k} \exp(\alpha_{t,s} + \beta_{t,s} - \log Z)$
  $L \leftarrow L - \log Z$; $g_{t,b,k} \leftarrow (p_t(k) - q_{t,k})/B$
end for
return $L/B, g$

A.6 Expanded numerical calculation checks

For one image, let X_c contain flattened input patches, so the matrix formulation of convolution gives $Y = WX_c + b$ [40]. Applying the chain rule to $Y_{oj} = \sum_k W_{ok}(X_c)_{kj} + b_o$ gives $\partial\mathcal{L}/\partial W_{ok} = \sum_j G_{oj}(X_c)_{kj}$ and $\partial\mathcal{L}/\partial(X_c)_{kj} = \sum_o W_{ok}G_{oj}$. Equation 45 collects these derived sums in matrix form.

$$\frac{\partial\mathcal{L}}{\partial W} = GX_c^T, \qquad \frac{\partial\mathcal{L}}{\partial X_c} = W^T G. \tag{45}$$

Weight and bias derivatives are summed over the batch and output positions. Overlapping patch derivatives in Equation 45 are added back into the padded input, not overwritten. Max-pool derivatives return to the recorded maxima, width-average derivatives are distributed uniformly, and the LSTM backward pass accumulates both hidden-state and cell-state contributions through time. Appendix A records the remaining derivative conventions and optimiser calculations without repeating the architecture here.

For the traced Owen crop, the first convolution uses one 64×640 input and a 3×3 kernel. Its patch matrix contains $(1)(1)(3)(3)(64)(640) = 368640$ values, requiring $368640\times4/2^{20} = 1.40625$ MiB in float32. This determines one intermediate array's storage, not peak process memory. Later-layer arrays and any training gradients add storage; training and forward-only inference footprints must be assessed separately.

Algorithm 6 One numerical training batch and validation checkpoint decision

Require: Training batch, task, parameters θ and optimiser state.

$(X, y) \leftarrow$ task-specific preparation and training augmentation
$(F, \mathcal{K}) \leftarrow \text{BackboneForward}(X, \theta)$; retain caches
if task is reader **then**
    $(z, \mathcal{K}_h) \leftarrow \text{BiLSTMHead}(F)$; $(L, g) \leftarrow \text{CTC}(z, y)$
else
    $(z, \mathcal{K}_h) \leftarrow \text{WriterHead}(\text{MeanWidth}(F))$
    $(L, g) \leftarrow \text{CrossEntropy}(z, y)$
end if
$g_F \leftarrow \text{HeadBackward}(g, \mathcal{K}_h)$
$\nabla\theta \leftarrow \text{BackboneBackward}(g_F, \mathcal{K})$
if task is writer **then** clip total gradient norm to 5
$(\theta, \text{state}) \leftarrow \text{OptimiserStep}(\theta, \nabla\theta, \text{state})$
if epoch is complete **then**
    Validate with fixed preparation, stored normalisation and dropout disabled
    Update schedule; save progress; replace best only if its criterion improves
end if
return L, θ, state

Numerical perturbation provides a finite-difference sensitivity check [48, pp. 18–19]. For the loss function here, the central-difference expression is derived by subtracting the two Taylor expansions $\mathcal{L}(\theta \pm \varepsilon e_i) = \mathcal{L}(\theta) \pm \varepsilon \partial_i \mathcal{L} + \frac{1}{2}\varepsilon^2 \partial_{ii} \mathcal{L} + O(\varepsilon^3)$. Division by 2ε cancels the constant and even terms, giving Equation 46 with truncation error $O(\varepsilon^2)$ where the loss is sufficiently smooth. One logit or parameter is perturbed at a time.

$$g_i^{\text{num}} = \frac{\mathcal{L}(\theta + \varepsilon e_i) - \mathcal{L}(\theta - \varepsilon e_i)}{2\varepsilon}. \tag{46}$$

For the real Owen line's ▽ logit at frame 11, the two losses with $\varepsilon = 10^{-4}$ are $\mathcal{L}_+ = 0.4703364611173033$ and $\mathcal{L}_- = 0.4703403997759956$. Equation 46 gives $g^{\text{num}} = -0.0196932934615$, compared with the analytic CTC derivative -0.0196932934309; the absolute difference is 3.06×10^{-11}. This verifies that logit derivative on a real input without claiming a complete network-gradient or training-run test.

Here θ is a parameter vector, e_i selects one component and ε is a small perturbation. Equation 46 requires care at ReLU or max-pool boundaries and with stochastic dropout. The input or mask must be controlled so that the two loss evaluations test the same local operation.

A.7 Implementation and interface reference tables

These tables preserve supporting implementation distinctions and representation contracts. The main chapter explains the corresponding choices and transitions.

Aspect	First principles route	Framework route
Layer calculations	Explicit forward and backward operations	Framework modules and automatic differentiation
Architecture	Project's reader and writer definitions	Corresponding layer pattern and parameter names
Data preparation	Project crop, label and split functions	Uses the same preparation functions
Weights for inference	Named array archive	Export to the same named array layout
Verification role	Implementation being checked	Reference for matched numerical comparisons

Table 18. Roles of the numerical and framework implementations.

Quantity	Source	Use in synthesis
Slant	Projection concentration under trial shears	Reapplied to stored upright glyphs
X-height	Height of the estimated line body band	Normalises stored paths; physical x-height sets drawing size
Ascender/descender extent	Relative glyph heights	Vertical proportions and line spacing
Letter advance	Character-cell and ink geometry	Placement of following characters
Word spacing	Measured gaps in real line ink	Separation of words
Connectedness	Observed skeleton crossings between adjacent characters	Probability of eligible in-word joins
Glyph variants	Filtered normalised trajectories	Writer-specific letterforms and variation

Table 19. Principal stored style quantities and their roles.

Stage	Required input	Output and governing convention
Segmentation	Photographed or saved page	Ordered raw crops and full-page boxes; image y points down
Recognition	Line crops and model arrays	Text plus writer probabilities interpreted by the checkpoint mapping
Profile construction	Original lines, known transcripts and reader	Writer-keyed glyph paths in x-height units, baseline origin and y up
Synthesis	Text, stored profile, scale and search settings	Millimetre trajectory, source tags and internal selection scores
File conversion	Trajectory and workspace configuration	Absolute millimetre G-code, pen events, placement and timing budget
Physical execution	G-code, connected GPIO and completed calibration	Step-counter updates and job outcome; physical accuracy requires observation

Table 20. Representation boundaries that connect the three main processing stages to the gantry.

A.8 Motion-file conversion procedure

Algorithm 7 Trajectory placement and motion-file preparation

Require: Selected trajectory T and calibrated workspace configuration.

Measure bounds; determine available width and height after margins and offset
$a \leftarrow$ uniform fit scale; apply robust height cap when ruled fitting is enabled
Group paths by text row; determine ruled or free baseline shifts
$\mathcal{P} \leftarrow$ machine coordinates from Equation 35
Decimate short segments; optimise path order within rows
Clamp remaining out-of-bounds points; record their count
Emit absolute millimetre mode and initial pen state
for each path $p \in \mathcal{P}$ **do**
    Close eligible near-ring; join previous path if configured gap permits
    if a new pen-down path is needed **then**
        Raise pen; travel to start; lower pen; emit configured dwells
    end if
    Emit drawing moves with modal drawing feed; accumulate ink and travel budgets
end for
Raise pen; save file, placement and separate timing budgets
return file path and job metadata

A.9 Controller pin and supply planning

Header pin / GPIO line	Connection	Direction and nominal level	Notes / verification
USB host	ODROID USB host → IMX179 camera	USB data and bus power	Camera current draw and captured resolution are not established by the sensor's advertised maximum.
Ethernet / network	ODROID RJ45 or Wi-Fi ↔ laptop/browser	Network data; no GPIO voltage	Operator request and response path.
ODROID DC jack	Supplied adapter → board power input	Actual adapter output unknown	The manufacturer's 12 V/2 A recommendation is not confirmation of the adapter in use [33].
External supply +	24 V supply → VMOT on both A4988 carriers	24 V DC; supply limit set to 2.11 A	A separate 0.35 A supply indication was recorded under one operating condition; neither value is the motor phase-current setting.
3.3 V header rail	ODROID dedicated 3.3 V output → both A4988 VDD inputs and pen-motor positive rail	3.3 V supply	Physical header pin and total rail loading are not recorded.
Ground return	ODROID GND, driver logic/motor returns, switch returns and BC337 emitter	0 V reference	A common signal reference is required; exact return routing and physical GND pin are not documented.

Table 21. Controller pin, signal-level and supply map for the installed system.

Header pin / GPIO line	Connection	Direction and nominal level	Notes / verification
7 / 64	DIR1_PIN → X-driver DIR	Output, nominal 0/3.3 V	Direction state; software inversion is configured for X.
11 / 68	STEP1_PIN → X-driver STEP	Output pulses, nominal 0/3.3 V	A4988 advances on STEP pulses; this is a logic input, not the motor supply.
12 / 81	DIR2_PIN → Y-driver DIR	Output, nominal 0/3.3 V	Direction state; software inversion is configured for Y.
13 / 69	STEP2_PIN → Y-driver STEP	Output pulses, nominal 0/3.3 V	A4988 advances on STEP pulses; this is a logic input, not the motor supply.
15 / 72	X-min switch → X_MIN_PIN	Input, active high, nominal 0/3.3 V	Reported 10 kΩ pull-down.
16 / 65	X-max switch → X_MAX_PIN	Input, active high, nominal 0/3.3 V	Reported 10 kΩ pull-down.
18 / 66	Y-min switch → Y_MIN_PIN	Input, active high, nominal 0/3.3 V	Reported 10 kΩ pull-down.
22 / 67	Y-max switch → Y_MAX_PIN	Input, active high, nominal 0/3.3 V	Reported 10 kΩ pull-down.
33 / 70	Pen-up switch → PEN_SWITCH_PIN	Input, active high, nominal 0/3.3 V	High indicates the raised state. Switch model and pull-down location are not recorded.
35 / 71	PEN_MOTOR_PIN → 1 kΩ base resistor → BC337	Output, nominal 0/3.3 V	GPIO controls the transistor only; motor current is supplied by the dedicated 3.3 V rail. A 10 kΩ base pull-down was reported; its connection is not documented. No flyback diode is fitted.
A4988 outputs	X/Y carrier 1A/1B and 2A/2B → motor coil pairs	Switched/chopped winding drive from VMOT	The coil voltage is not a fixed 24 V STEP signal. Phase-current limit depends on VREF and sense resistance; actual VREF remains unmeasured [31, pp. 4–5, 9].

Table 21. Controller pin, signal-level and supply map (continued).

A.10 Worked values from the implemented system

A.10.1 Page conditioning and line segmentation

The recorded Owen01 trace uses a stored photograph reduced before analysis, producing an 800×600 pre-deskew working image. At pixel $(317, 343)$, $G = 100$. Its 41×41 background window has prefix values 20696850, 18404173, 18359673 and 16338366. Equation 15 gives $\Sigma = 271370$, hence $B_r = 271370/1681 = 161.43367$ and $R = 100/161.43367 = 0.619449$.

For that image, Otsu's criterion selects bin 171. Its class masses are 0.07069375 and 0.92930625, with mean bin indices 124.98356 and 217.71913, giving between-class variance approximately 564.97942. The corresponding ratio threshold is $(171 + 0.5)(1.2/256) = 0.80390625$. Since the example pixel's ratio is lower, it is retained as ink. These values describe the reduced calculation trace; the native-page examples below are separate runs.

In the Owen01 trace, the scores at +0.5°, +1.0° and +1.5° are 442973, 464105 and 439403. The largest score across all seventeen angles is 464105, so the selected rotation is +1.0°.

In the recorded boundary between centres 38 and 61, $y_m = 49.5$ and $h_g = 11.5$. At row 49 the closed mask is zero, giving $C = 0.012[(49 - 49.5)/11.5]^2 = 0.0000226843$. At column 283, the selected predecessor is row 50 with stored cost 0.00641966. Its one-row change adds 0.02, so the accumulated cost is $0.00641966 + 0.02 + 0.0000226843 \simeq 0.02644234$. This connects the local penalty to an actual step of the path search.

A separate sensitivity check varies the midpoint and step penalties over six fixed gaps from two pages. At the implemented 0.012/0.02 setting, mean midpoint offset is 0.363 working pixels; removing midpoint attraction raises it to 4.055 pixels. Every tested setting still crosses two raw-ink pixels across those seams. The check shows how the weights affect the path without establishing a unique optimum. The complete settings and outcomes are retained in Appendix B.6.

A.10.2 Network calculations

For the first Owen01 line, channel 0 of the first convolution at canvas row 34, column 462 gives $\sum WX = -1.150897$ with learned bias $b = -0.000435125$. Equation 22 therefore produces $z = -1.151332$. The nine actual input and learned-weight values are retained in Appendix B.7.

At the same activation, the checkpoint stores $\mu = 0.2163021$, $\sigma^2 = 0.02585394$, $\gamma = 1.0193636$ and $\beta = 0.1324657$. Using $\varepsilon = 10^{-5}$ in Equation 23 gives $y = 1.0193636(-1.1513321 - 0.2163021)/\sqrt{0.02585394 + 10^{-5}} + 0.1324657 \simeq -8.53619$. The following ReLU sets this negative response to zero. These are the stored inference statistics, rather than statistics calculated from this single line.

In the first forward LSTM layer for that line, frame 10 and cell 0 have $c_{t-1} = -2.7277257$, $f_t = 0.7306111$, $i_t = 0.6666368$ and $g_t = -0.8560218$. Equation 24 gives $c_t \simeq -2.5635625$. With $o_t = 0.000145395$, the cell emits $h_t \simeq -0.000143680$. This records the memory calculation used while processing the real feature sequence.

For the complete labelled line “1.9. Vloggers using fame to support charities is a positive way”, the recorded forward–backward total is $\log Z = -0.47033843$. Equation 25 gives $\mathcal{L}_{\mathrm{CTC}} = 0.47033843$ and $Z \simeq 0.6247908$. This loss was evaluated offline from the trained reader’s actual logits and known transcript, without updating its weights.

At frame 11, the probability of V is $p_{11,V} = 0.98030665$ and the target-constrained occupancy is $q_{11,V} = 0.99999995$. The CTC logit derivative is therefore $p - q = -0.01969329$. It is negative because the admissible target paths require more V probability than the current output supplies at that frame; the real-input finite-difference check in Appendix A.6 checks this derivative.

The same Owen crop has an estimated body-band height of 38 pixels. At the selected $\rho = 0.17$, the width calculation gives $w = \mathrm{clip}(\mathrm{round}(0.17 \times 38), 2, 9) = 6$, then $k = 2\lfloor 6/2 \rfloor + 1 = 7$. The actual redraw therefore uses a seven-pixel square dilation before fitting the line to the network canvas.

A.10.3 Glyph extraction and reproduction calculations

For the V in that Owen line, frame 11 corresponds to state 11 of the blank-interleaved transcript. Its log emission is −0.01988991 and the three admissible predecessor scores are −13.91888, −10.01139 and −1.3624485. Equation 28 selects their maximum and gives $V = -0.01988991 - 1.3624485 = -1.3823384$. Backtracking assigns V the frame interval $[11, 12)$.

The refined right boundary of this V is source column 193. Although smoothing gives $\overline{p}_{193} = 2.8$ and the positive-column reference is 14.36, the raw ink count at column 193 is zero. The explicit blank-column rule therefore overrides 2.8/14.36 and assigns zero cut cost. This is a real case in which the gap rule prevents smoothing alone from penalising a clean separation.

A traced point on this V is $(x, y) = (173, 59)$, with left cut $x_L = 146$, baseline $b = 107$, $h_x = 38$ and $\tan\phi = -0.0787017$. Equation 30 gives $v = 48/38 = 1.263158$ and $u_0 = 27/38 - 1.263158(-0.0787017) = 0.809939$. After the implementation’s three-decimal intermediate rounding and measured post-trim shift $\delta = 0.4472489$, the stored point is $(u, v) = (0.363, 1.263)$ and the recorded lead is 0.447 x-heights.

For unit-length descriptor v_g and the normalised median reference r_c for character c, the prior distance is $d(g, c) = 1 - v_g^T r_c$. Internal consensus instead uses pairwise L_1 descriptor distances and chooses the medoid with the lowest total distance. These compare a pooled character reference with this writer’s own forms. Dylan’s first stored a variant has recorded prior distance 0.114. Recomputing the pairwise rule on all nine final stored variants gives the smallest total distance, 33.3413, for variant index 4; the other totals range from 34.4306 to 42.9091. This demonstrates the selection rule on the final pool, without treating a new descriptor calculation as the historical filtering log. Algorithm 3 gives its place in profile construction.

In the generated Owen sentence “Writing with style.”, the first placement call, after calibration and jitter, uses $(u, v) = (-0.01134565, 1.47270173)$, lead $l = 0.25117988$, zero shear and drift, and $h_x^{\mathrm{mm}} = 4$. With the initial placement origin $(0, 0)$, Equation 32 gives $x = 4(0.25117988 - 0.01134565) = 0.9593369$ mm and $y = 4(1.47270173) = 5.8908069$ mm. This is the actual assembly point before later path processing and machine placement, not a replay of the earlier V crop.

The final Owen profile's stored glyph distances have median $\widetilde{d} = 0.18355$. Equation 33 gives $\lambda = \mathrm{clip}[(0.18355 - 0.24)/0.08, 0, 1] = 0$. With no stored override, this is the anchoring value selected by the automatic rule for the generated job.

For the recorded Owen candidate 1, the 19-character target "Writing with style." differs from its reader output by one substitution, giving $A = 18/19 = 0.9473684$. The selected-writer posterior is $P = 0.5786133$, so Equation 34 gives $J = 0.7184359$. This is the actual score used to select the candidate in Figure 24; it evaluates the software rendering rather than physical stylistic agreement.

In the worked Owen example, three pre-repair trials use the fixed text "Writing with style.", seeds 2026, 3003 and 3980, and the selected eleven-writer shape model with an explicit width fraction of 0.17. Candidate 1 has $(A, P, J) = (0.947, 0.579, 0.718)$, while candidate 3 has the same character score but $(P, J) = (0.464, 0.623)$. The combined criterion therefore selects candidate 1. Its reader output begins with a lower-case "w", which accounts for one edit rather than perfect text recovery. This is an inspectable scoring example before repair, not a claim that the full default search or the physical system attained those scores.

The generated Owen trajectory has $x_0 = 0.9593369$ mm and $y_1 = 6.6340616$ mm. A point $(2.9829210, 5.3303251)$ mm is placed using the configured origin $(0, 251.3)$ mm, margins $(15, 10)$ mm, unit fit scale and zero line shift. Equation 35 gives $(x_m, y_m) = (17.0235840, 239.9962635)$ mm. Three-decimal file emission writes this point as `X17.024 Y239.996`, connecting the trajectory calculation to the motion segment examined in Section 3.3.6.

For this same generated file, the unrounded distance accumulators are $D_d = 139.0263729$ mm and $D_u = 158.1555743$ mm. At $F_d = 900$ and $F_u = 2400$ mm/min, with 56 pen transitions and two 0.35 s dwell settings, Equation 36 gives $t_{\mathrm{nom}} = 9.2684249 + 3.9538894 + 19.6 = 32.8223142$ s. The job metadata reports 32.8 s after rounding. This is the converter's actual geometric budget, not a physical completion-time measurement.

Figure 26 applies the same budget calculation to a separate offline generation of "Writing with style." in four final profiles. The Owen job contains 58 pen transitions: its recorded components are 9.249 s drawing, 3.198 s travel and $58(0.35) = 20.3$ s dwell, giving 32.747 s in total. Thus dwell contributes about 62% of this nominal budget; across the four jobs it contributes approximately 61–65%. The different path and transition counts distinguish these jobs from the earlier worked file. The breakdown identifies a drawing-side optimisation opportunity, but shortening dwell would require a physical test that the pen still clears and settles reliably. These estimates exclude the execution costs stated above and are not additional R3 timing measurements.

A.10.4 System integration calculation

Applying this vote to the first two archived Owen01 crops gives ink weights 20204 and 21750, with Owen probabilities 0.9996674 and 0.9999279 from the selected 0.17-width model. Equation 37 gives $\overline{p}_{\mathrm{Owen}} = (20204 \times 0.9996674 + 21750 \times 0.9999279)/41954 \simeq 0.9998024$. This is a two-crop calculation trace, not the full-page evaluation reported in Results.

A.10.5 Hardware calculation detail

This keeps the GPIO current far below the motor current, but the GPIO must still be rated to source at least 2.6 mA. The BC337 datasheet's saturation test uses 50 mA of base current for 500 mA of collector current, a conservative forced-current-gain ratio of 10 [34]. Using that ratio as a design check, 2.6 mA of base current supports about 26 mA of saturated collector current. That is slightly below the listed 30 mA rated motor current. To provide the same conservative saturation drive at 30 mA would require $R_B \leq (3.3 - 0.7)/(0.030/10) \approx 870\,\Omega$; at the listed 230 mA stall current it would require $R_B \leq (3.3 - 0.7)/(0.230/10) \approx 110\,\Omega$. The latter would draw about 23 mA from the GPIO, so the 1 kΩ circuit is not guaranteed to saturate the transistor at rated or stall load. The 1 kΩ resistor is the prototype's GPIO-current-limiting choice. A suitable logic-level MOSFET or motor driver would avoid relying on this base-current margin if the installed motor approaches those loads.

Using the nominal 80 microsteps/mm, a 1.6 ms homing-step period corresponds to 7.8125 mm/s. The counted switch-to-switch traverses alone would then need $(200.6 + 261.3)/7.8125 \approx 59.1$ s; this excludes the initial searches, retractions and parking, and is not a measured calibration duration. The result is deliberately not stored between sessions, since a bump between sessions would silently invalidate it, and a failed calibration leaves movement unavailable until calibration succeeds. Algorithm 3 states the procedure.

The hand measurement of travel also sets the scale of every subsequent movement. An error δL scales every commanded position by $x\,\delta L/L$, so to keep that scale error below half of the 0.5 mm allowed by R5 over the 190.6 mm of usable X travel,

$$\delta L \leq \frac{0.25\text{ mm}}{190.6\text{ mm}} L = \frac{0.25}{190.6} \cdot 200.6\text{ mm} = 0.26\text{ mm} \quad (0.13\ \%), \tag{47}$$

and switch-trigger repeatability adds directly to the zero error on top of that. This is an allocation of half the positional-error allowance to scale error, not a measured uncertainty budget. Switch repeatability and mechanical compliance use part of the remaining allowance [48, pp. 18–19].

B. Subsystem visual evidence and independent software checks

This appendix retains the offline checks and figures that extend the Design explanations. No training or gantry motion was performed during report revision. The original inputs, scripts and recorded outputs are preserved in the separate supporting-evidence package. The integrated qualification tests remain in Results.

B.1 Actual capture variation and input resolution

The three camera-roll inputs in Figure 16 are `WIN_20261028_23_23_30_Pro.jpg`, `WIN_20261028_23_28_34_Pro.jpg` and `WIN_20261028_23_32_40_Pro.jpg`. Each stored image is 3264×2448 pixels. The page visibly contains three handwritten text rows. Automatic axis decisions were retained, including the failure in the third capture. The observed crop counts were 4, 3 and 28, respectively. Shadows are visually different, but neither lux levels nor physical angles were measured. These observations therefore explain a design limitation rather than qualifying operation across a specified lighting range.

The native Owen01 page (2448×3264) produced 20 crops for 20 labelled lines. A separate exploratory bench first fitted each of three original pages inside 1600×1600 pixels. From each reduced page it produced an unchanged view, a synthetic left-to-right multiplier from 0.42 to 1.00, a synthetic +3° rotation using bicubic interpolation on an expanded white canvas, and a quarter turn. Each was passed through the same automatic axis and segmentation functions. Table 22 retains every case, including unsuccessful ones.

Page	Label lines	Reduced	Gradient	+3°	90°
Owen	20	19	20	11	19
Dylan	24	24	24	25	24
Yeukita	26	26	26	26	26

Table 22. Exploratory crop counts after extra input reduction and digital perturbation. Equal counts do not establish correct ink ownership.

The reduced Owen case produced 19 crops and lost the usable first-line crop, whereas the native page produced 20. Its rotated reduced image produced only 11. The gradient image happened to restore the count to 20, which is not evidence that adding a shadow improves segmentation: thresholds, interpolation and component geometry changed together. These cases show why nominally scale-relative thresholds do not guarantee resolution invariance. A suitable follow-up would compare matched stroke ownership at controlled resolutions and improve capture guidance; the present bench does not supply pixel-level annotations or attribute every error to one operation.

B.2 Independent component and seam references

Algorithm 8 states the two independent reference comparisons. The run-based eight-connected component labeller was compared with an independently written flood-fill reference on 100 binary masks of size 9×11, using NumPy seed 2026 and foreground probabilities spanning

0.1–0.8. The test compared the full same-component relation between foreground pixel pairs, not arbitrary numerical label values. All 100 partitions agreed. Small random masks exercise diagonal contact and label equivalences, but do not demonstrate large-image speed or prove every possible topology.

For the seam solver, 20 random 4×6 cost arrays (seed 83) were checked against exhaustive enumeration of all row sequences with adjacent row changes of at most one. Both objectives included the implementation's $0.02|\Delta y|$ penalty. The maximum objective discrepancy was 0. The exhaustive reference tests optimal cost on those arrays; it does not establish that the chosen image-derived cost always follows the correct handwritten line boundary.

Algorithm 8 Independent partition and seam verification

Require: Fixed random seeds, mask cases and small seam-cost arrays.

for each binary mask M **do**
  $A \leftarrow \text{LabelComponents}(M)$; $B \leftarrow \text{FloodFill8}(M)$
  assert $[A_p = A_q] = [B_p = B_q]$ for every foreground pair (p, q)
end for
for each small cost array C **do**
  $p^* \leftarrow \text{LeastCostSeam}(C, 1)$
  $\mathcal{P} \leftarrow$ all paths with adjacent row differences at most one
  $E(p) \leftarrow \sum_x C(p_x, x) + 0.02 \sum_x |p_x - p_{x-1}|$
  Record $|E(p^*) - \min_{p \in \mathcal{P}} E(p)|$
end for
return partition agreements and seam-cost discrepancies

B.3 CTC probability and derivative check

Algorithm 9 gives the exhaustive probability and derivative procedure. The loss check used float64 logits of shape $5 \times 1 \times 4$, blank index 0 and target $[1, 1]$. All 1024 frame paths were enumerated. Consecutive repeats were collapsed before blanks were removed, and only paths yielding the target contributed to its probability. This independently exercises the repeated-label restriction that a single uninterrupted run of label 1 cannot encode two labels. The path sum gave negative log likelihood 3.875974036731708; the dynamic programme gave 3.875974036731709. The absolute difference was 4.44×10^{-16}.

All 20 logits were perturbed individually with central-difference step 10^{-5}. The maximum absolute discrepancy from `train.py::CtcLoss` was 3.63×10^{-11}. Logits were drawn after the fixed-seed component-mask draws in the bench; the script preserves that order. These checks concern the implemented CTC loss and derivative on a finite feasible target. They do not test impossible alignments, every transcript length, trained recognition accuracy or the full CNN–LSTM backward pass.

B.4 Stored profiles, emitted files and physical page

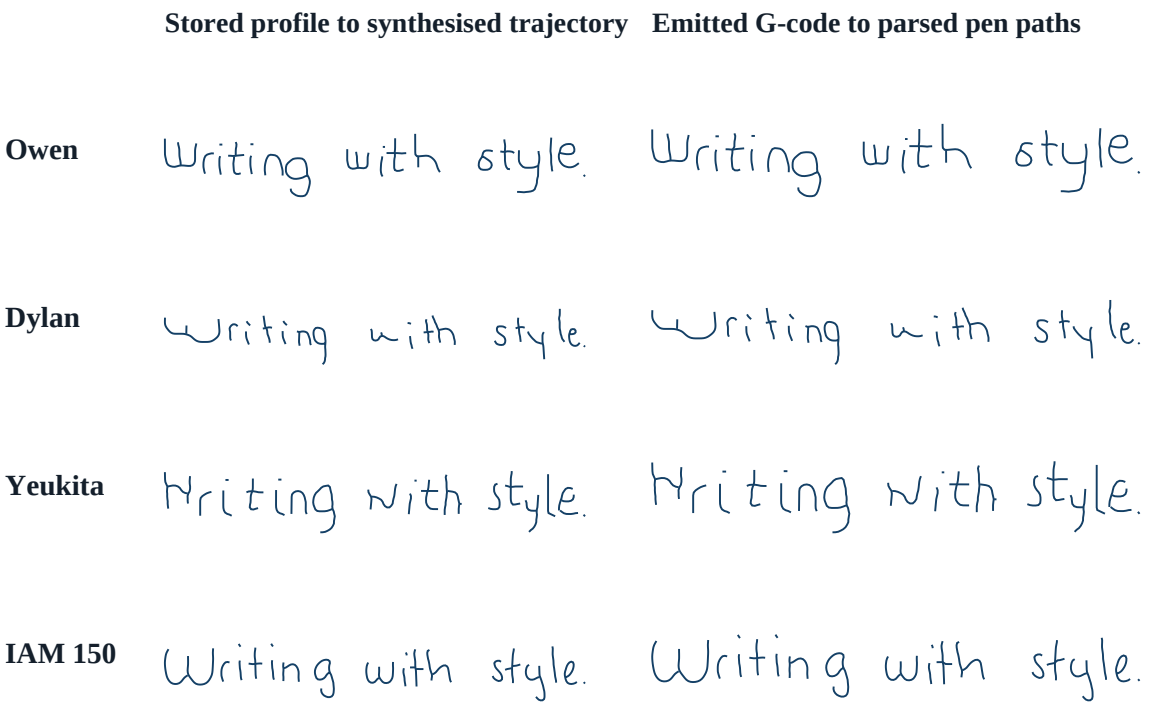
Figure 32 uses the final Owen, Dylan, Yeukita and IAM 150 profiles with the same text, seed 2026, requested x-height 4 mm and jitter 0.15. The left column shows generated pen paths. The right column reconstructs pen-down paths from each emitted G-code file with the independent

Algorithm 9 Exhaustive CTC likelihood and finite-difference check

Require: Small logits z, target ℓ, blank index and step ε.

$(L, g) \leftarrow \mathrm{CtcLoss}(z, \ell)$; $p \leftarrow \mathrm{Softmax}(z)$; $Z \leftarrow 0$
for each frame path $\pi \in \{0, \dots, C-1\}^T$ **do**
  if $\mathcal{B}(\pi) = \ell$ **then** $Z \leftarrow Z + \prod_t p_t(\pi_t)$
end for
Record $|L + \log Z|$
for each logit index i **do**
  $g_i^{\mathrm{ref}} \leftarrow [L(z + \varepsilon e_i) - L(z - \varepsilon e_i)]/(2\varepsilon)$
  Record $|g_i - g_i^{\mathrm{ref}}|$
end for
return loss discrepancy and maximum derivative discrepancy

parser; each panel is fitted separately for legibility. The normal converter was used, including its path ordering, joins and near-ring closure. All four generated jobs had coordinates within the configured bed bounds. The figures illustrate profile-dependent geometry and file conversion, rather than measuring similarity to a human writer.



Same text and seed; independently fitted panels. These are software renders, not physical drawings.

Figure 32. Four profiles reconstructed from emitted G-code.

Algorithm 10 describes the serialization check. Nine controlled fixtures combined scale factors 1, 3 and 30 with line offsets 0, 8.4 and 25.2 mm. Ring closure and joining were disabled, minimum-segment filtering was set to zero, and the independently parsed paths were paired with `ToMachine`'s output. Each fixture retained path and vertex counts, with no clamped points. Maximum coordinate error was 0.0004 mm, within the 0.0005 mm per-coordinate rounding bound from three-decimal emission. This comparison checks serialization after placement; it

does not independently prove that the placement equation itself is correct or that a motor reached a coordinate.

Algorithm 10 Controlled G-code serialization correspondence

Require: Known multi-path trajectory and configurations with joins and closure disabled.

1: **for each** fixture configuration c **do**
2: $P \leftarrow \mathrm{ToMachine}(T, c)$
3: $f \leftarrow \mathrm{WriteGcode}(T, c)$; $Q \leftarrow \mathrm{ParseGcode}(f)$
4: Compare path counts and corresponding vertex counts
5: Record $\max_i \|P_i - Q_i\|_\infty$ and clamped-point count
6: **end for**
7: **return** count agreements, rounding discrepancies and clipping records

Figure 33 is separately identified as a photograph of actual robot output, from the existing Owen reproduction sheet. The line crops are newly obtained from that stored photograph. This closes the visual explanation from a physical page to the segmentation interface, without presenting the earlier drawing as a newly executed hardware experiment. No exact same-job correspondence with Figure 32 is claimed.

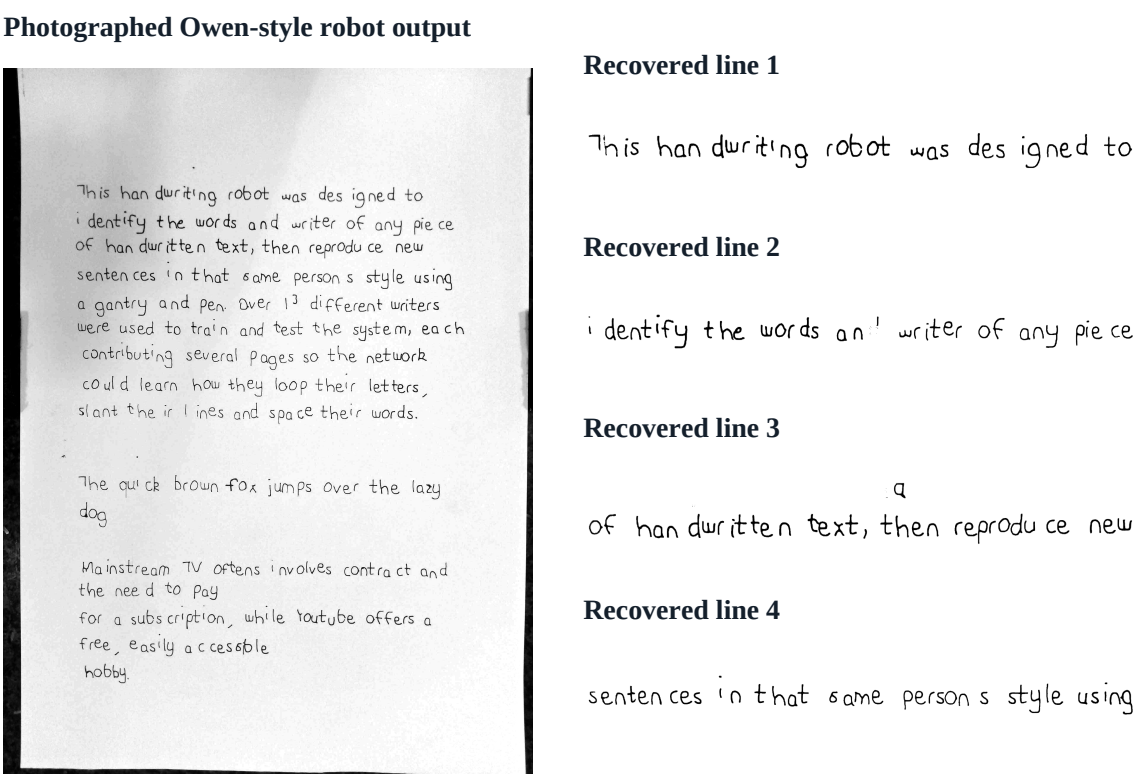


Figure 33. Photographed output and extracted line crops.

B.5 Candidate-scoring configuration and scope

The Owen trials use the joint reader and eleven-writer `pw170` checkpoint, with explicit 0.17 normalisation and uniform-ink rendering. They score character accuracy and requested-writer posterior before repair, independently of inherited server defaults. Exact inputs, readings and full-precision scores are retained in `Evidence/bench.json`.

B.6 Post-design seam-coefficient sensitivity

A new one-at-a-time check examines the existing seam weights. Owen01 and Yeukita01 are reduced to a maximum dimension of 1600 pixels, then passed through orientation, conditioning and line-centre estimation. The first, middle and last gaps on each page are selected by index before inspecting outputs. Eight settings share these six gaps, step limit seven and ink weight four: midpoint weight varies at step penalty 0.02, then step penalty varies at midpoint weight 0.012. This is post-design sensitivity, not a joint search or recovered historical tuning.

The parameterised reference matches all six unmodified default paths. Table 23 reports raw-ink intersections, total row movement and mean midpoint distance in working pixels; the midpoint is a preference, not a labelled correct boundary.

Midpoint weight	Step penalty	Ink intersections	Row movement	Mean offset / px
0.000	0.020	2	35	4.0553
0.006	0.020	2	38	0.5517
0.012	0.020	2	42	0.3625
0.024	0.020	2	44	0.3261
0.048	0.020	2	47	0.2622
0.012	0.000	2	48	0.2622
0.012	0.010	2	44	0.3261
0.012	0.040	2	38	0.5517

Table 23. New coefficient-sensitivity check across six fixed real-page gaps; the implemented setting is 0.012/0.020.

Increasing midpoint weight from zero to 0.012 reduces mean offset from 4.055 to 0.363 pixels, while row movement rises from 35 to 42 pixels. Doubling the weight gives 0.326 pixels offset. Increasing step penalty has the opposite trade-off. Every setting retains two raw-mask and three closed-mask intersections. These cases therefore show competing effects without establishing a unique optimum, eliminating intersections or measuring recognition accuracy. Input/code hashes and all 48 runs are supplied; no pipeline coefficient was changed.

B.7 Real-input calculation traces

The numerical substitutions in Design were captured from offline processing of existing project inputs. They are execution traces of the stated calculations, not reconstructed historical training decisions or measurements of the mechanism. The complete settings, input/model hashes and full-precision values are stored in `Evidence/real_calculations.json`;

`trace_real_calculations.py` reproduces them without changing model weights or final profiles.

The page-conditioning trace uses Owen01 reduced to 1600 rows by 1200 columns, giving an 800-row by 600-column working image before deskew. The network and extraction trace instead uses the separately identified archived first original Owen01 crop (156×2170), its known transcript and the joint reader. The latter produces $160 \times 1 \times 75$ log probabilities: the checkpoint head has 75 rows, consistent with 74 alphabet symbols and blank. The selected eleven-writer model uses explicit 0.17-width preparation for the two-crop vote. These inputs must not be mistaken for one continuous new whole-system test.

For the first convolution calculation, the actual input patch and learned kernel, listed in the same row-major order, are

$$X = \begin{bmatrix} -1 & -1 & -1 \\ -0.552941 & -0.380392 & -0.568627 \\ 0.866667 & 0.960784 & 0.866667 \end{bmatrix}, \qquad W = \begin{bmatrix} 0.213027 & -0.215063 & 0.234474 \\ 0.068398 & 0.083297 & 0.187499 \\ -0.312354 & -0.291350 & -0.221197 \end{bmatrix}.$$

Using the full-precision values with bias -0.000435125 gives -1.15133214 at output channel 0, row 34, column 462. Rounded values above explain the sum; the manifest retains the precise arrays needed to reproduce it. The subsequent batch-normalisation and LSTM substitutions follow this same crop through the reader.

The path and timing trace generates “Writing with style.” using the final Owen profile, seed 2026, requested x-height 4 mm and jitter 0.15. The generated file is `Evidence/real_trace_owen.gcode`. Its default configuration supplies drawing/travel feeds 900/2400 mm/min, nominal conversion 80 microsteps/mm and the 190.6 by 251.3 mm workspace. No physical command was sent. The pulse-delay calculation uses one emitted segment at that nominal scale; actual homing counts remain a separate hardware measurement. The trace also records the pre-file coordinate mapping, distance accumulators, 56 pen transitions and the resulting 32.8 s nominal budget.

The existing random-mask, exhaustive seam and small-lattice checks remain independent numerical tests. Their synthetic inputs are intentional test fixtures. They are separate from these real-input traces and from the qualification results.

B.8 Visual checks of numerical training operations

The retained visual-test record contains 249 scalar perturbation comparisons: 73 sampled convolution-weight entries, 73 bias comparisons, 73 input entries and all 30 logits of a small CTC lattice. The sampled entries need not be distinct parameters. Central differences use a step of 10^{-6} for the convolution and 10^{-5} for CTC, following the numerical-check principle in Equation 46. Figure 34 compares the resulting numerical derivatives with the implemented backward operations. Relative difference is the absolute discrepancy divided by the sum of the two derivative magnitudes, with a 10^{-12} denominator floor.

The largest relative difference is 7.50×10^{-7}, among the sampled convolution inputs; the largest absolute input discrepancy is 7.26×10^{-9}. The CTC relative difference is at most 1.69×10^{-9}. These checks support the tested local derivatives under the fixed fixtures. They do not exercise every layer, parameter, floating-point regime or complete training run.

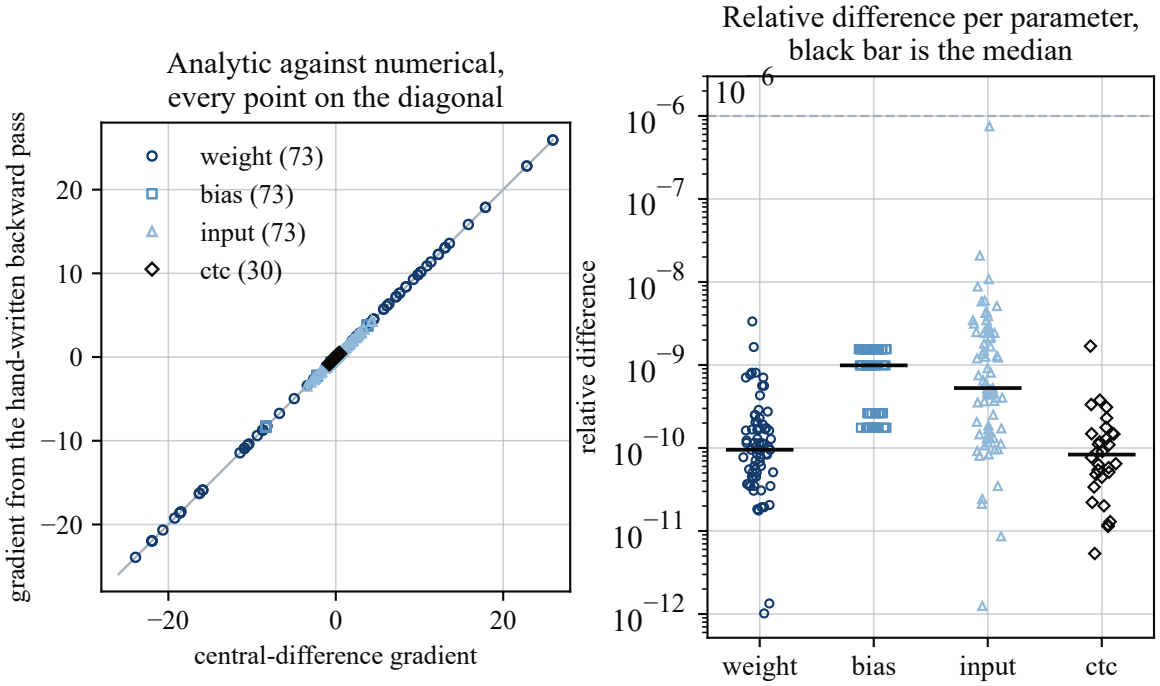


Figure 34. Analytic and finite-difference gradients.

B.9 Stroke-width quantisation in the validation sweep

Figure 35 shows four real crops processed at the four sweep fractions. The image transformation depends on the resulting odd integer dilation kernel as well as the fraction. For Owen and Abhinav, 0.06 and 0.09 both select $k = 3$ on the illustrated crop; for Dylan, 0.09 and 0.11 both select $k = 5$. Identical kernels give identical normalised images for that same crop. This does not imply that two settings produce identical training datasets, since x-height varies across crops. The validation bars reproduce the supplied training sweep, not a new accuracy measurement. Its best observed setting remains 0.17, while a single run at each setting does not isolate width effects from training variability.

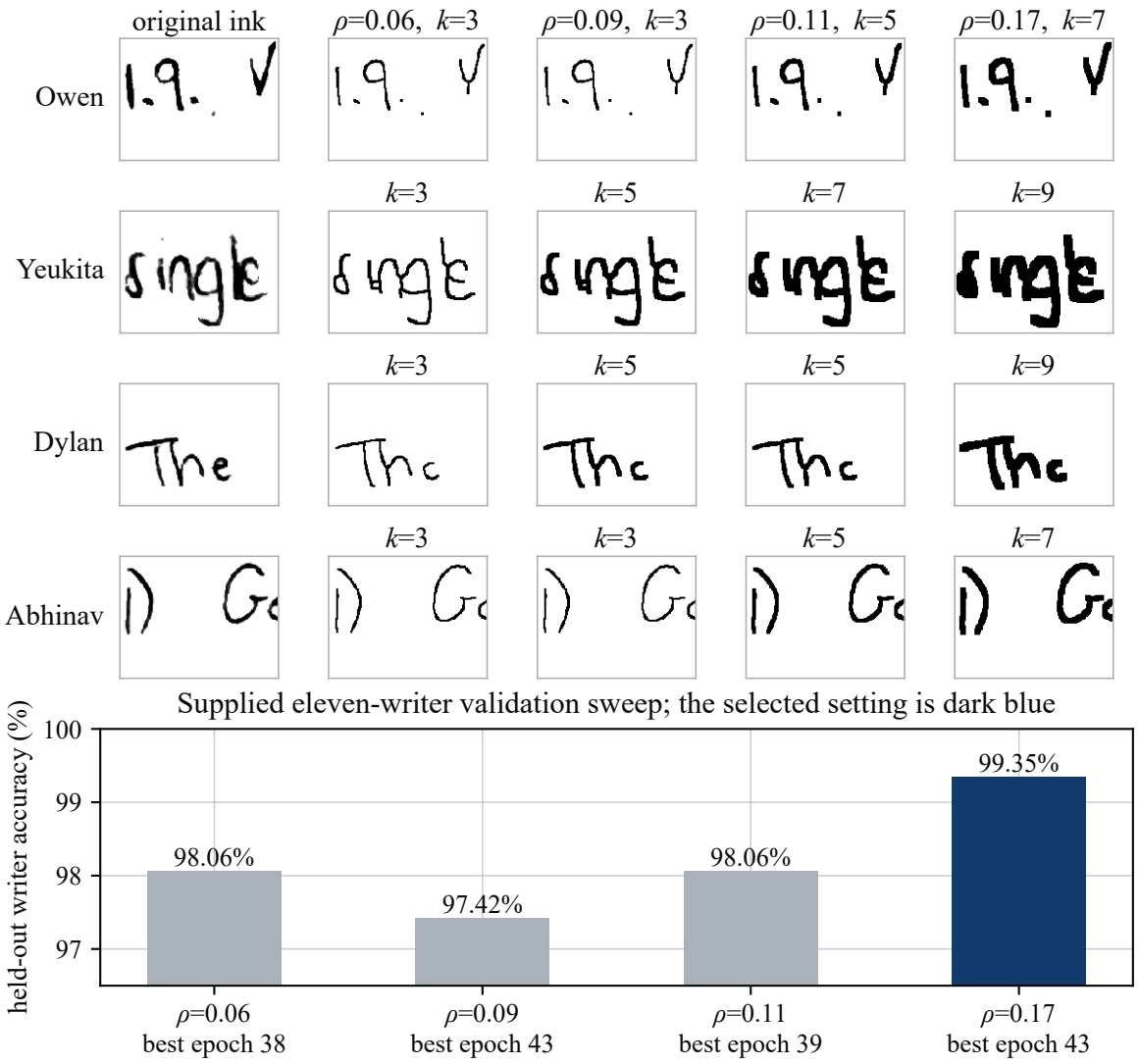
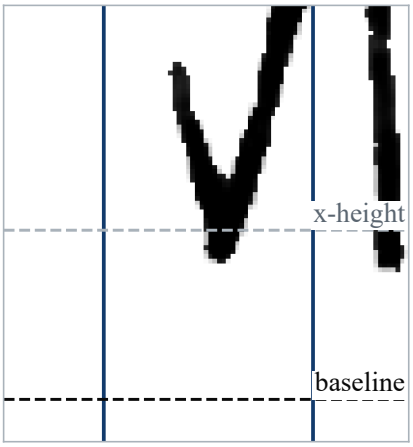


Figure 35. Stroke-width kernels and validation sweep.

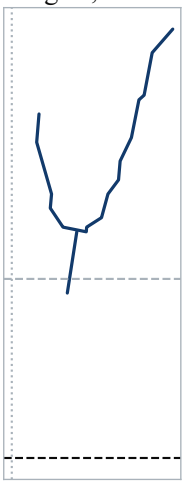
B.10 Glyph coordinates and measured connectedness

Figure 36 shows the actual Owen V used in the coordinate calculation in Design: left boundary 146 pixels, baseline 107 pixels, x-height 38 pixels and slant −4.5°. The stored path uses the baseline as its origin, measures height upwards and expresses lengths in x-heights. The lead is stored separately from the path so that source-cell spacing is not confused with the first surviving ink point.

(a) the cell for 'V' in image pixels
baseline b=107 px, h_x=38 px
measured slant -4.50°



(b) stored, de-slanted
v in x-heights, baseline at 0



(c) the stored transform

$$v = (b - y)/h_x$$
$$u_0 = (x - x_L)/h_x - v\tan\phi$$
$$u = u_0 - \delta$$

(d) what is stored for this cell

x_L = 146.0 px
b = 107 px
h_x = 38 px
$\tan\phi$ = -0.0787017

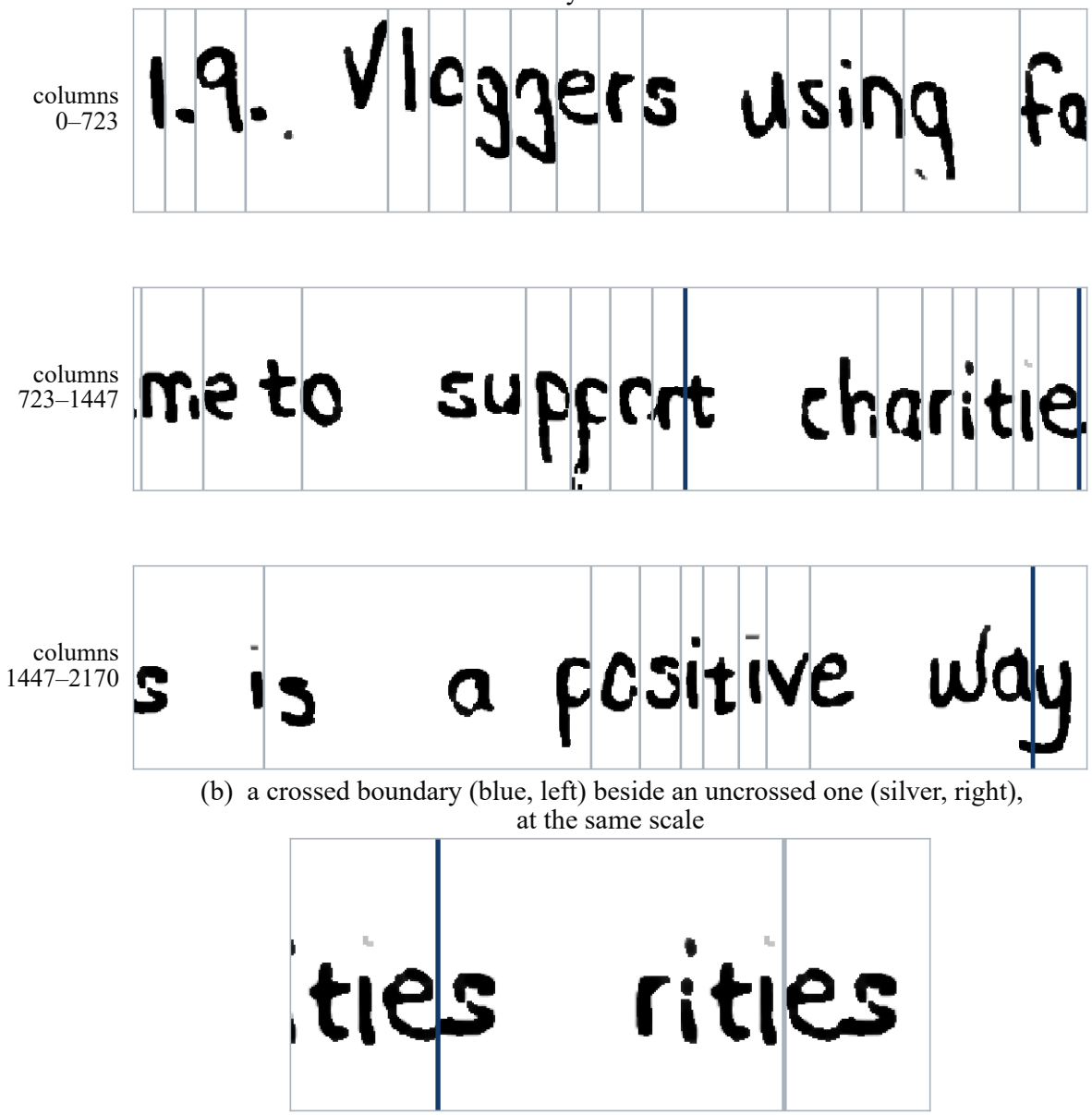
stored width = 0.710 h_x
advance = 1.237 h_x
lead δ = 0.447 h_x
top = 2.395, bottom = 0.921

entry = (0.152, 1.921)
exit = (0.363, 1.263)
joined left = False
joined right = False

Figure 36. Image and stored glyph coordinate frames.

Figure 37 makes the connectedness definition inspectable on the same line. Three of its 39 eligible adjacent non-space boundaries are crossed by a traced centreline, giving 3/39 = 0.076923. This is the line-level example already used in Design. A crossing records connected source ink under the extraction convention; it does not reveal the writer's natural pen-lift sequence or establish a connection fraction for every page by itself.

(a) 39 eligible adjacent non-space boundaries in silver, 3 of them crossed by a traced centreline in blue



(b) a crossed boundary (blue, left) beside an uncrossed one (silver, right), at the same scale

Figure 37. Observed connections between neighbouring characters.

B.11 Extended figures from the Design section

Figure 38 extends Figure 13 with both selected gaps from Abhinav01. One was selected for the greatest ink saving and the other for the largest departure from the midpoint. These are demonstrations of the cost function, not average accuracy measurements.

Figure 39 extends the first-row example in Figure 17. It retains the three selected channels at every stage and the final height reduction for the same Owen line. Independent display scaling makes local patterns visible without implying equal activation ranges between stages.

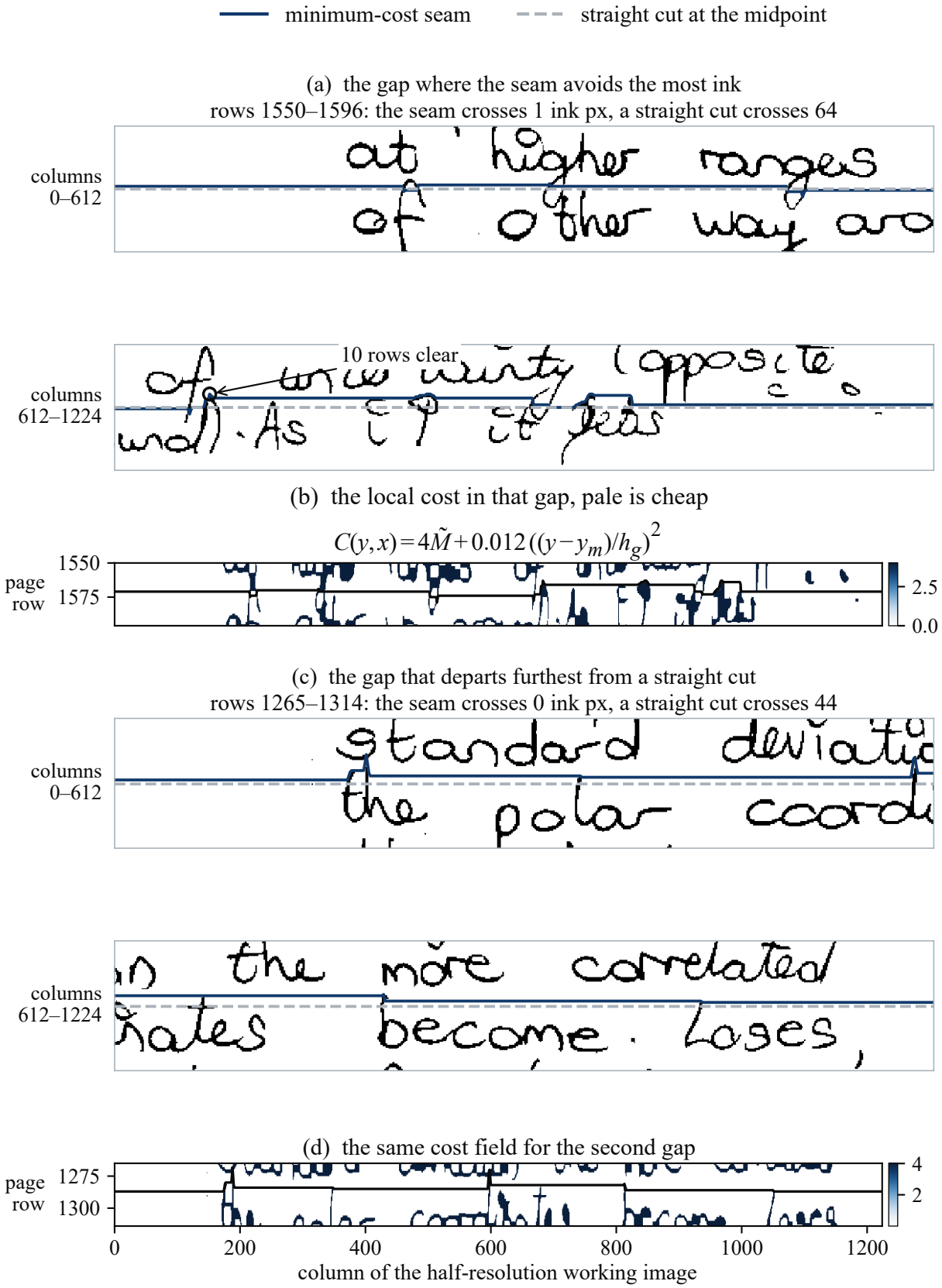


Figure 38. Complete seam and cost-field examples.

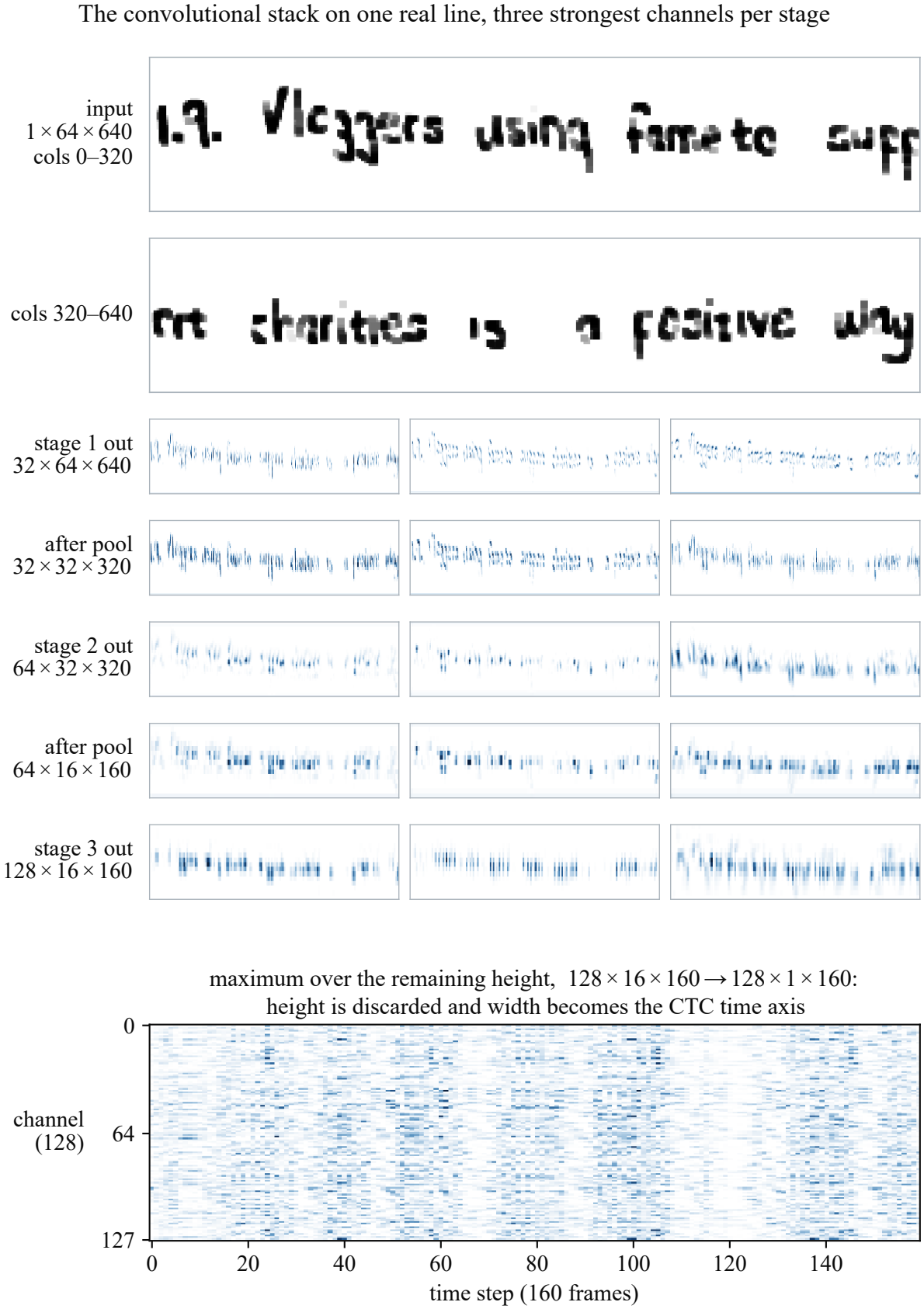


Figure 39. Complete convolutional feature sequence.

C. Supporting qualification evidence

This appendix preserves the detail needed to interpret the main qualification tests without repeating the Design's subsystem derivations. The separate supporting-evidence package contains the original v6 measurement archive, reviewed records and figure-generation scripts. Newly computed analyses are identified separately from the original measurements. No physical measurement is generated by a plotting script.

C.1 Evaluated checkpoints and populations

The selected demonstration reader is the library-trained joint reader, as confirmed by the author. The numerical scratch reader is a comparison model. The selected writer model is the library-trained eleven-writer 0.17 model. Table 24 records hash prefixes; the evidence manifest retains the full SHA-256 values. A filename alone is insufficient because the archived reader study identified two different files named `scratch_best`.

Role	Checkpoint identifier	SHA-256 prefix
Selected reader	joint_best	`336802d535992323`
Comparison reader	evaluated scratch_best	`2add315ded078de5`
Selected writer	11_pw170	`1884fb86467538eb`
Comparison writer	11w_pw170_scratch	`b23a709a30144f67`

Table 24. Identities of the measured model files.

The numerical-writer training excerpt supplied by the author records 1 031 training lines and 155 validation lines, 0.17 normalisation, a 50-epoch budget and a best displayed validation accuracy of 0.968 by epoch 41. The retained excerpt continues through epoch 47 and does not document a completed epoch 50. Its later common-set score is measured on 156 re-extracted validation lines; the different population is retained rather than silently equated with training validation. The numerical-reader excerpt records 6 481 public training lines, 569 personal training lines, 976 public validation lines and 100 personal validation lines, and a resume from an existing checkpoint. These are training-history records, while QT 2 evaluates saved weights directly.

An additional inventory reads each selected stored profile and verifies nonempty glyph mappings. The numbers of distinct character keys are 42, 38, 37, 36, 40 and 41 for the six database writers, and 56, 42, 43, 42 and 58 for A, B, O, T and Y. All eleven names agree with the selected writer checkpoint. The full source paths and content hashes are preserved in `ResultsEvidence/profile_inventory.json`. These counts establish stored mappings; they do not imply equal coverage of every character in the reader alphabet.

C.2 Reader validation breakdown

Table 25 extends the population totals in QT 2. The personal contributors are retained separately, using the same reader checkpoints and greedy decoder.

Evaluation set	Lines	Characters	Joint / %	NumPy / %
Public IAM validation	976	42072	94.12	94.39
Photographed A	31	934	94.00	94.65
Photographed B	19	739	95.81	97.70
Photographed O	22	1017	94.40	94.79
Photographed T	11	345	85.51	89.28
Photographed Y	17	640	92.03	92.50
All	1076	45747	94.06	94.39

Table 25. Reader character accuracy by validation population.

C.3 Physical reproduction by writer

Writer	Lines	Character / %	Writer / %	Word match / %
Y	13	97.01	92.31	74.16
150	13	96.75	53.85	76.40
O	16	94.25	100.00	58.51
B	16	93.11	93.75	68.24
384	16	86.93	87.50	43.18
552	15	86.17	80.00	48.39
152	15	85.74	80.00	49.44
A	27	77.29	100.00	18.09
153	19	75.72	57.89	31.46
588	18	73.19	94.44	22.99
T	16	60.97	100.00	13.41

Table 26. Physical reproduction scores by writer.

The paired read-back study contains 184 transcript-bearing lines. A separate model comparison contains 185 writer-labelled crops, including one crop with no corresponding transcript line. It can support a writer-prediction comparison, but that extra crop is not inserted into a character score. The model-comparison pooled groups are writers, not a claim that only eleven physical sheets were drawn.

C.3.1 Per-line classification rates and the board-to-host projection

The two ODROID classification requests are the only embedded records that separate the reader from the writer namer. Restating their logged stage seconds per line rather than per character uses the same two measurements and no new one: $19.024/16$ and $24.658/22$ for the reader, $11.091/16$ and $14.014/22$ for the namer. The per-line agreement, 5.9% and 8.4%, is much closer than the per-character agreement of 41.0% and 43.4%, because both networks are given one fixed 64×640 canvas for every line.

Host rates were then measured on ten photographed pages and 221 lines with the deployed code and checkpoints, timing `SegmentLines` through its own stage timer and then reading and naming every line crop, which is the order and the work a request performs. Dividing the board rates by the host medians gives the stage ratios quoted in Section 4.2.1. The record, including every per-page host figure and the leave-one-out check, is `ResultsEvidence/measurements/host_classify_stage_times_2026-11-02.json`.

Three limits apply. The board was not available when this was prepared, so nothing here is a new embedded measurement and every projected figure is labelled as such. The ratio rests on two board observations. The `reading back` stage recorded in the reproduction records is a different measurement, one rendered or drawn line rather than a photographed page, and is not pooled with these rates.

C.4 New geometric style comparison on physical pages

The new analysis applies the archived descriptor definitions with the current segmentation primitives to 156 original reference lines and 184 physical-output lines. The original population consists of the same six held-out database pages and 100 seeded photographed reference lines used for writer evaluation. The physical population comprises all fourteen robot pages after the recorded orientation convention. Each page's crop count agrees with its transcript-bearing records, and every line returns usable descriptors. All eleven writers are retained.

Each line is binarised, its body band estimated and its image resized toward a 28-pixel body height before the descriptors are evaluated. The same procedure is applied to both domains. Writer means give equal weight to available lines for each feature, and the pooled correlation gives equal weight to writers and selected features. Word-gap values are absent where no qualifying gap exists: 155/156 original and 180/184 physical lines have a value; the other descriptors are available for every line. All eleven writer means remain usable. The within-domain variant separately standardises each feature in each domain; the absolute variant uses original-domain means and standard deviations for both. Source images, the personal-crop cache, scripts and full measurements are recorded by hash. The raw outputs are in `ResultsEvidence/physical_style/results.json`.

The per-feature correlations are slant 0.9911; stroke width 0.7986; ink density 0.7392; word gap 0.4861; components per word 0.6688; character width 0.7978; ascender extent 0.8173; descender extent 0.5394; and extent ratio 0.5762. Figure 40 shows the actual writer means. Each point represents one writer, and dashed lines mark equal feature values. The 10 000-repeat writer-resampling range for the seven-feature within-domain coefficient is 0.5555–0.8272, using seed 23539195. It is a sensitivity range over this writer set. This retrospective comparison does not use identical sentences between domains and should not be interpreted as a controlled perceptual-equivalence experiment.

C.5 Archived geometric style study

The archived study covers writers 150, 151, 152, 153, 384, 551, 552, 588, B and Y. There are six synthetic lines per writer, sixty in total, compared with 123 original reference lines marked as excluded from profile building. This earlier ten-writer set differs from the final eleven-writer

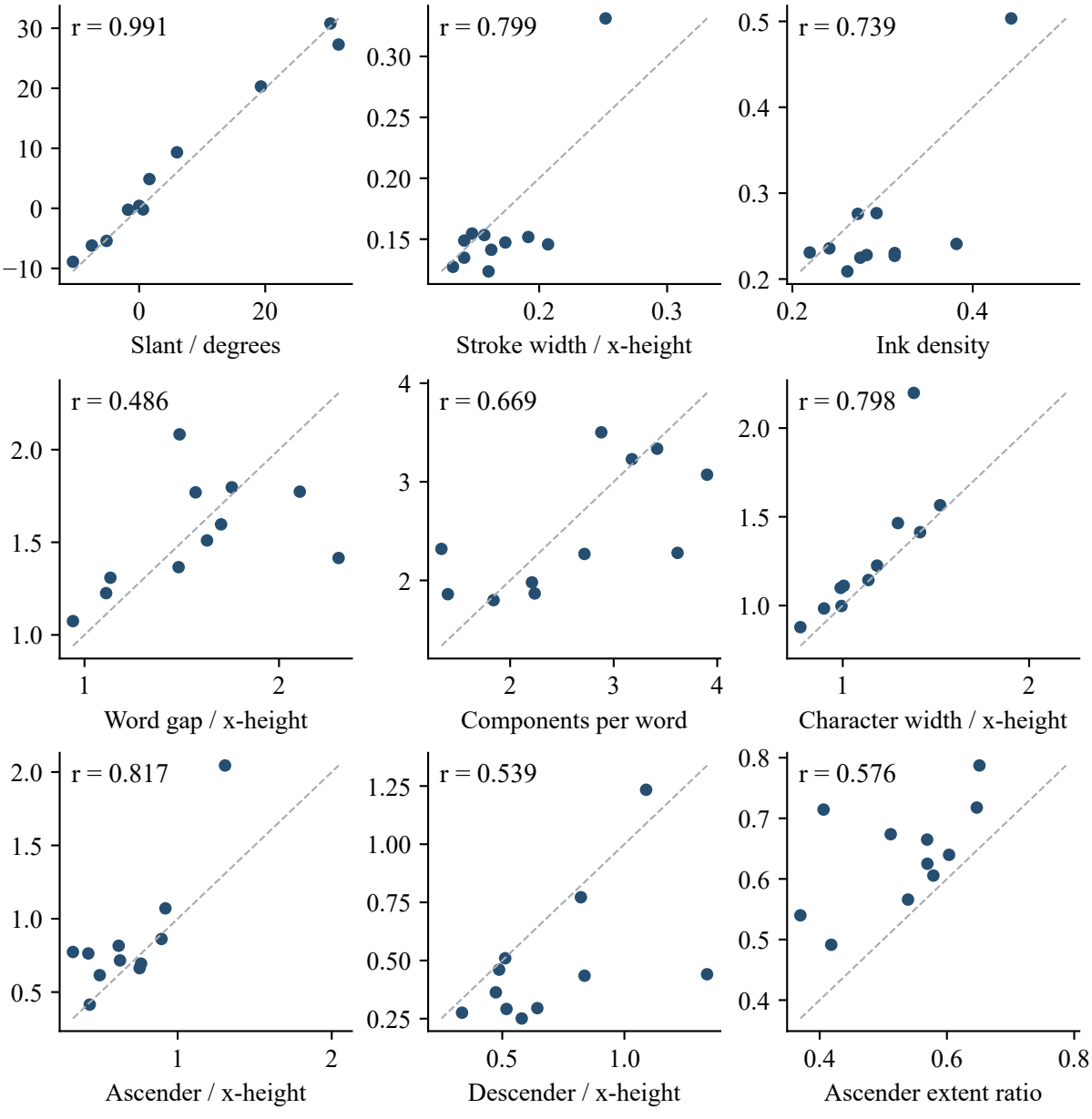


Figure 40. Original and reproduced writer descriptor means.

evaluation. The study is retained to explain geometric behaviour of the method; it is not relabelled as a measurement of final physical pages.

Nine deterministic descriptors measure slant, normalised stroke width, ink density, word gap, components per word, character width, ascender extent, descender extent and the ascender-to-total-extent ratio. Seven shape descriptors omit stroke width and ink density. The within-domain pooled correlation first standardises each feature across writers separately in the real and synthetic domains, then correlates the flattened arrays. It is therefore insensitive to a uniform offset or scale difference between those domains. The alternative absolute variant standardises both arrays using the real-domain mean and standard deviation. This distinction matters when interpreting preservation of magnitude rather than writer ordering.

Feature set	Separate-domain standardisation	Real-domain standardisation
Nine features	0.7013	0.5994
Seven shape features	0.7142	0.6711

Table 27. Archived synthetic-to-original pooled correlations against the profile-held-out reference. The 0.714 value is the seven-shape-feature variant.

The per-feature coefficients are 0.9964 for slant, 0.6315 for stroke width, 0.6802 for ink density, 0.6050 for word gap, 0.8323 for components per word, 0.9358 for character width, 0.9735 for ascender extent, 0.1480 for descender extent and 0.5085 for the extent ratio. Correlation describes variation across writers; it does not prove pointwise agreement of all letter shapes. The recorded 2 000 writer-resample interval for the seven-feature within-domain result is 0.6118–0.8586.

The archived overlay experiment uses thirty source texts across those ten writers. With x-height normalisation, mean matched chamfer distance is 0.6302, mean mismatched distance is 1.3868, their ratio is 2.2007 and the recorded AUC is 0.7074. When the archive also applies its width-normalisation variant, AUC is 0.4630. Thus the favourable separation depends on the chosen normalisation; an AUC above 0.5 in one variant must not be presented as universal writer fidelity. These remain software comparisons, without a new independent human panel.

C.6 Timing selection and optimisation scope

QT 3 uses a deliberately checkable completion boundary. The earlier archive retained 36 records after deduplication and a drawing-rate filter. The revised primary count uses 25 positive-character console records with explicit program-end evidence. The original 36-record aggregation remains in the preserved provenance document. A rate threshold alone does not establish completion: a job stopped after substantial drawing can exceed that threshold. This change does not alter the observed worst 3.875 s per character. A terminal marker confirms the interpreter finished, while geometry and content remain separate qualification questions.

The 500- and 982-character counts for QT 4 are the counts printed in the timer record. A final repeat should retain independent transcript lengths and an acquisition-to-output timing boundary. The two current overruns are large, so no inferred sub-second camera transfer time is added to create a favourable pass.

The G-code study of the same 378-character paragraph yields pen-down path lengths from 2 015.4 to 3 942.8 mm and recorded pen-state counts from 544 to 1 188 across eleven profiles. These are

Comparison	Measured outcome and scope
Sequential NumPy vs batched library networks	359.8 s vs 130.3 s over fourteen stored pages on the development host; factor 2.76. Both backend and batching change. Reader text is identical; writer-label equivalence is not inferred from that fact.
Full vs half linear image resolution	4.37-fold recorded image-stage factor on comparable successful page pairs, with 247 vs 182 detected lines. Zero of twelve comparable pages retain line count; two of fourteen reduced pages fail segmentation.
Projected network-only replacement	Applying 2.76 to ODROID network stages, with image stages unchanged, gives approximately 27.05 s and 30.82 s. Neither is an observed ODROID result or a total-time pass.
Projected combined replacement	Applying both host factors gives approximately 14.61 s and 17.86 s. The second is numerically below its allowance, but the image reduction changes the output and the timing is transferred between hosts. It is not accepted as R4 compliance.

Table 28. Acceleration evidence and the boundary between a measured comparison and a projected compliant configuration.

arithmetic properties of generated paths. They demonstrate profile-dependent geometry and help explain timing variation; they are not independent proof of similarity to genuine handwriting, and their values can still be affected by the candidate-selection process.

C.7 Cost, consent and final records

The photographed ledger supplied by the author is retained as evidence. The present build estimate is approximately R4 000, with a borrowed ODROID entered at zero purchase cost. The later listed clipboard (R90), paper (R130), gel pen (R30) and estimated training electricity (R500) total R750. Their overlap with the approximate build estimate is not established, so they are not added to produce an unsupported exact total. Replacement value and estimated operating costs should be distinguished from actual purchased parts.

Signed consent forms are held by the author for the five personal contributors. The report records that fact without reproducing signatures. Participant consent is recorded separately from institutional clearance or exemption.

C.8 Supporting timing and dimensional figures

Figure 41 compares the worst completed reproduction total at each tested length with its allowance. Labels retain the number of records. Figure 42 separates image and network processing for the two ODROID requests. The dashed markers show their complete allowances. Figure 43 compares each measured feature length with its nominal value. The dashed limits apply to length differences and do not establish absolute bed coordinates.

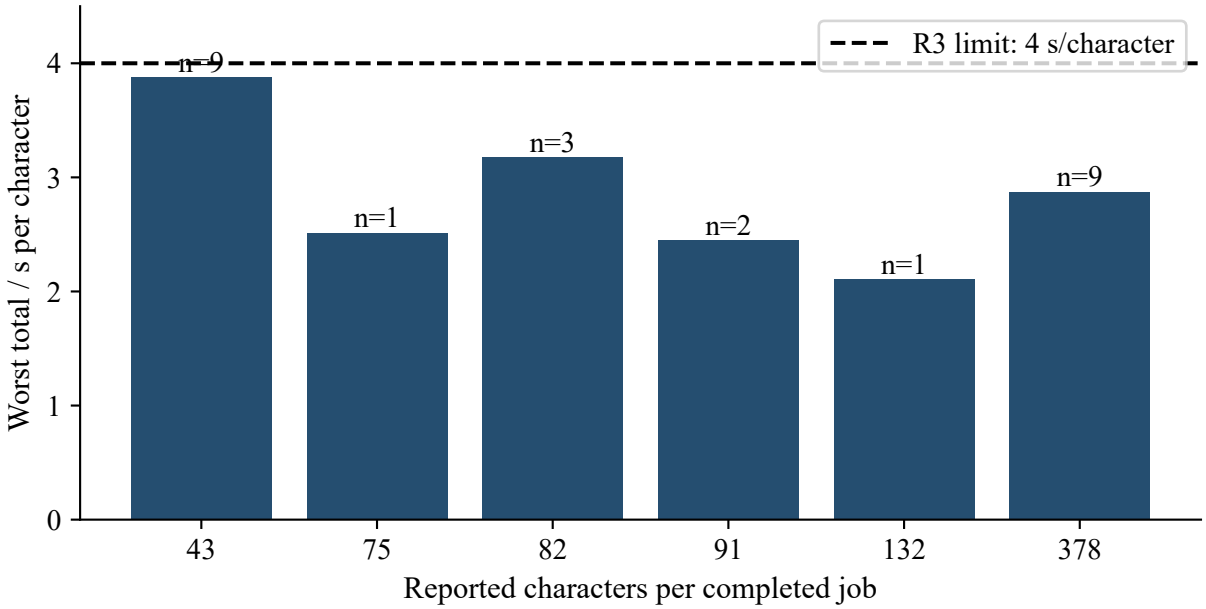


Figure 41. Completed reproduction times and allowances.

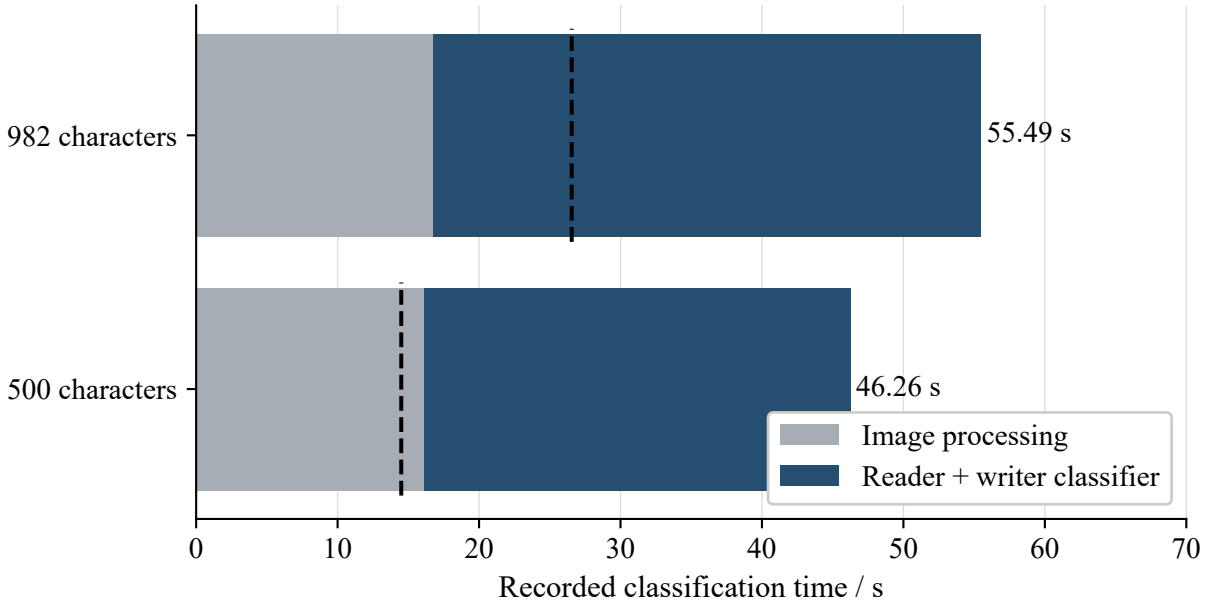


Figure 42. ODROID classification stages and allowances.

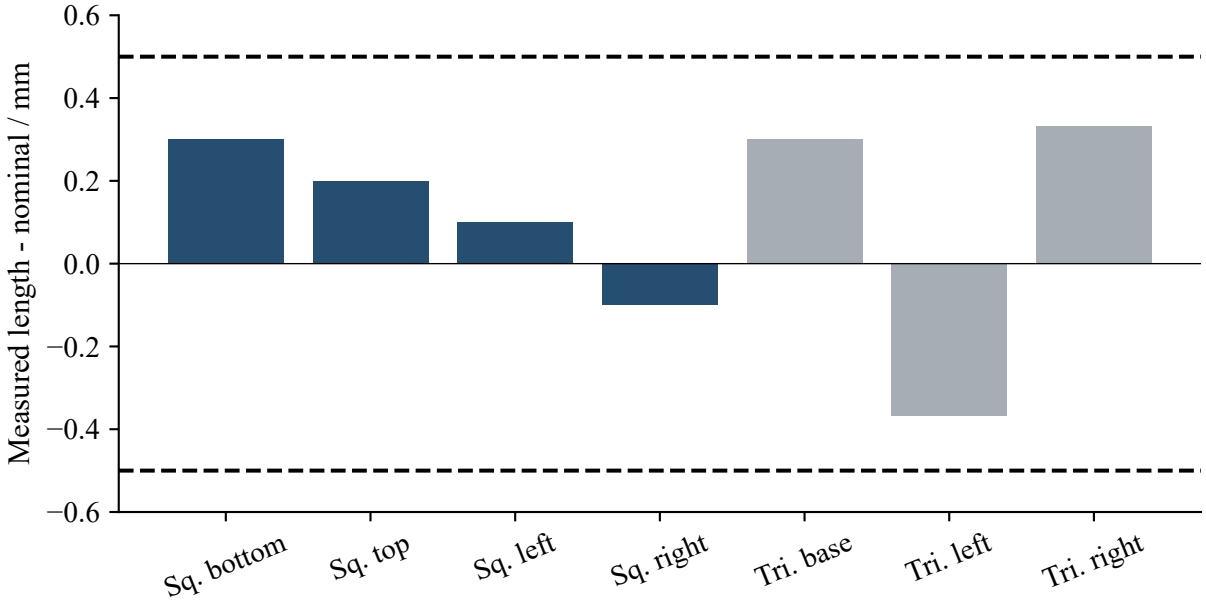


Figure 43. Signed errors in the measured feature lengths.

C.9 Triangle geometry diagnostic

Place the measured base endpoints at $(-b/2, 0)$ and $(b/2, 0)$ and the apex at (d, h). Subtracting the two squared-distance equations gives the project's geometric diagnostic:

$$d = \frac{\ell^2 - r^2}{2b}, \qquad \phi_{\text{app}} = \arctan(d/h). \tag{48}$$

Using $b = 80.30$ mm, $\ell = 56.20$ mm, $r = 56.90$ mm and the nominal $h = 40$ mm gives $d = -0.493$ mm and $\phi_{\text{app}} = -0.706°$. The angle is conditional on the assumed height and a shear interpretation; it is not an independent measurement of rail squareness. Both an ideal square and a rhombus can have equal side lengths, which is why diagonal or coordinate measurements would add information absent from the square-side test.

C.10 Complete physical-page records

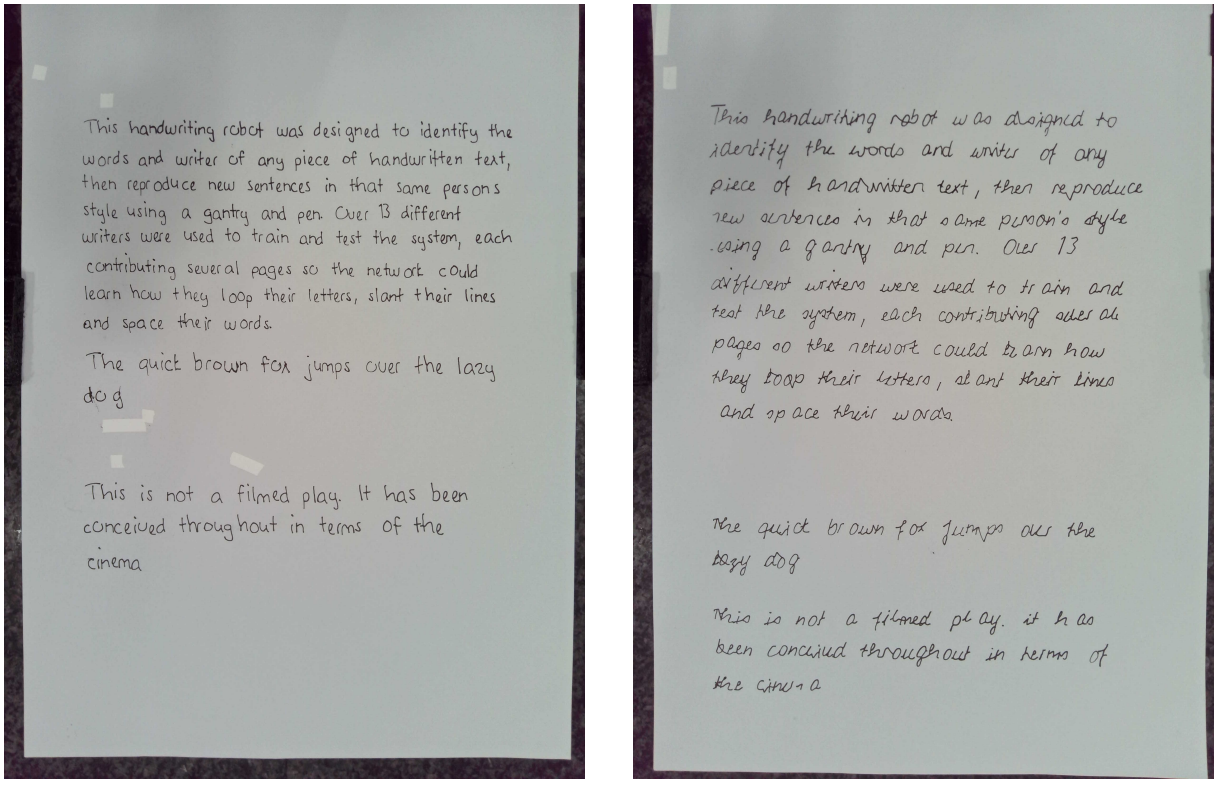
Table 29 retains all fourteen pages. Photographs are rotated for reading without changing their image data. Character accuracy pools edit counts within each page. Writer identification uses its paired lines. Text and timing records have no verified one-to-one job link, so the timing table below is kept separate.

Table 29. All physical pages and paired-line scores.

Page	Lines	Characters	Edits	Character / %	Writer / %
Author150	13	493	16	96.75	53.85
Author152	15	491	70	85.74	80.00
Author1531	15	407	106	73.96	60.00

Page	Lines	Characters	Edits	Character / %	Writer / %
Author1532	4	79	12	84.81	50.00
Author3841	13	409	52	87.29	92.31
Author3842	3	73	11	84.93	66.67
Author552	15	506	70	86.17	80.00
Author588	18	496	133	73.19	94.44
AuthorA1	17	361	82	77.29	100.00
AuthorA2	10	163	37	77.30	100.00
AuthorD	16	479	33	93.11	93.75
AuthorO	16	539	31	94.25	100.00
AuthorT	16	474	185	60.97	100.00
AuthorY	13	501	15	97.01	92.31

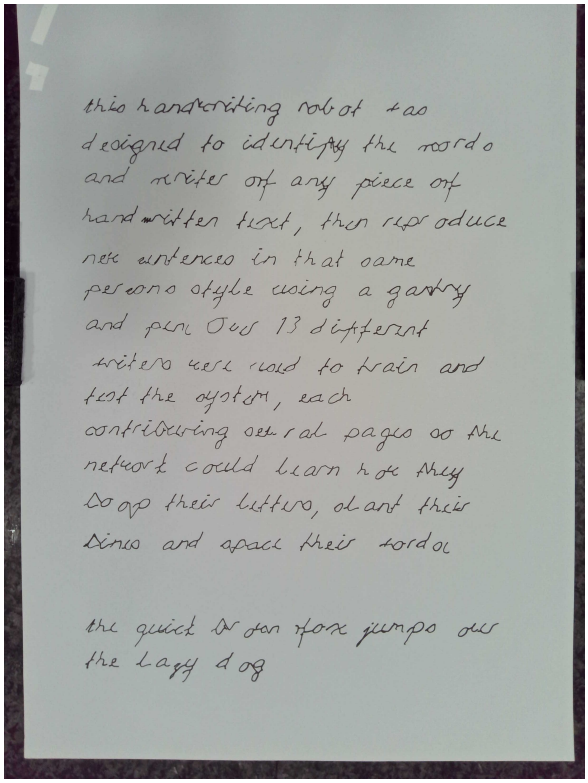
Figure 44 preserves the photographed pages Author150, Author152. The scores come from their paired line records, not a visual estimate.



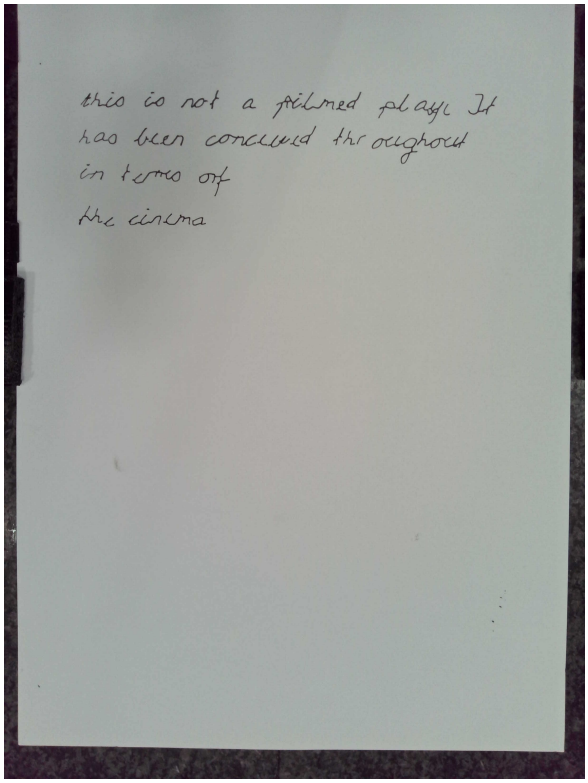
Author150 Author152

Figure 44. Physical pages Author150, Author152.

Figure 45 preserves the photographed pages Author1531, Author1532. The scores come from their paired line records, not a visual estimate.



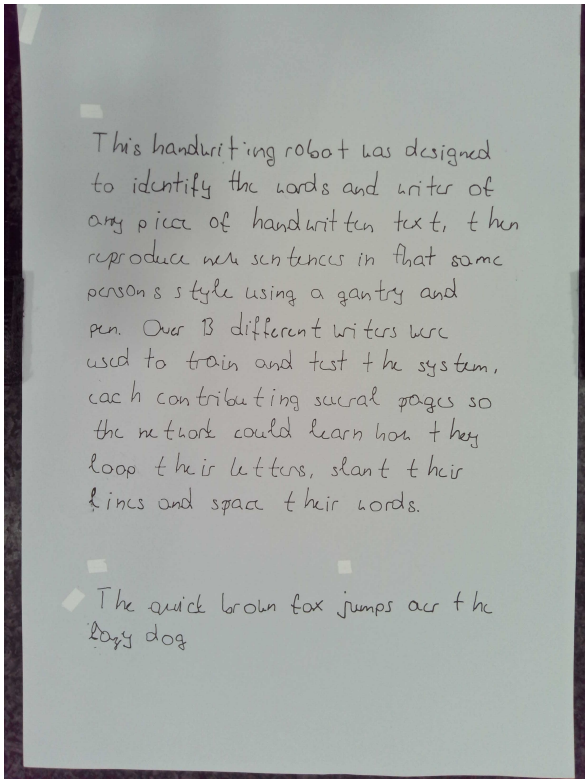
Author1531



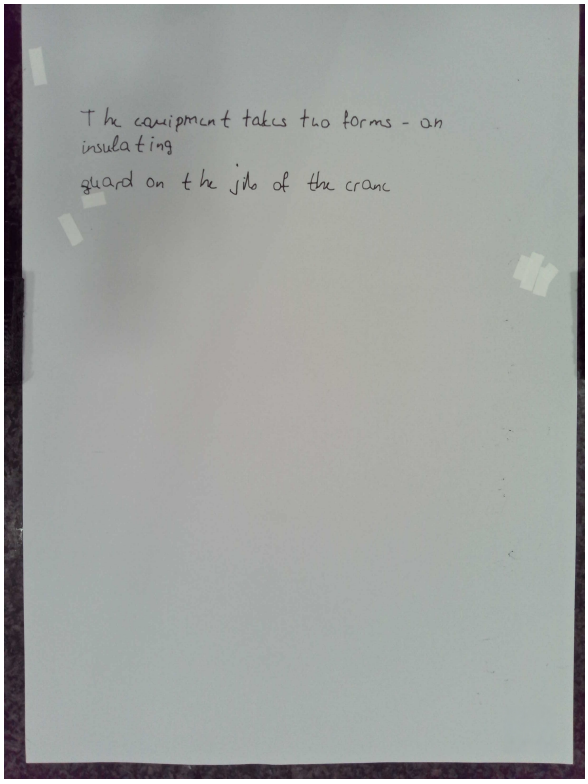
Author1532

Figure 45. Physical pages Author1531, Author1532.

Figure 46 preserves the photographed pages Author3841, Author3842. The scores come from their paired line records, not a visual estimate.



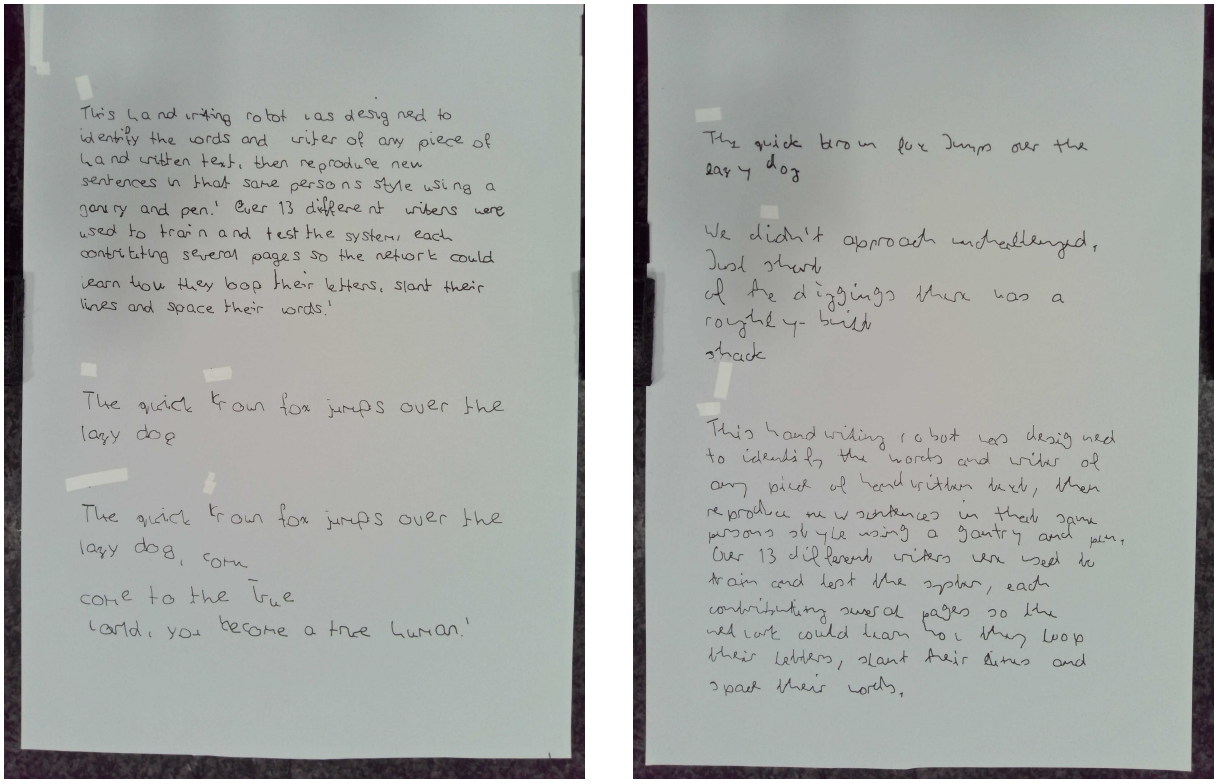
Author3841



Author3842

Figure 46. Physical pages Author3841, Author3842.

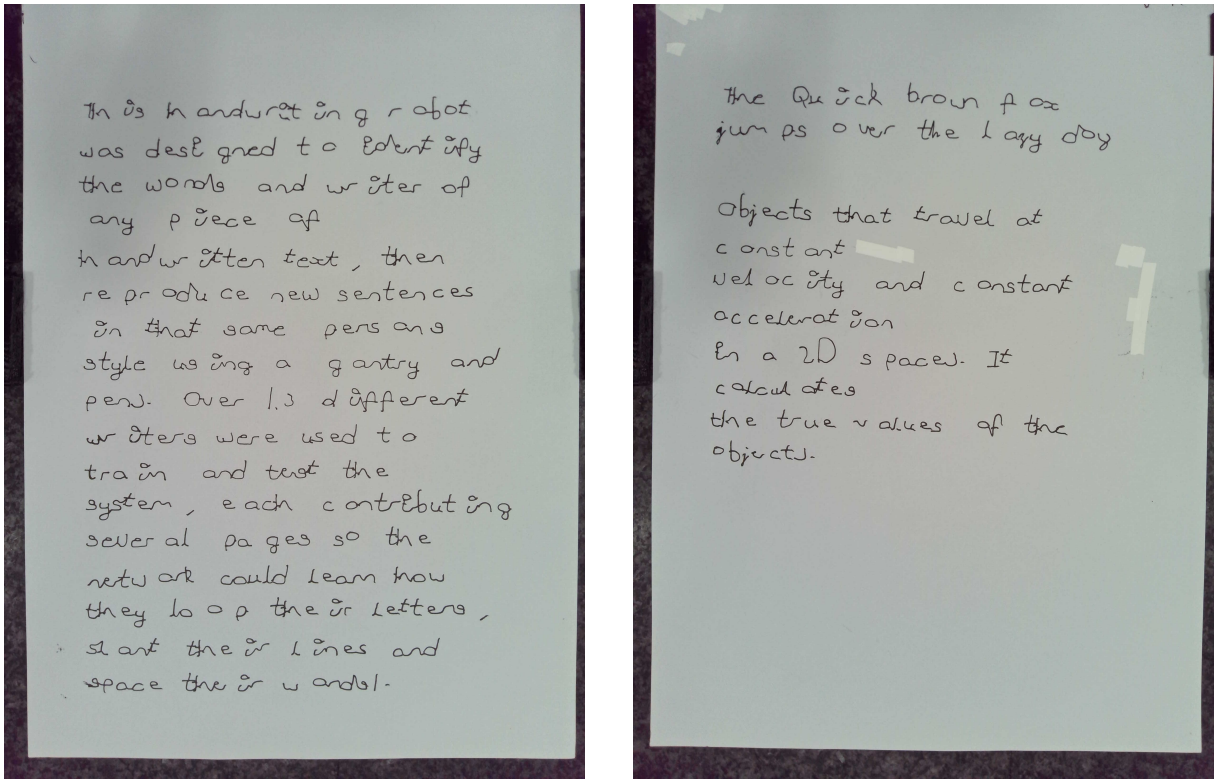
Figure 47 preserves the photographed pages Author552, Author588. The scores come from their paired line records, not a visual estimate.



Author552 Author588

Figure 47. Physical pages Author552, Author588.

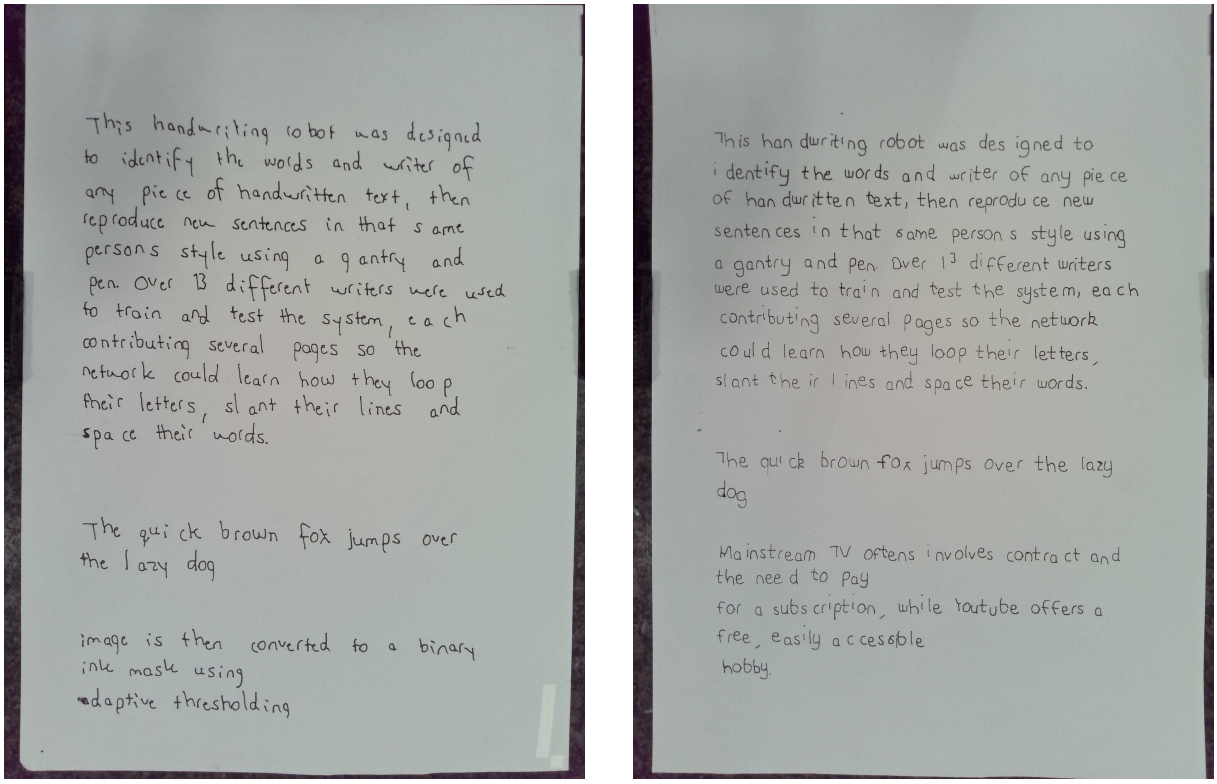
Figure 48 preserves the photographed pages AuthorA1, AuthorA2. The scores come from their paired line records, not a visual estimate.



AuthorA1 AuthorA2

Figure 48. Physical pages AuthorA1, AuthorA2.

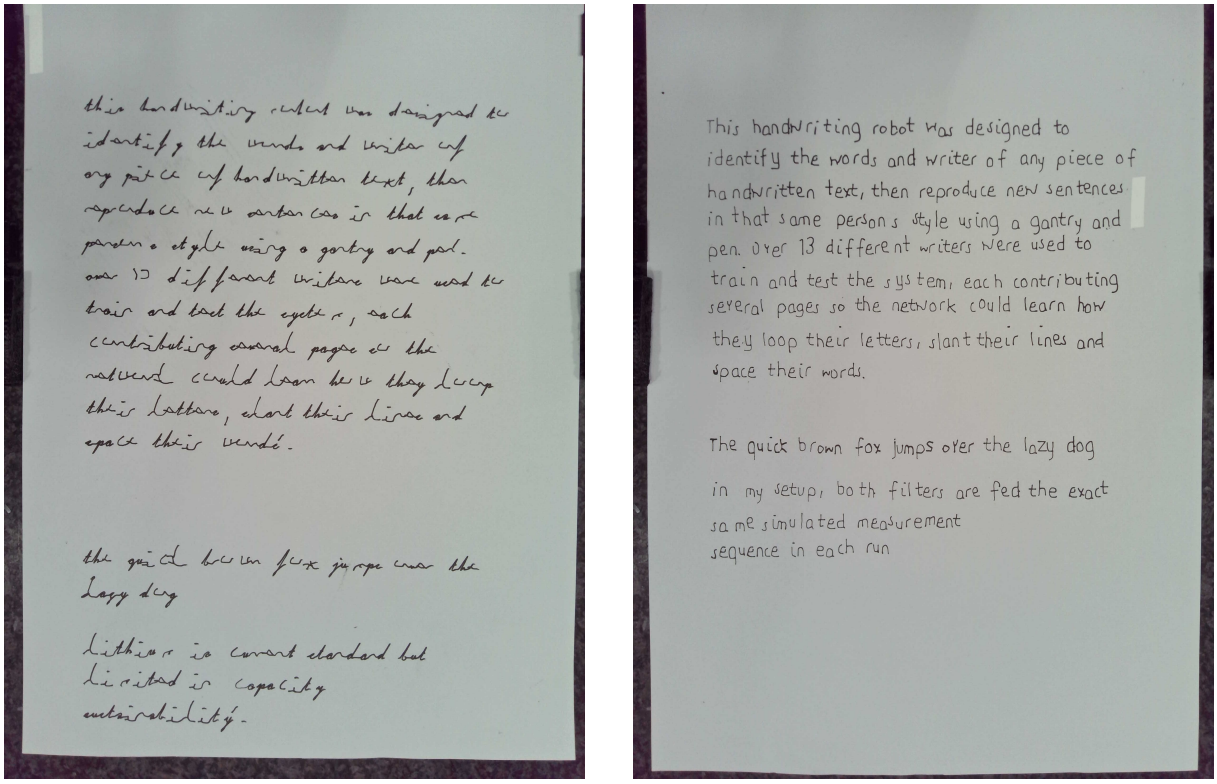
Figure 49 preserves the photographed pages AuthorD, AuthorO. The scores come from their paired line records, not a visual estimate.



AuthorD AuthorO

Figure 49. Physical pages AuthorD, AuthorO.

Figure 50 preserves the photographed pages AuthorT, AuthorY. The scores come from their paired line records, not a visual estimate.



AuthorT AuthorY

Figure 50. Physical pages AuthorT, AuthorY.

C.11 Paired-line scoring record

Table 30 gives every paired line, its reference length, edit distance and writer decision. Line numbering starts at one for each page. Reference and recognised text are retained verbatim in the accompanying evidence CSV so that these summaries can be recalculated.

Table 30. All 184 paired physical lines.

Page	Line	Characters	Edits	Character / %	Predicted writer
Author150	1	51	2	96.08	150
Author150	2	50	1	98.00	Y
Author150	3	50	1	98.00	150
Author150	4	47	3	93.62	Y
Author150	5	52	1	98.08	Y
Author150	6	47	3	93.62	150
Author150	7	52	1	98.08	Y
Author150	8	22	0	100.00	150
Author150	9	39	2	94.87	150
Author150	10	3	1	66.67	O
Author150	11	38	1	97.37	150
Author150	12	36	0	100.00	150

Page	Line	Characters	Edits	Character / %	Predicted writer
Author150	13	6	0	100.00	Y
Author152	1	38	4	89.47	152
Author152	2	36	5	86.11	152
Author152	3	41	4	90.24	152
Author152	4	41	7	82.93	588
Author152	5	31	1	96.77	152
Author152	6	40	6	85.00	152
Author152	7	42	6	85.71	152
Author152	8	36	3	91.67	152
Author152	9	42	6	85.71	152
Author152	10	22	3	86.36	152
Author152	11	34	7	79.41	152
Author152	12	8	3	62.50	Y
Author152	13	33	7	78.79	152
Author152	14	37	5	86.49	152
Author152	15	10	3	70.00	T
Author1531	1	26	12	53.85	153
Author1531	2	30	8	73.33	153
Author1531	3	26	4	84.62	153
Author1531	4	32	6	81.25	153
Author1531	5	26	4	84.62	384
Author1531	6	28	6	78.57	T
Author1531	7	26	8	69.23	T
Author1531	8	30	7	76.67	588
Author1531	9	21	5	76.19	153
Author1531	10	33	10	69.70	153
Author1531	11	28	8	71.43	153
Author1531	12	31	7	77.42	T
Author1531	13	28	8	71.43	153
Author1531	14	30	10	66.67	153
Author1531	15	12	3	75.00	T
Author1532	1	29	5	82.76	153
Author1532	2	29	3	89.66	153
Author1532	3	11	4	63.64	T
Author1532	4	10	0	100.00	T
Author3841	1	35	2	94.29	384
Author3841	2	35	2	94.29	384
Author3841	3	35	7	80.00	384
Author3841	4	36	3	91.67	384
Author3841	5	32	2	93.75	384
Author3841	6	35	3	91.43	384
Author3841	7	34	4	88.24	384
Author3841	8	34	6	82.35	384
Author3841	9	32	5	84.38	384
Author3841	10	31	4	87.10	384
Author3841	11	28	4	85.71	384

Page	Line	Characters	Edits	Character / %	Predicted writer
Author3841	12	34	7	79.41	384
Author3841	13	8	3	62.50	A
Author3842	1	34	6	82.35	384
Author3842	2	10	1	90.00	O
Author3842	3	29	4	86.21	384
Author552	1	38	6	84.21	552
Author552	2	45	5	88.89	552
Author552	3	36	4	88.89	552
Author552	4	44	2	95.45	552
Author552	5	46	12	73.91	552
Author552	6	39	1	97.44	552
Author552	7	47	4	91.49	552
Author552	8	46	5	89.13	552
Author552	9	28	3	89.29	552
Author552	10	34	6	82.35	552
Author552	11	8	2	75.00	A
Author552	12	34	5	85.29	552
Author552	13	14	4	71.43	A
Author552	14	16	4	75.00	A
Author552	15	31	7	77.42	552
Author588	1	34	5	85.29	588
Author588	2	8	2	75.00	588
Author588	3	32	8	75.00	588
Author588	4	10	4	60.00	588
Author588	5	27	4	85.19	588
Author588	6	13	7	46.15	588
Author588	7	5	4	20.00	A
Author588	8	35	10	71.43	588
Author588	9	35	7	80.00	588
Author588	10	35	13	62.86	588
Author588	11	36	13	63.89	588
Author588	12	37	6	83.78	588
Author588	13	38	6	84.21	588
Author588	14	31	11	64.52	588
Author588	15	33	10	69.70	588
Author588	16	33	11	66.67	588
Author588	17	36	5	86.11	588
Author588	18	18	7	61.11	588
AuthorA1	1	22	12	45.45	A
AuthorA1	2	24	7	70.83	A
AuthorA1	3	23	4	82.61	A
AuthorA1	4	12	2	83.33	A
AuthorA1	5	22	5	77.27	A
AuthorA1	6	23	1	95.65	A
AuthorA1	7	20	4	80.00	A
AuthorA1	8	24	3	87.50	A

Page	Line	Characters	Edits	Character / %	Predicted writer
AuthorA1	9	22	5	77.27	A
AuthorA1	10	20	5	75.00	A
AuthorA1	11	18	2	88.89	A
AuthorA1	12	25	4	84.00	A
AuthorA1	13	20	4	80.00	A
AuthorA1	14	23	6	73.91	A
AuthorA1	15	24	6	75.00	A
AuthorA1	16	21	6	71.43	A
AuthorA1	17	18	6	66.67	A
AuthorA2	1	19	8	57.89	A
AuthorA2	2	23	9	60.87	A
AuthorA2	3	22	1	95.45	A
AuthorA2	4	8	2	75.00	A
AuthorA2	5	21	4	80.95	A
AuthorA2	6	12	1	91.67	A
AuthorA2	7	18	4	77.78	A
AuthorA2	8	10	3	70.00	A
AuthorA2	9	22	4	81.82	A
AuthorA2	10	8	1	87.50	A
AuthorD	1	35	5	85.71	B
AuthorD	2	35	3	91.43	B
AuthorD	3	35	2	94.29	B
AuthorD	4	36	3	91.67	B
AuthorD	5	32	0	100.00	B
AuthorD	6	40	4	90.00	B
AuthorD	7	34	3	91.18	B
AuthorD	8	33	0	100.00	B
AuthorD	9	33	5	84.85	B
AuthorD	10	36	1	97.22	B
AuthorD	11	18	1	94.44	B
AuthorD	12	30	1	96.67	B
AuthorD	13	12	1	91.67	B
AuthorD	14	35	0	100.00	B
AuthorD	15	14	2	85.71	552
AuthorD	16	21	2	90.48	B
AuthorO	1	38	3	92.11	O
AuthorO	2	42	2	95.24	O
AuthorO	3	39	2	94.87	O
AuthorO	4	42	0	100.00	O
AuthorO	5	43	3	93.02	O
AuthorO	6	44	2	95.45	O
AuthorO	7	41	1	97.56	O
AuthorO	8	40	1	97.50	O
AuthorO	9	40	2	95.00	O
AuthorO	10	39	4	89.74	O
AuthorO	11	3	1	66.67	O

Page	Line	Characters	Edits	Character / %	Predicted writer
AuthorO	12	42	1	97.62	O
AuthorO	13	15	2	86.67	O
AuthorO	14	42	3	92.86	O
AuthorO	15	23	3	86.96	O
AuthorO	16	6	1	83.33	O
AuthorT	1	38	16	57.89	T
AuthorT	2	32	8	75.00	T
AuthorT	3	35	13	62.86	T
AuthorT	4	36	15	58.33	T
AuthorT	5	37	12	67.57	T
AuthorT	6	38	16	57.89	T
AuthorT	7	31	9	70.97	T
AuthorT	8	33	12	63.64	T
AuthorT	9	33	22	33.33	T
AuthorT	10	36	12	66.67	T
AuthorT	11	18	6	66.67	T
AuthorT	12	34	20	41.18	T
AuthorT	13	8	5	37.50	T
AuthorT	14	31	10	67.74	T
AuthorT	15	19	4	78.95	T
AuthorT	16	15	5	66.67	T
AuthorY	1	38	2	94.74	Y
AuthorY	2	45	1	97.78	Y
AuthorY	3	46	0	100.00	Y
AuthorY	4	45	2	95.56	Y
AuthorY	5	43	4	90.70	Y
AuthorY	6	44	2	95.45	Y
AuthorY	7	44	1	97.73	Y
AuthorY	8	46	0	100.00	Y
AuthorY	9	18	0	100.00	Y
AuthorY	10	43	0	100.00	Y
AuthorY	11	43	1	97.67	Y
AuthorY	12	26	2	92.31	Y
AuthorY	13	20	0	100.00	O

C.12 Complete confirmed reproduction timings

Table 31 retains the 25 completed, positive-character records used in QT 3. The line number identifies the timing block in the preserved ODROID log. Synthesis and drawing are component stages. The total also includes the other logged stages and rounding, so it is not replaced by their sum.

Table 31. All confirmed ODROID reproduction timings.

Log line	Characters	Synthesis / s	Drawing / s	Total / s	s per character
32	378	8.23	497.55	507.57	1.343
185	378	17.98	637.31	657.72	1.740
203	43	17.66	92.59	110.59	2.572
595	91	16.44	175.50	192.47	2.115
631	378	12.36	651.18	665.14	1.760
682	43	14.80	91.30	106.44	2.475
718	82	17.06	165.46	183.04	2.232
816	378	39.27	899.55	941.48	2.491
864	43	20.34	113.94	134.65	3.131
887	82	27.53	204.05	232.16	2.831
1004	378	58.78	1022.83	1085.01	2.870
1027	43	26.52	139.66	166.61	3.875
1050	82	33.67	225.88	260.33	3.175
1099	378	16.63	839.33	859.00	2.272
1122	43	18.81	106.99	126.19	2.935
1145	75	25.39	162.76	188.60	2.515
1189	378	12.11	663.01	676.64	1.790
1225	43	15.10	96.53	112.00	2.605
1260	43	15.22	96.44	112.03	2.605
1348	378	104.08	656.92	762.84	2.018
1384	43	15.65	81.40	97.28	2.262
1407	91	23.34	198.59	222.63	2.446
1495	378	14.17	726.40	742.72	1.965
1518	43	15.19	95.64	111.18	2.586
1541	132	22.88	254.16	277.90	2.105

C.13 Supplied first-principles training records

The following tables transcribe the console excerpts supplied by the author. They extend the training-route comparison in Design. The writer run uses 1031 training and 155 validation lines, eleven writers and 0.17 stroke width. Its output is `author_shape_11w_pw170_scratch_weights.npz`. Table 32 retains the displayed values. The excerpt ends with an interruption after epoch 47, rather than a completed 50-epoch run.

Table 32. Supplied numerical writer training log.

Epoch	Loss	Validation / %	Time / min
1	1.979	47.7	20.1
2	1.393	54.2	19.7
3	1.178	58.1	19.9

Epoch	Loss	Validation / %	Time / min
4	0.985	34.2	21.6
5	0.817	72.3	22.1
6	0.726	59.4	22.7
7	0.730	43.9	22.1
8	0.642	59.4	21.7
9	0.578	73.5	20.3
10	0.531	87.7	20.1
11	0.532	66.5	20.8
12	0.499	80.6	20.4
13	0.465	63.9	20.6
14	0.404	89.7	20.6
15	0.442	59.4	20.7
16	0.373	63.2	20.1
17	0.313	84.5	20.2
18	0.378	94.2	20.6
19	0.265	86.5	19.4
20	0.344	86.5	19.5
21	0.294	90.3	19.8
22	0.319	92.3	19.7
23	0.250	89.0	19.4
24	0.233	93.5	19.4
25	0.211	89.7	20.2
26	0.213	94.8	22.2
27	0.208	90.3	21.7
28	0.210	91.6	21.8
29	0.193	85.8	20.4
30	0.159	94.2	19.5
31	0.154	95.5	19.6
32	0.167	94.2	19.6
33	0.154	94.8	19.5
34	0.143	96.1	19.5
35	0.108	95.5	19.5
36	0.107	95.5	19.5
37	0.092	92.3	19.5
38	0.121	91.6	19.5
39	0.108	95.5	19.5
40	0.125	95.5	19.6
41	0.068	96.8	19.5
42	0.106	96.8	19.9
43	0.071	96.8	19.5
44	0.106	96.1	19.9
45	0.076	96.8	20.8
46	0.066	96.8	21.0
47	0.082	96.8	20.9

The reader excerpt resumes at epoch 7 after loading 90 of 90 tensors from its numerical checkpoint. Its data preparation reports 6481 public and 569 personal training lines, with 976 public and 100 personal held-out lines. Table 33 records the completed epochs shown. Epoch 17 is only a partial batch report. These excerpts describe training progress. The later saved-checkpoint accuracy in QT 2 comes from direct evaluation and is not inferred from an unfinished log.

Table 33. Supplied numerical reader training log.

Epoch	Train loss	Val loss	Val char / %	Personal / %	Time / min
7	45.339	44.488	70.17	66.56	122.3
8	24.865	18.597	87.62	88.54	120.1
9	18.473	40.667	73.19	75.48	118.6
10	15.332	17.133	88.89	90.07	118.7
11	12.862	13.298	91.43	91.40	111.6
12	11.047	13.509	91.34	92.00	111.3
13	9.818	15.953	89.91	92.16	111.7
14	8.549	13.378	91.65	92.65	111.6
15	7.642	14.057	91.44	92.46	111.5
16	7.045	11.663	92.89	93.09	112.3